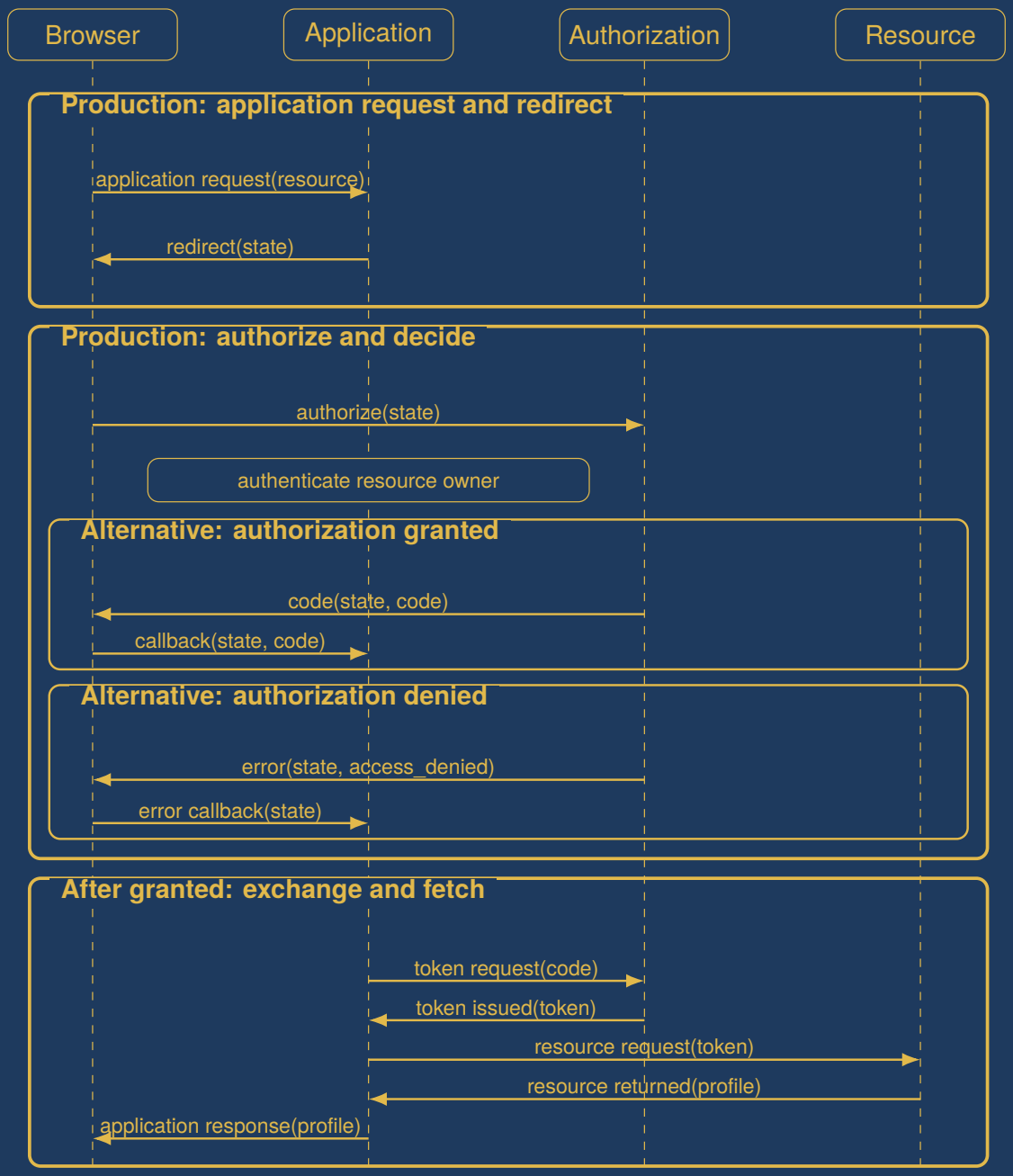
LeanFM Software Engineering



Rob Fielding and ChatGPT

Working draft—October 14, 2026

Preface

LeanFM is a software-engineering methodology for arguing requirements into existence with an LLM. Stakeholders challenge assumptions, add scenarios and constraints, and inspect generated diagrams, traces and measurements. The argument produces durable project artifacts rather than relying on chat memory.

The book is the main artifact. Every project begins with `./book/main.pdf`: its purpose, vocabulary, boundaries, examples and design argument. It acts as a maintained, project-specific section of the LLM's system-prompt context, whether supplied directly or discovered through the agent interface. Final diagrams come from the specification and are reused in the book, not maintained as separate designs.

The book is itself vibecoded: missing views, checks and editing tools are built as the argument needs them. Each project owns one directory containing its book, specifications, schemas, conversations, source, tests and generated artifacts. Copying that directory preserves the project; deleting it removes its local state. External deployments require explicit teardown.

This repository is also a LeanFM project. Its book, specifications and source are the inputs used to develop LeanFM itself. Other projects can be copied into their own repositories and opened in Codex without reconstructing the conversation.

`book/main.pdf` remains a central input in Codex; `book/main-dark.pdf` is its accessible rendering from the same source. Requirements and implementation specifications have stable project-relative paths. Each revision identifies one canonical implementation specification; deliberate alternatives name their targets.

The workflow has two acceptance gates. First accept `Requirements.lean`: observable behavior, legal conversations and properties. Then accept `Implementation.lean`: runtime, transport, actor, file and test mappings. These specifications refine the book. Together with the current repository, they are intended to support one complete, reviewable PR. "One-shot" describes this implementation pass; requirements discovery remains iterative. The integrated requirements-to-PR workflow is still under development.

This differs from software engineering organized primarily around coordinating people who debate separate drawings. LeanFM centers the shared, executable account of the design and records communication between actors down to the bytes represented by protobuf objects. Parameterized multiparty grammars describe legal message sequences; generated views let people reason about them; and Lean theorems check their consistency. Many important requirements, however, are not themselves observable messages. Security properties, referential integrity, and constraints between protobuf objects also belong in the theory. Protobuf has no standard foreign-key mechanism: it can encode an identifier and field presence, but it neither declares that the identifier resolves to another object nor enforces what must exist. Those relationships are predicates over decoded objects and reachable system states, stated and proved as Lean theorems in the requirements specification. The

implementation specification must then identify the storage, transaction, or validation mechanism that preserves each theorem.

For example, if A carries an identifier for B, every reachable A may be required to resolve to an existing B. If B in turn requires a C, while policy disallows every admissible C, Lean should expose that the predicates are jointly unsatisfiable: A cannot consistently exist. The design must change, perhaps by making the relationship to C optional or by changing the policy. Such contradictions are requirements discoveries even when no new wire event reveals them; they are not foreign-key declarations hidden in a `.proto` file.

Formal methods should therefore be the easy way to build software. This working book develops that claim into a practical method, while the LeanFM repository contains the evolving language, tooling, examples, and implementation [2].

Contents

Preface i

I The Grammar-Based Method 1

1 Formal Methods as the Default Path 2
1.1 Behaviour before machinery . . . 2
1.2 Comparison with TLA+ . . . 2
1.2.1 An executable queue model . . . 3
1.2.2 The cost of irrelevant state . . . 4
1.2.3 Proposed message-first notation . . . 4
1.2.4 Requirements at the message boundary . . . 5
1.3 Discovery first, consolidation before commitment . . . 6
1.4 Three ingredients . . . 7
1.5 A test for every abstraction . . . 7

2 Histories Made of Events 8
2.1 Compute everything from a fork/join event list . . . 9
2.1.1 Nested joins: choose a winner, then cancel the rest . . . 9
2.1.2 Durations from boundary events . . . 10
2.2 Derived state . . . 12
2.3 Time is more than a timestamp . . . 12
2.4 Round trip through bytes . . . 13

3 The Bakery Event Corpus 14
3.1 The grammar represented by events . . . 14
3.2 Money by day is the primary XY plot . . . 15
3.3 Waste explains part of the margin . . . 15
3.4 XY plots as questions over scenarios . . . 15
3.5 AI rendering from durable requirements to pseudocode . . . 17

4 A Multiparty Grammar of Behaviour 20
4.1 A small authoring algebra . . . 20
4.2 A BNF for composable interactions . . . 20
4.3 Parameters carry information . . . 23
4.4 Composition without omniscience . . . 23
4.5 Actors multiplex bounded conversations . . . 23

5 The Scenario Authoring Language 25
5.1 The smallest useful scenario . . . 25
5.1.1 Variables and assignment . . . 25
5.2 Vague messages first, typed alphabets later . . . 26
5.3 One concern per scenario . . . 26
5.4 Interaction diagram: accepted quote . . . 27
5.5 Choices and unresolved probability . . . 28
5.6 Worked MDP: a stochastic user and deterministic server . . . 28
5.7 Actor specifications and instances . . . 31
5.8 Non-functional requirements: rates and scalability . . . 31
5.8.1 The three USL regimes and capacity . . . 33
5.9 Outages, recovery, and replica populations . . . 36
5.9.1 Redundant, load-balanced runtimes and blast radius . . . 37
5.10 Infrastructure cost and demand-constrained supply . . . 42
5.11 Specified dwell time . . . 43
5.12 Estimating completion from an event stream . . . 43
5.13 Composing requirements means intersecting constraints . . . 44
5.14 State-machine rendering . . . 45
5.15 Charts are declarative trace reductions . . . 45

6 Arguing Requirements through the Lean Web Server 48
6.1 The argument loop . . . 48
6.2 The server is a staging environment . . . 49
6.3 Using the agent in Codex . . . 49
6.4 MCP: the agent-facing entrance . . . 50
6.5 One project, one directory . . . 52
6.6 What the Lean artifact must settle . . . 53
6.7 Interrogate, characterize, synthesize, replay . . . 54
6.8 The protobuf artifact is part of the requirement . . . 55
6.9 A ladder of checks . . . 55
6.10 How to argue well . . . 56

7 Worked Example: OAuth Authorization Code Flow 57
7.1 Actors, values, and observable events . . . 57
7.2 Production 1: application request and redirect . . . 58
7.3 Production 2: authorize, then grant or deny . . . 59
7.4 Scenario 3: exchange the code and use the token . . . 60
7.5 Reading the scenarios together . . . 60
7.6 The parallel protobuf alphabet . . . 61
7.7 Generate one legal trace and its artifacts . . . 63
7.7.1 Envelope stream and concrete bytes . . . 63
7.7.2 Event graph with stored backpointers . . . 65
7.7.3 Actor projections . . . 66
7.7.4 Event order, queues, and memory . . . 66
7.7.5 Measurements, plots, and verdicts . . . 66
7.8 Interaction diagrams by scenario . . . 67

7.9 Requirements as observable constraints . . . 67
7.9.1 Who knows the password? . . . 67
7.9.2 Byte-level refinement without choosing a transport . . . 69
7.10 Probabilities and charts . . . 69
7.11 Residual state machine . . . 70

8 CTL over Generated Futures **72**
8.1 Basic modal logic . . . 72
8.2 LTL: time along one path . . . 73
8.3 CTL: path modality combined with time . . . 73
8.3.1 Least upper bounds and greatest lower bounds . . . 74
8.4 Projecting to LTL-style F and G . . . 75
8.5 Persistence and recurrence . . . 76
8.6 Knowledge, obligation, and permission . . . 76
8.7 Possibility means preparedness . . . 78
8.8 Strong implication and the Chewbacca argument . . . 78
8.9 Weak until: AW and EW . . . 79
8.10 Past claims . . . 80
8.11 Fairness and time bounds . . . 80

9 Lean as the Rendered Language **81**
9.1 A Ramanujan-inspired checked discrete account of $1 + 2 + 3 + \cdots = -1/12$ . . . 81
9.2 From specification to service . . . 86

10 Scope and Open Questions **87**
10.1 Proof obligations . . . 87

II Security Protocol Exercises from the CSP Approach **88**

11 From CSP Roles to Multiparty Grammars **89**
11.1 Can actors really be specified separately? . . . 89
11.1.1 Local state without global implementation state . . . 90
11.2 A transport-neutral byte algebra . . . 90
11.3 Yahalom as three separate actors . . . 90
11.3.1 Alice, the initiator . . . 90
11.3.2 Bob, the responder . . . 91
11.3.3 The trusted server . . . 91
11.3.4 Global assembly and context conditions . . . 92
11.4 The attacker is another grammar plus deduction . . . 92
11.5 Knowledge and agreement become theorems . . . 93
11.6 Otway–Rees: guards are not optional decoration . . . 93
11.7 Needham–Schroeder: public-key lookup as behaviour . . . 94
11.8 What these exercises demand from the library . . . 94

III Machine Reference **96**

12 The LeanFM Web-Server Protocol **97**
12.1 One exchange across four representations . . . 97
12.2 The conversion, field by field . . . 98
12.3 The running adapter . . . 99
12.4 The accepted HTTP projection . . . 100
12.5 Complete protobuf payload vocabulary . . . 101
12.6 Complete LeanFM task grammar . . . 105
12.7 Authenticate an individual user . . . 105
12.8 Create an account using a user-supplied provider API key . . . 106
12.9 List, create, open, and delete owned projects . . . 106
12.10 Select user-funded or platform-funded model access . . . 106
12.11 Conduct one durable project-scoped requirements turn . . . 107
12.12 Sketch and promote multimodal requirement artifacts . . . 107
12.13 Inspect current, accepted, superseded, and stale artifacts . . . 107
12.14 Inspect one task's linked interaction, state machine, constraints, and protobuf events 108
12.15 Search the public web, follow supplied HTTP URLs, and return clickable citations . . 108
12.16 Recall diagram syntax or propose AI-generated detail for one markerboard revision . 109
12.17 What is checked, and what remains . . . 109

Referenced Books **110**

Bibliography **114**

Index **115**

Part I

The Grammar-Based Method

Chapter 1

Formal Methods as the Default Path

Formal methods are usually presented as an extra stage: first build a system, then translate a simplified version into a model, and finally ask a specialist tool a few questions. That workflow taxes correctness. The model drifts, the difficult translation is postponed, and proofs concern yesterday's architecture.

Our aim is the reverse. The shortest path to a running service should pass through an executable behavioural specification. Writing the specification should remove work: it supplies message types, scenario tests, model-checking input, documentation, and the skeleton of the service itself. Going without it should be the inconvenient choice.

1.1 Behaviour before machinery

The primary object is a set of permitted, visible histories. An implementation may use threads, actors, queues, databases, caches, or continuations. Those decisions matter operationally, but they are not automatically part of the contract. Promoting all internal state into the model creates a product state space whose size grows exponentially with independently varying components. Worse, it makes the specification brittle under harmless refactoring.

Protocols and persistent disk formats tend to live forever. Once another process, customer, or stored data set depends on their bytes and ordering, they become compatibility boundaries that must survive many implementations. The internal structure of software is more chaotic: data structures, control flow, threads, caches, and retry machinery are routinely replaced while the external contract remains. Modeling every such choice would couple the specification to short-lived machinery and introduce many more independently varying states to explore. Concentrating first on the durable external boundary therefore both protects what is hardest to change and keeps the tractable state space focused on behavior that implementations must continue to preserve.

We therefore begin with *behaviour*: who communicated with whom, what kind of event occurred, what information it carried, and which predecessor order it extends. Internal state enters only when it changes an observable promise.

1.2 Comparison with TLA+

LeanFM is not an extension of TLA+ and does not try to replace it. TLA+ commonly describes a system with variables, an initial-state predicate, a next-state relation, and temporal formulas over the

resulting state behaviors [6]. A message queue can be represented as a sequence- valued variable, and an action can append to or remove from that sequence. This is expressive and makes internal state transitions available to invariants and model checking.

1.2.1 An executable queue model

This TLA+ module sends an order to Bob's inbox, then lets Bob consume it and change his control state:

```
---------------------------- MODULE OrderQueue ----------------------------
EXTENDS Integers, Sequences, FiniteSets, TLC
CONSTANTS Clients, Servers, Orders, QueueBound, CacheBits, ClockBound
Actors == Clients \cup Servers
Messages == [kind : {"placeOrder", "placedOrder"},
             order : Orders, replyTo : Clients]
NoMessage == [kind |-> "none", order |-> "none", replyTo |-> "none"]
VARIABLES inbox, pc, current, sent, placed, received, scratch, tick
vars == <<inbox, pc, current, sent, placed, received, scratch, tick>>

Init == /\ inbox = [a \in Actors |-> <<>>]
        /\ pc = [b \in Servers |-> "idle"]
        /\ current = [b \in Servers |-> NoMessage]
        /\ sent = {} /\ placed = {} /\ received = {}
        /\ scratch = [i \in 1..CacheBits |-> FALSE]
        /\ tick = 0

Send(a, b, o) ==
  /\ o \notin sent /\ Len(inbox[b]) < QueueBound
  /\ inbox' = [inbox EXCEPT ![b] = Append(@,
       [kind |-> "placeOrder", order |-> o, replyTo |-> a])]
  /\ sent' = sent \cup {o}
  /\ UNCHANGED <<pc, current, placed, received, scratch, tick>>

Receive(b) ==
  /\ pc[b] = "idle" /\ Len(inbox[b]) > 0
  /\ current' = [current EXCEPT ![b] = Head(inbox[b])]
  /\ inbox' = [inbox EXCEPT ![b] = Tail(@)]
  /\ pc' = [pc EXCEPT ![b] = "busy"]
  /\ UNCHANGED <<sent, placed, received, scratch, tick>>
```

The excerpt above defines actions but does not yet connect them to a behavior. That connection appears later in the same module:

```
Next == (\E a \in Clients, b \in Servers, o \in Orders: Send(a,b,o))
     \/ (\E b \in Servers: Receive(b) \/ Finish(b))
     \/ (\E a \in Clients: ReadReply(a)) \/ CacheStep \/ ClockStep
Spec == Init /\ [][Next]_vars
```

TLA+ does not call `Send` or `Receive` like procedures. `Next` is a relation between the current variables and their primed next values. Its disjunction says that one step may be a `Send` for some client, server and order; a `Receive` or `Finish` for some server; a `ReadReply` for some client; a cache step; or a clock step. A candidate action whose guards are false is disabled. TLC explores every enabled choice.

Read `Spec == Init /\ [][Next]_vars` as: "the first state satisfies `Init`; thereafter, every

state transition satisfies `Next` or leaves every variable in `vars` unchanged." The configuration closes the final loop with `SPECIFICATION Spec`; it tells TLC which temporal formula to explore.

Read `inbox'` as "the next value of inbox," `@` as "the old value at this selected entry," and `UNCHANGED` as "these variables retain their values." `Append` adds a message; `Head` selects the first message; `Tail` removes it. Alice's send updates Bob's inbox, not Bob's private control state. Bob's receive performs the latter transition. The configuration instantiates `Clients = {"alice"}` and `Servers = {"bob"}`; functions indexed by those sets hold instance state. The full module commits the order, sends a reply, and checks that no reply precedes placement. No eventual-reply property is checked.

1.2.2 The cost of irrelevant state

The module includes optional cache-bit toggles and a clock increment that cannot affect an order message. TLC nevertheless distinguishes their values:

Model	Cache bits	Clock bound	Distinct states
Baseline	0	0	26
Cache4	4	0	416
Cache8	8	0	6,656
Clock7	0	7	208

These exhaustive finite-model safety-check counts come from TLC 1.7.4 via `make tla-check`; evidence and provenance are in `book/generated/tla-state-space.json`. With S protocol states, b independent Boolean cache bits and a counter ranging from zero through k, the example has

$$S\,2^b(k+1)$$

states. Read: "S times two to the b times k plus one." S is the protocol state count, b the cache-bit count and k the counter bound. Each irrelevant bit doubles this model's state count. `ClockBound = -1` allows ticks 0, 1, 2 and onward: infinitely many states with the same order behavior. Exhaustive explicit-state search cannot finish that variant. Unbounded queues or retained histories can also make a model infinite.

This is checker state growth, not measured application memory. More states usually require more exploration time and storage; the table gives no fixed bytes-per-state or timing law. TLA+ supports abstraction, hiding and refinement. The lesson is to omit irrelevant state, not to blame the language for its inclusion.

1.2.3 Proposed message-first notation

This is invented TLA+-inspired syntax, not standard TLA+ or a compiled LeanFM feature:

```
// Proposed TLA+-inspired notation. Not standard TLA+, not compiled by LeanFM.
message placeOrder(orderId: OrderId, replyTo: ClientId)
message placedOrder(orderId: OrderId)

actor class Client {
  input placedOrder
  output placeOrder
}
actor class Shop {
```

```
  input placeOrder
  output placedOrder
  inbox FIFO(capacity = 2)
  on placeOrder(orderId, replyTo) {
    require commitRecorded(orderId)
    self:replyTo ! placedOrder(orderId)
  }
}

alice = new Client()
bob = new Shop()
alice:bob ! placeOrder(o1, replyTo = alice)
bob ? placeOrder(o1, replyTo = alice)
// Shop handler records the commit and sends placedOrder.
alice ? placedOrder(o1)

web = instances(WebServer, count = 3, route = roundRobin)
documents = shards(DocumentStore, count = 8,
                   key = documentPartition, replicas = 3,
                   writeQuorum = 2)
```

Read `alice:bob ! placeOrder` as "Alice sends placeOrder to Bob," and `bob ? placeOrder` as "Bob receives placeOrder." Bob consumes the request `placeOrder`; Alice consumes the reply `placedOrder`. A send changes the destination's inbox, a first-class queue owned by its actor instance, not an arbitrary private variable. Receive consumes the message; the receiver updates its own protocol state. Full queues block send without mutation; empty queues block receive. The handler's commit precondition is an observable obligation, not a database algorithm.

Actor classes describe behavior; instances have distinct identities, queues and local state. WebServer instances share a class, not an inbox. DocumentStore shards partition ownership by key; replicas protect each shard, and a quorum controls write acknowledgment. These deployment roles are placeholders, not classes implemented by the order module. First-class syntax still needs capacity, routing, failure and scheduling rules; it does not eliminate their state space.

1.2.4 Requirements at the message boundary

LeanFM chooses a different primary boundary. The object available to us is often an event stream scraped from a running system. We can decode its messages, correlate them, follow predecessor backpointers, and classify the resulting sequence as in-spec or out-of-spec. We generally cannot observe the implementation's private variables at every step, so `Requirements.lean` does not pretend to maintain them. Its grammar is intended to recognize legal traces and generate them; state machines are projections of visible histories rather than declarations of every variable stored by the program.

The distinction is methodological, not a claim that TLA+ is unable to express queues or traces. This project began with an attempt to extend a TLA+-style model with the desired message queues and MDP probabilities. The practical difficulty was upstream: explaining an ordinary real scenario required choosing and maintaining a small internal-state model before the observed behavior could even be recorded. The abstraction work could exceed the apparently trivial example. LeanFM instead asks first for the messages that would be visible in production.

The trade is deliberate:

Question	TLA+-style state specification	LeanFM event-language specification
Primary behavior	sequence of states	sequence or partial-order graph of events
Queue representation	usually an explicit variable	observable enqueue, send, receive, blocking, and capacity facts
Conformance input	modeled states or action correspondence	decoded production event stream
Hidden variables	available when declared	omitted unless they affect an observable contract
Generation	behaviors satisfying the state relation	words/traces generated by the grammar
Probabilities	require an additional probabilistic interpretation or tool	weights and observed distributions are first-class model data
Main risk	state model may drift from instrumentation	event model may omit a hidden cause needed for a property

This boundary also separates the two durable Lean files. `Requirements.lean` contains specified observable behavior: accepted messages, legal ordering, probabilities, timing and reliability constraints, and properties measurable from events. `Implementation.lean` contains the additional decisions needed to generate software: target language, files, transports, runtime APIs, and bindings. Existing source code may be supplied when revising an implementation, but it does not redefine the requirement.

The separation is designed for loss of conversational context. The original LLM conversation is expected to disappear repeatedly. After each argument, the current understanding must survive in the requirements, protobuf, and implementation files, with requirement-to-code justifications. A new model session reads those files and continues from them; it need not reconstruct intent from a transcript.

1.3 Discovery first, consolidation before commitment

The working book is deliberately allowed to begin as a vibecoded stream of consciousness. Stakeholders can introduce examples, possible abstractions, warnings, abandoned alternatives, and future work without pretending that every sentence is already an accepted requirement. That freedom is useful during discovery, but an unedited transcript is not a finished design.

LeanFM therefore has a distinct cleanup phase before acceptance and again before one-shot implementation. Each normative idea is classified as an observable requirement, an implementation decision, deferred work, or material to remove. The edit preserves definitions, constraints, formula readings, and necessary qualifications while removing repetition, throat-clearing, and defensive restatement. An idea mentioned in the book does not become an implementation obligation merely by surviving an early draft; the accepted Lean artifacts say which ideas bind the software.

Wordy prose is also evidence that a missing view may already be latent in the requirements. Paragraphs that mainly enumerate actors, messages, state changes, ordering, dependency gates, or measured quantities should normally collapse into a generated interaction diagram, state machine,

order graph, dependency view, or chart followed by a short interpretation. The view must be reduced from the current typed requirements and retain its prompt, question, event source, reducer, units, and boundary. Drawing a persuasive picture independently would only create another specification that can drift.

1.4 Three ingredients

The method combines three traditions.

1. Language theory gives compact generative descriptions of legal histories. Regular and context-free grammars provide the approachable starting point. Dana Richards and Henry Hamburger, the author's college professors, are cited in Richards and Hamburger [11] as the source from which the author learned these methods, not as their originators. For the underlying habits of symbolization, validity, proof, quantification, and modal reasoning, we also draw on the accessible progression of Gensler [3].

2. Protocol analysis contributes explicit participants, adversarial choices, trace semantics, and tool-supported checking. The CSP treatment of Ryan et al. [12], including Steve Schneider, is cited as the source from which the author learned this treatment.

3. Temporal logic turns expectations into queries over the generated transition system. CTL is central because it distinguishes claims about some possible future from claims about every possible future [1].

1.5 A test for every abstraction

Each abstraction in this book should answer a practical question:

> Can a developer state a meaningful behaviour once, compose it with other behaviours, check its consequences, and keep it stable while implementation details change?

If an abstraction does not improve that workflow, it does not belong in the core library.

Chapter 2

Histories Made of Events

Every execution is represented by an event sequence. A deliberately small common envelope makes histories comparable across protocols:

```
structure Event (Actor Session Task Kind Payload : Type) where
  timeAt  : ClockTimestamp
  id      : String
  priorIds : List String
  session : Session
  task    : Task
  from    : Actor
  to      : Actor
  kind    : Kind
  payload : Payload

abbrev Trace (Actor Session Task Kind Payload : Type) :=
  List (Event Actor Session Task Kind Payload)
```

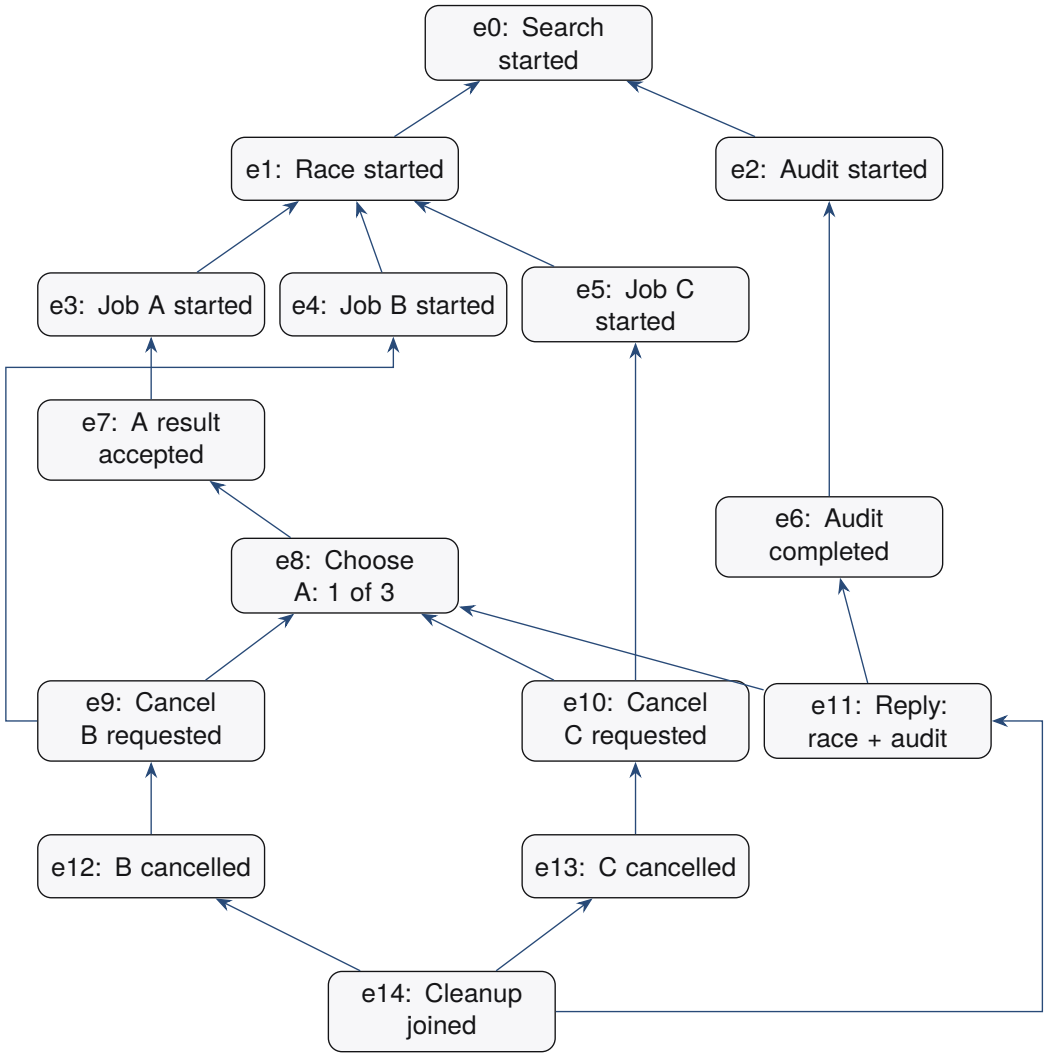
The fields have different jobs. `id` identifies a visible fact; `priorIds` records zero or more immediate predecessor backpointers; `from` and `to` name the parties; `(session, task)` correlates independently progressing conversations; and `kind` supplies a stable discriminator. The payload is protocol-specific and carries the parameters that flow from one production to the next. `timeAt` is sampled from the model's shared monotonic clock. Its differences measure elapsed time; it is not a civil-time date and its origin is immaterial.

The predecessor links form a directed acyclic graph rather than a singly linked list. An initial event has no prior identifiers. A fork is represented by several later events that each name the same prior event. A join is represented by one event whose `priorIds` names every branch on which it depends. An end event of any kind may also name its corresponding start event. That backpointer makes elapsed time simply `end.timeAt - start.timeAt`, even when several attempts with the same `(session, task)` overlap. A backpointer to an already-ancestral start is a redundant ordering edge, but it is useful correlation data and does not alter the partial order. The edge says "ordered after," not "caused by." A causal interpretation requires a separate grammar or domain assertion.

For an m-of-n join, the event additionally records m, n, and the exact m branch-terminal IDs that satisfied the threshold. The selected IDs must be distinct members of `priorIds`. Merely seeing two predecessors is not enough to distinguish a two-of-three threshold from an ordinary two-branch all-of join.

2.1 Compute everything from a fork/join event list

2.1.1 Nested joins: choose a winner, then cancel the rest



Arrows show predecessor order with redundant transitive edges removed; full backpointers remain in the event list. The event list and provenance are generated from `NestedForkJoinExample.lean`; this is not a claim about Google's implementation.

The winner event records threshold 1, population 3 and selected terminal `e7`. Its predecessors are `[e1,e7]`, not the results of all three jobs. The outer reply records `[e0,e8,e6]`: task start, chosen result and completed audit. Reply time is 12 synthetic milliseconds; cleanup ends at 18. These times illustrate ordering, not measured search performance.

For a general n-of-m join, choose n distinct qualifying terminals from m declared branches. Selection is atomic, so racing results cannot select two winners for a 1-of-m task. "First" means the coordinator's observed order, with a declared tie rule, not unsynchronized worker clocks. A cancellation request is not a cancellation acknowledgment; late results must not replace the chosen result. The requirement must say whether a reply waits for losing-task cleanup. Here it does not.

Selected-ID and threshold metadata belongs in the join event, not inferred from the picture's number of incoming arrows.

2.1.2 Durations from boundary events

Each event is one observation at one instant. Starting and completing are different messages, so an event does not contain an interval. Durations are derived by selecting the two boundary events and subtracting their timestamps.

```
structure AnalysisEvent where
  id       : EventId
  priorIds : List EventId
  timeAt   : ClockTimestamp
  bytes    : ByteArray
  reveals  : List ValueRef

def elapsed (started completed : AnalysisEvent) : Duration :=
  completed.timeAt - started.timeAt
```

Consider the first retail purchase in `examples/bakery-events.jsonl`. Its bread was baked and shipped before the storefront opened. The purchase itself reserves that inventory, tenders payment, and completes in about a second:

Event	Prior backpointers	`timeAt`	Meaning	Bytes	Reveals
e_0	$[]$	28859991	purchase requested	59	product and quantity
e_1	$[e_0]$	28860343	inventory reserved	62	batch allocation
e_2	$[e_1]$	28861004	payment tendered	55	amount
e_3	$[e_1, e_2]$	28861248	sale completed	72	bread and payment

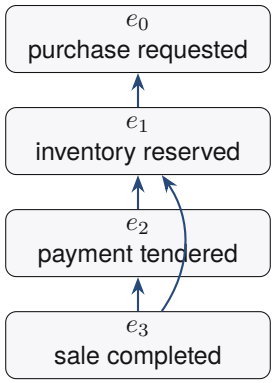


Figure 2.1: The table's immediate predecessor backpointers rendered as an order graph. Each arrow points from an event to an ID in its `priorIds` list: for example, $e_2 \to e_1$. The extra $e_3 \to e_1$ edge preserves direct correlation with the inventory allocation even though e_1 is already an ancestor through e_2.

The list order is one serialization convenient for storage. Event order is the transitive closure of `priorIds`. This sale is sequential; baking and shipping belong to the morning production stream, not to the customer purchase. The following quantities are folds or graph queries over this one list:

$$\begin{aligned}
\text{reservationTime}(e_0, e_1) &= 28860343 - 28859991 = 352 \text{ ms},\\
\text{paymentTime}(e_1, e_2) &= 28861004 - 28860343 = 661 \text{ ms},\\
\text{purchaseLatency}(e_0, e_3) &= 28861248 - 28859991 = 1257 \text{ ms},\\
\text{wireBytes} &= 59 + 62 + 55 + 72 = 248.
\end{aligned}$$

Read these as follows. Inventory reservation takes 352 milliseconds; payment takes 661 milliseconds; and the complete storefront exchange takes 1257 milliseconds. Wire bytes is the sum of the four listed event sizes, or 248. Each subscript identifies the exact event used as a boundary.

Two events are concurrent in this order when neither is an ancestor of the other:

```
def concurrent (events : List AnalysisEvent) (a b : EventId) : Bool :=
  !reachable events a b && !reachable events b a
```

The maximum concurrency permitted by the trace is the width of this partial order: the size of its largest antichain. This purchase is a chain, so its width is one. Other purchases and the morning production and transport streams may still be concurrent. Incomparable events may commute in a linear serialization, provided each event still appears after all of its named predecessors. A timestamped point event has no active interval; runtime concurrency requires separate start and completion messages. This is stronger than counting adjacent list elements and different from measuring average runtime concurrency.

Start and completion boundaries make nested concurrent work especially readable as an order graph. In the following trace, the request forks into two task attempts. Each completion points back to its own start, and the response joins the two completions. The shaded columns are visual grouping only; the arrows are still exactly the stored predecessor pointers.

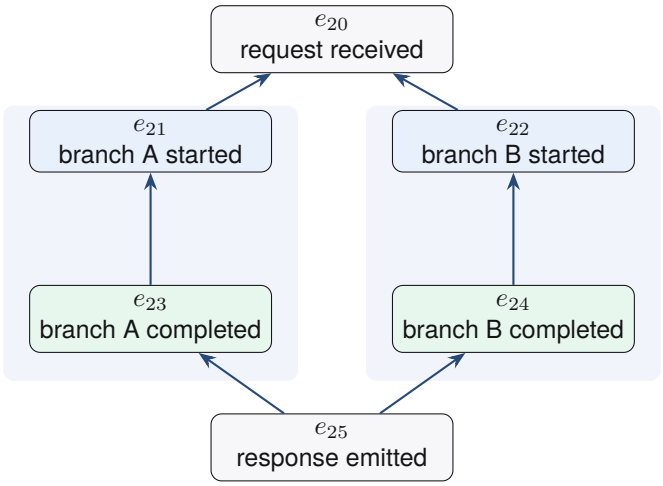


Figure 2.2: A separate synthetic example of task boundaries nested inside a concurrent fork/join. The intervals $e_{21} \ldots e_{23}$ and $e_{22} \ldots e_{24}$ may overlap, while the two branches remain incomparable in predecessor order. Either branch may appear first in a linear serialization without changing the graph.

Timestamps and predecessor links also give elapsed time along a selected path:

```
edgeTime(prior, event) := event.timeAt - prior.timeAt
elapsed(started, completed) := completed.timeAt - started.timeAt
```

The retail critical path is e_0, e_1, e_2, e_3, with elapsed time 1257 ms. The direct $e_3 \to e_1$ correlation edge is redundant for reachability and does not change that duration. Queue occupancy, queued bytes, active `(session,task)` keys, and memory pressure are computed by adding enqueue/dequeue events to the same alphabet and folding them in clock order.

The entire checked-in corpus contains 20,000 orders and 81,039 events. Streaming that file produces the following model inputs; the numbers in this table are generated LaTeX macros, not independently typed constants:

Reduction over `bakery-events.jsonl`	Observed value
sales completed / purchase requests	20,000/20,000
stockouts / purchase requests	0/20,000
morning batches baked	140
FIFO shipments dequeued / enqueued	140/140
maximum simultaneously active purchases	1

These ratios are suitable probability estimates for this corpus. Regeneration with a different source stream changes the macros and therefore changes every book example that consumes them.

Knowledge is another event reduction. For a value v, calculate the actors that can derive it from initial knowledge and the bytes visible to them:

```
def knowers (actors : List Actor) (v : ValueRef)
    (events : List AnalysisEvent) : List Actor :=
  actors.filter fun actor =>
    Derives (knowledgeFrom actor events) v
```

Encryption, signing, projection, key possession, and compromise contribute rules to `Derives`. The primary result is the set `knowers v events`. "Only these actors know v" is a comparison between that computed set and an allowed set, not a separate primitive called secrecy.

2.2 Derived state

A current session, an authenticated principal, or an outstanding request can often be computed by folding the trace. Such a fact need not also exist as primitive model state. This discipline has two benefits: evidence for a fact remains visible, and the state representation does not multiply independently of the history that justifies it.

Some implementations necessarily maintain private state. The model includes it only when two executions with the same visible trace may make different future behaviour legal. Even then, the first remedy is to expose the relevant distinction as an event or a grammar parameter rather than copying an implementation object graph into the specification.

2.3 Time is more than a timestamp

The central modelling problem is maintainable time evolution. We separate three ideas that are often conflated:

- sequence order: one visible event occurs before another;
- declared dependency or causality: a grammar says that one event extends or responds to another;
- duration: an amount of physical or logical time elapses.

Grammar structure declares legal sequence and domain dependency, `priorIds` encodes the resulting predecessor order, and typed constraints handle duration. A predecessor link alone is not evidence of causation. This keeps a change in timeout policy from rewriting the protocol's entire behavioural shape.

For two events from the same `(session, task)`, elapsed time is defined when the first event's `timeAt` value does not exceed the second's:

```
def elapsed (start finish : Event) : Option Duration :=
  if start.session = finish.session &&
     start.task = finish.task &&
     start.timeAt <= finish.timeAt
  then some (finish.timeAt - start.timeAt)
  else none
```

Latency queries name their boundary event kinds and follow the end event's backpointer to the start before subtracting timestamps. The `(session, task)` values must agree, but are not sufficient to choose a start when attempts overlap. Ordering comes from `priorIds`; any causal meaning comes from the grammar or another explicit domain relation. Equal timestamps may occur, and neither a timestamp nor a predecessor pointer alone establishes that one event caused another.

2.4 Round trip through bytes

A generated scenario is not complete until it can cross the byte boundary in both directions. `ScenarioEvent` in `Requirements.proto` carries the event identity, complete `prior` list, correlation key, actors, `timeAt`, and the selected protobuf atom. Encoding a `Scenario` therefore produces bytes for both its ordering envelope and payload. Decoding those bytes must reconstruct the same ordered event list. The executable round-trip check compares the decoded protobuf value with the generated value and separately checks the reconstructed `prior` graph; loss of a fork or join fails the build.

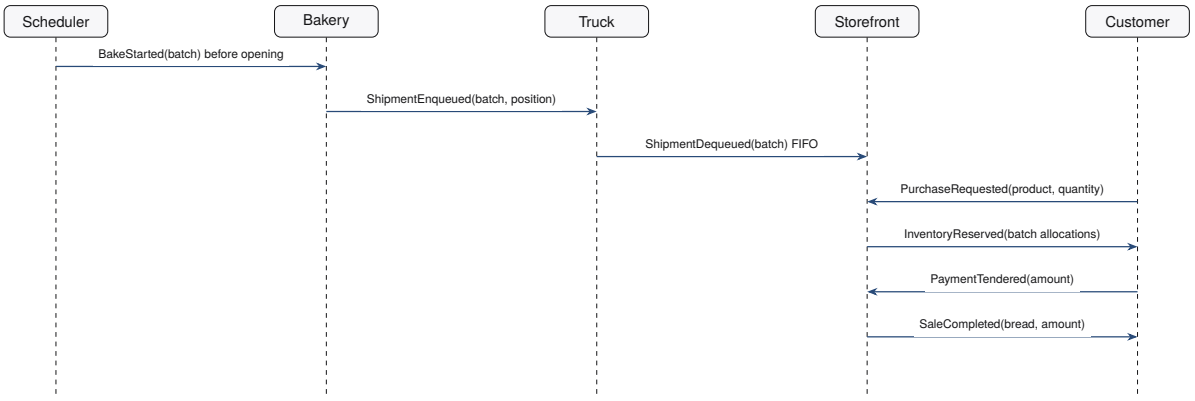
Chapter 3

The Bakery Event Corpus

The bakery is the quantitative worked example. Its source is not a table of percentages copied into this chapter. It is the 81,039-line `examples/bakery-events.jsonl` stream. The reducer reads those events and generates both `examples/bakery-stats.json` and the coordinates plotted below. Changing the stream changes the book.

3.1 The grammar represented by events

Bread is not baked in response to an individual order. A predictable schedule creates batches by bread kind; each batch is baked, shipped before opening, and stocked at the storefront. Customer orders reserve fresh inventory from those lots. After the declared two- or three-day freshness window, remaining units move to a charity rack and produce a donation receipt. An `InventoryReserved` event names both the paid order and the stocked batch lots that supplied it, making FIFO allocation, age, quality, and stockouts replayable. No event permits a customer to wait for a same-day custom loaf.



The truck is a bounded FIFO queue. `ShipmentEnqueued` records its queue position and capacity; `ShipmentDequeued` backpoints to both the matching enqueue and the preceding dequeue. The retail interaction begins only after `ShipmentDequeued` has added the batch to storefront inventory.

For each calendar-day bucket d, the reducer computes

$$
\begin{aligned}
\text{grossSales}_d &= \sum_{e.kind=\text{SaleCompleted}} e.amount, \\
\text{refunds}_d &= \sum_{e} e.refund, \\
\text{netRevenue}_d &= \text{grossSales}_d - \text{refunds}_d, \\
\text{ingredientCost}_d &= \sum_{e.kind=\text{BatchBaked}} e.ingredientCost, \\
\text{donationValue}_d &= \sum_{e.kind=\text{DonationReceiptRecorded}} e.eligibleValue, \\
\text{employeePay}_d &= \sum_{e.kind=\text{ShiftPaid}} e.pay, \\
\text{profit}_d &= \text{netRevenue}_d - \text{ingredientCost}_d - \text{employeePay}_d \,.
\end{aligned}
$$

Read every line at day d. Gross sales is the sum of amount on events e whose kind is SaleCompleted; refunds is the sum of each event's refund; net revenue is gross sales minus refunds; ingredient cost is the sum of ingredient cost on BatchBaked events; donation value is the sum of eligible value on DonationReceiptRecorded events; employee pay is the sum of pay on ShiftPaid events; and profit is net revenue minus ingredient cost minus employee pay. The subscript d means that every sum and result is restricted to the same calendar-day bucket.

Donated bread is already part of batch ingredient cost, so subtracting it again would double count. Donation value is not revenue and is not itself a tax saving. The tax effect is calculated only after the applicable rules and rate are supplied. This modeled profit excludes other costs for which the stream has no event.

3.2 Money by day is the primary XY plot

The employee-pay line is not a constant assumption. Each `ShiftPaid` event carries employee, role, minutes, hourly rate, and calculated pay. This lets the same chart answer counterfactual questions after a staffing event stream is changed.

3.3 Waste explains part of the margin

Cost and units are deliberately separate. A donated sourdough loaf and a donated croissant do not have the same economic effect. Grouping the same events by product, batch age, employee shift, or day produces further XY datasets without changing the scenario.

3.4 XY plots as questions over scenarios

The XY chart is not restricted to time on the horizontal axis. The reducer also generates daily order count against daily profit:

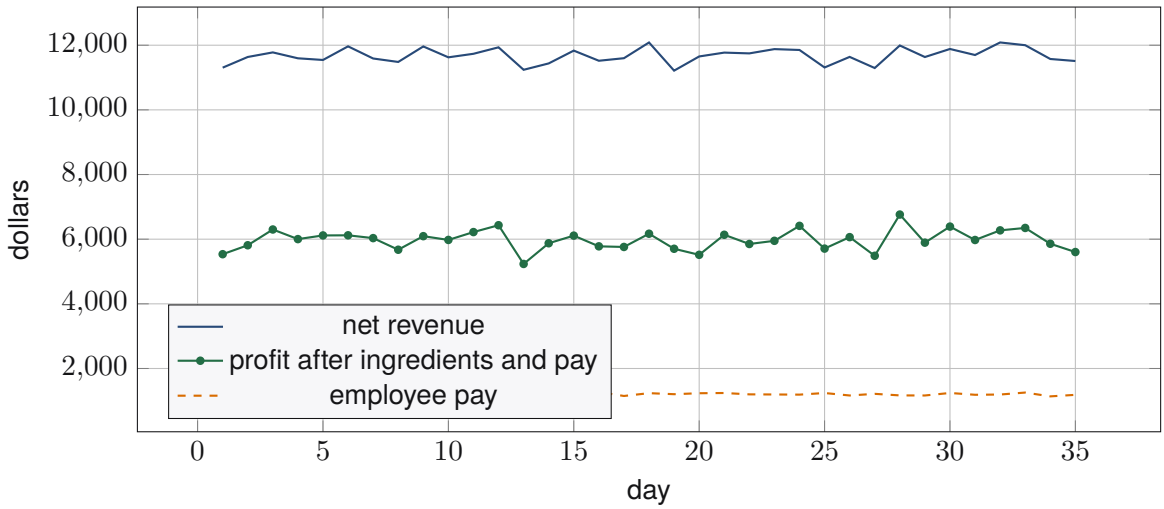


Figure 3.1: Daily bakery economics reduced from morning production, retail sales, payroll and charity-transfer events.

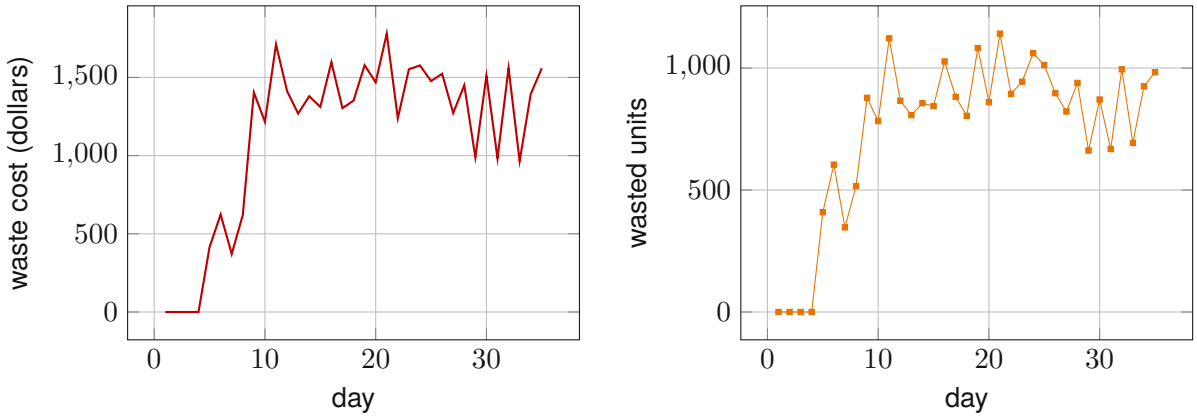


Figure 3.2: Donation-eligible cost and units by day, derived from batch inventory remaining when its freshness window closes.



Figure 3.3: Daily order volume against profit. Each point is one day reduced from the event corpus.

The same mechanism can measure many scenarios: offered load against latency, staffing hours against profit, price against cancellation probability, queue capacity against completed orders, or waste against bake batch size. In each case the axes are named reductions over event streams. A parameter sweep produces several streams; observed operation supplies streams from different days. The chart renderer need not know which kind it received.

The old made-to-order order-lifetime statistic is intentionally absent. Useful service metrics are payment and inventory-reservation time, storefront fulfillment time, stockout rate, inventory age at sale, freshness-window sell-through, and units and eligible value transferred to charity.

3.5 AI rendering from durable requirements to pseudocode

The grammar is useful for more than recognizing a saved corpus. It generates legal traces, so the same durable specification can drive software generation. The inputs are files, not the conversation that produced them. Let R be the complete current `Requirements.lean`, I the complete current `Implementation.lean`, P the protobuf schema, and C_k an optional existing implementation. Code generation has two forms:

$$(R, I, P) \longrightarrow C_0, \qquad (R, I, P, C_k) \longrightarrow C_{k+1}.$$

Read the first mapping as: “requirements R, implementation specification I, and protobuf schema P generate initial code C_0.” Read the second as: “those same three specifications plus current code revision C_k generate the next revision C_{k+1}.” Subscript zero is the initial revision; subscripts k and $k + 1$ are the current and next revisions.

The first creates software from scratch. The second revises existing software against the same stable inputs, retaining conforming code and replacing code that conflicts. Prior chat messages are deliberately absent from both functions. If a necessary decision is not present in R, I, or P, the generator reports the missing durable fact instead of recovering it from a long conversation.

For the bakery, the selected implementation backend is Go. The generated code uses Go protobuf bindings, deterministic `proto.Marshal`, `proto.Unmarshal`, and bounded channels. Requirement

comments are emitted beside the symbols and statements they justify:

```
// Code generated from the bakery specification. DO NOT EDIT.
package bakery

import "context"

// A Lot is bread already baked for sale. Production is scheduled before the
// storefront opens; it is not started by a customer purchase.
type Lot struct { BatchID, Product string; Units uint32 }

// Shipment is a batch waiting in a bounded truck FIFO.
type Shipment struct { EnqueueEventID string; Lot Lot }
type Truck struct { ID string; Queue chan Shipment }

func NewTruck(id string, capacity int) *Truck {
        return &Truck{ID: id, Queue: make(chan Shipment, capacity)}
}

// Enqueue blocks when the truck queue is full. ShipmentEnqueued records its
// batch, truck, and queue position.
func (t *Truck) Enqueue(ctx context.Context, shipment Shipment) error {
        select {
        case t.Queue <- shipment:
                return emit("ShipmentEnqueued", shipment.EnqueueEventID)
        case <-ctx.Done(): return ctx.Err()
        }
}

// Deliver is the sole FIFO dequeue path. ShipmentDequeued points to its enqueue
// and the preceding dequeue, making transport order observable.
func (t *Truck) Deliver(ctx context.Context, store *Storefront) error {
        var previousDequeue string
        for {
                select {
                case shipment := <-t.Queue:
                        dequeueID := nextID("shipment-dequeued")
                        if err := emitWithPrior("ShipmentDequeued", dequeueID,
                                shipment.EnqueueEventID, previousDequeue); err != nil { return err }
                        store.Receive(shipment.Lot)
                        previousDequeue = dequeueID
                case <-ctx.Done(): return ctx.Err()
                }
        }
}

type Storefront struct { Inventory map[string][]Lot } // FIFO lots by product.
func NewStorefront() *Storefront { return &Storefront{Inventory: make(map[string][]Lot)} }
func (s *Storefront) Receive(lot Lot) { s.Inventory[lot.Product] = append(s.Inventory[lot.Product], lot)
    }

// Purchase exchanges pre-made storefront inventory for money. Its messages are
// PurchaseRequested, InventoryReserved, PaymentTendered, and SaleCompleted.
// There is no per-order bake and no courier delivery.
func (s *Storefront) Purchase(ctx context.Context, product string, units uint32,
        priceCents uint64) error {
        requestID := nextID("purchase-requested")
        if err := emit("PurchaseRequested", requestID); err != nil { return err }
        allocation, ok := reserveFIFO(s.Inventory[product], units)
        if !ok { return emitWithPrior("StockoutReported", nextID("stockout"), requestID) }
        reservedID := nextID("inventory-reserved")
        if err := emitWithPrior("InventoryReserved", reservedID, requestID); err != nil { return err }
        paymentID := nextID("payment-tendered")
        if err := tender(ctx, priceCents); err != nil { return err }
        if err := emitWithPrior("PaymentTendered", paymentID, reservedID); err != nil { return err }
        commitAllocation(s.Inventory, product, allocation)
```

```
        return emitWithPrior("SaleCompleted", nextID("sale-completed"), reservedID, paymentID)
}

// Declarations standing for generated protobuf/event plumbing.
type Allocation struct{}
func emit(kind, id string) error
func emitWithPrior(kind, id string, prior ...string) error
func nextID(prefix string) string
func reserveFIFO(lots []Lot, units uint32) (Allocation, bool)
func commitAllocation(inventory map[string][]Lot, product string, allocation Allocation)
func tender(ctx context.Context, cents uint64) error
```

The pseudocode does not become a second source of requirements. It is discarded and regenerated when R or I changes. Generated source code is likewise a projection, except that revision mode accepts the current code as an additional input so hand-written conforming sections need not be destroyed. In both modes, the grammar supplies positive tests by generating legal traces and negative tests by rejecting mutations outside its language. Byte round trips then check that the implementation and its transport bindings preserve the observable scenario.

Every generated code unit carries a machine-readable requirement reference and a short justification beside the code it explains. The reference points into the durable requirement namespace, not to a chat turn. Generation also emits a traceability manifest mapping requirement IDs to source files, symbols, and tests. An implementation mapping without a requirement reference is an orphan and fails implementation-plan validation; generated-source validation must apply the same rule to emitted symbols. Several code units may cite one requirement, and one code unit may cite several requirements; the mapping is many-to-many.

Chapter 4

A Multiparty Grammar of Behaviour

A protocol is a language whose terminal symbols are visible events. A production describes a legal continuation. The context-free grammar is the visual and conceptual starting point, not a claim that every useful protocol language is context-free.

4.1 A small authoring algebra

```
inductive Grammar (Event : Type) where
  | empty
  | event  : Event -> Grammar Event
  | seq    : Grammar Event -> Grammar Event -> Grammar Event
  | choice : List (Grammar Event) -> Grammar Event
  | par    : Grammar Event -> Grammar Event -> Grammar Event
  | where_ : (List Event -> Bool) -> Grammar Event -> Grammar Event
```

`seq` preserves time order. `choice` exposes alternatives rather than burying them in control flow. `par` admits interleavings while preserving the internal order of each side. A `where_` clause checks a fact accumulated from the prior trace and provides the escape from context-free structure when genuine context sensitivity is needed.

A choice does not need a separate annotation naming who makes it. Each branch eventually resolves to terminal events, and each terminal resolves to framed bytes with a source and destination. The first observed terminal that distinguishes two branches identifies both the selected branch and the actor that emitted the distinguishing bytes. If two branches resolve to the same observable bytes, then at this abstraction they are not distinct choices and should be merged. This removes an otherwise ambiguous second account of agency from the grammar.

Simple finite conversations can compile to regular languages. Nested request structure motivates context-free productions. Correlation, freshness, uniqueness, and authorization typically require parameter flow or guarded productions.

4.2 A BNF for composable interactions

The authoring notation can be presented independently of Lean as a small BNF. Named productions are interactions; referring to a production composes that interaction into a larger one.

```
<specification> ::= <production>
                  | <production> <specification>

<production>   ::= <name> "::=" <expression>

<expression>   ::= "empty"
                   | <event>
                   | <name>
                   | "(" <expression> ";" <expression> ")"
                   | "(" <expression> "|" <expression> ")"
                   | "(" <expression> "||" <expression> ")"
                   | "repeat" "(" <expression> ")"
                   | "join" "(" <natural> "of" <expressions> ")"
                   | "when" "(" <predicate> "," <expression> ")"
                   | "after" "(" <name> "," <expression> ")"

<event>        ::= <actor> "->" <actor> ":"
                    <message> "(" <fields> ")"

<fields>       ::= ""
                   | <field>
                   | <field> "," <fields>

<field>        ::= <name> "=" <value>
```

The operators have the same meanings as the grammar AST: semicolon is sequence, vertical bar is alternative, and double vertical bar is parallel composition by order-preserving interleaving. `after(P,E)` includes `E` only on a branch in which production `P` completed. Predicates and repeated field variables supply the context-sensitive correlation that plain context-free syntax does not express by itself.

A threshold join is explicit rather than inferred from the number of observed predecessors:

```
firstTwoReplicas ::= join(2 of replicaA, replicaB, replicaC)
firstTwoThenReply ::= (firstTwoReplicas ; reply)
```

The continuation becomes enabled after any two of the three branches complete. Its first emitted event records `joinRequired=2`, `joinTotal=3`, and exactly two selected branch-terminal IDs. Those IDs also occur in the event's ordinary `prior` list, which may contain other backpointers such as the task start. The threshold declaration says why two predecessors suffice; the selected IDs say which two satisfied this instance. Unselected branches are not implicitly canceled and may still emit events. Cancellation, if required, must be an explicit message or production.

The OAuth example decomposes into independently readable interactions. These generated views show the two complete branches without making a message arrow look like logical implication:

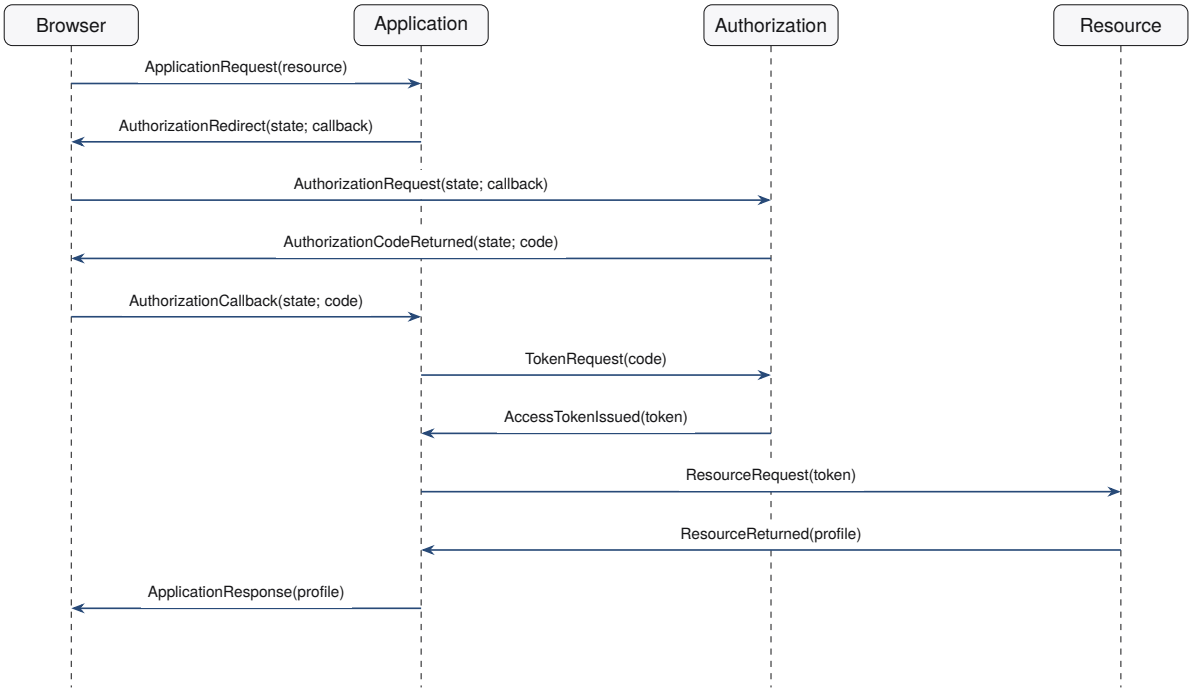


Figure 4.1: Granted OAuth interaction. The resource exchange occurs only after the authorization-code branch succeeds.

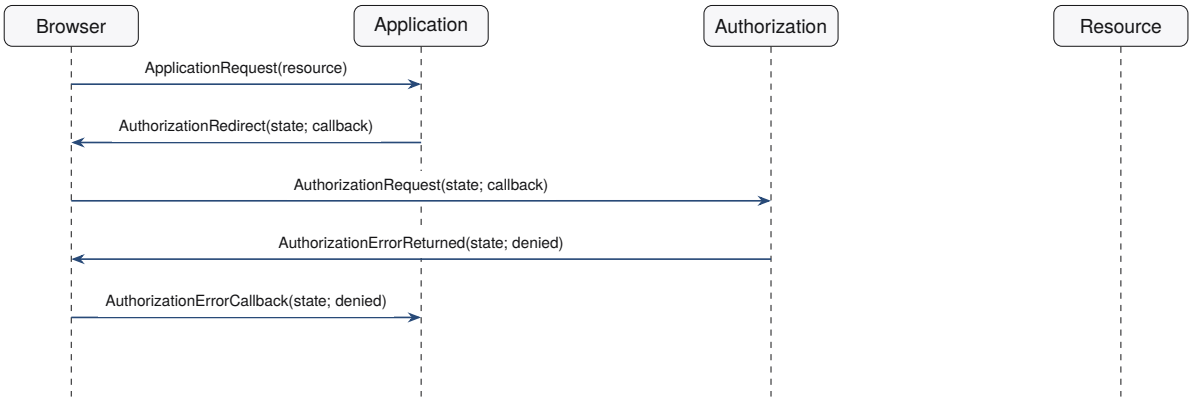


Figure 4.2: Denied OAuth interaction. No token or resource request follows the error.

At the composition level, `oauth` sequences the request-and-redirect interaction with the authorization choice. It includes the exchange-and-fetch interaction only after the granted branch.

This exposes two useful levels without defining two protocols. At the higher level, `oauth` is three production references. Expanding a reference reveals its events and nested alternatives. The same productions can be reused in another system, placed in parallel with independent interactions, guarded, or repeated. Composition operates on their accepted event languages; it is not textual copying.

4.3 Parameters carry information

A production should bind values at one event and reuse them later. Conceptually:

```
def login (browser server provider : Actor) (state : OAuthState) : Grammar Event :=
  event (redirect server browser provider state) >>>
  event (authorize browser provider state) >>>
  callback browser server state
```

The repeated `state` is not decorative. It states the information-flow relationship that distinguishes a matching callback from an unrelated one. Lean's types, constructors, and ordinary function parameters are the surface language; the fixed library supplies grammar composition and semantics.

4.4 Composition without omniscience

A participant's implementation need not know the global execution state. The global grammar states the contract, while projection derives each participant's view. Composition should preserve independently specified modules and make shared events explicit. Proving useful projection and composition theorems is a core research obligation, not an implementation footnote.

4.5 Actors multiplex bounded conversations

An actor has separate finite-capacity inbound and outbound message queues. Queue capacity is part of the observable execution model: producing a message may block when the outbound queue is full, and transport may block when the destination's inbound queue is full. Sending is therefore distinct from delivery and from the destination actor consuming the message.

Messages carry a session identifier as well as a task identifier. Their pair is the correlation key for actor-local work:

```
structure WorkKey where
  session : SessionId
  task    : TaskId

structure ActorRuntime where
  inboundCapacity  : Nat
  outboundCapacity : Nat
  inbound          : List Envelope
  outbound         : List Envelope
  inFlight         : List (Prod WorkKey GrammarResidual)
```

```
  inboundBounded   : inbound.length <= inboundCapacity
  outboundBounded  : outbound.length <= outboundCapacity
```

An actor may have zero or many distinct `WorkKey`s in flight. Consuming an inbound envelope selects or creates the residual associated with that envelope's `(session, task)` pair; it does not replace one actor-wide "current task." Messages from different keys may interleave while the grammar preserves the order required within each conversation.

The queues have Go-channel-like operational semantics. Capacity is enforced when an operation is attempted, not checked only after constructing a new state:

```
send(channel, message) =
  if channel.length < channel.capacity
  then sent(channel ++ [message])
  else blocked

receive(channel) =
  if channel = message :: rest
  then received(message, rest)
  else blocked

tryReceive(channel) =
  if channel = message :: rest
  then some(message, rest)
  else none
```

A blocked operation makes no progress and leaves the channel unchanged. A blocking actor may run again when the required space or data becomes available. `tryReceive` is a different operation: `none` reports that the channel is empty now and lets the actor continue. It does not put the actor to sleep, and it must not be confused with receiving a valid optional payload.

The resulting transitions are explicit:

- an actor step appends a produced envelope to its outbound queue;
- a transport step moves the outbound head to the destination's inbound queue when both operations are within capacity;
- a receive step removes an inbound envelope and advances the matching `(session, task)` residual;
- sending to a full outbound queue blocks the sender without enqueueing;
- transport to a full inbound queue blocks without removing the outbound head;
- blocking receive on an empty inbound queue blocks, while nonblocking `tryReceive` returns `none` and permits later work.

Queue order and predecessor order are not identical. FIFO determines which queued envelope is available next, while `priorIds` records the event-order graph, including forks and joins across actors. Incomparable branches may commute; the pointer relation does not itself claim that one event caused another.

Chapter 5

The Scenario Authoring Language

This chapter proposes the first concrete surface language. It is intentionally a small Lean DSL: ordinary `def`, function parameters, `let`, typed values, and `do` notation provide the host language; a fixed library adds scenario, endpoint, choice, probability, and rendering operations. The examples are a design target. Names may change as the Lean implementation makes their semantics precise.

5.1 The smallest useful scenario

A scenario is a named grammar fragment. Every emitted terminal is an observable message event. Endpoint commands establish the `from` and `to` context for the messages that follow.

```
def requestQuote (customer shop : Actor) : Scenario :=
  scenario "request a quote" do
    let requestId <- declare RequestId "requestId"

    from_ customer
    to_ shop
    emit "quote requested" [field "requestId" requestId]

    let quoteId <- declare QuoteId "quoteId"
    let amount  <- declare Money "amount"

    from_ shop
    to_ customer
    emit "quote offered"
      [ field "requestId" requestId
      , field "quoteId" quoteId
      , field "amount" amount
      ]
```

The spelling `from_` avoids collision with Lean syntax while remaining visually explicit. Endpoint commands affect authoring context only; elaboration copies both endpoints into every event. A scenario with an `emit` before both endpoints have been established is rejected.

5.1.1 Variables and assignment

There are two kinds of assignment.

- Ordinary `let x := expression` evaluates a known Lean expression.
- `let x <- declare T "x"` introduces a symbolic value of type `T`. The grammar binds it when generating or matching a trace, and later events may reuse it.

Variables are immutable. A later `let attempt := attempt + 1` creates a new binding rather than mutable hidden state. Information flow is consequently visible in the grammar's data dependencies. Equality of two occurrences of `requestId` means they carry the same value, not merely values with the same field name.

A value received from the environment is bound by matching an observable event:

```
from_ customer
to_ shop
receive "quote accepted" do
  let quoteId <- take QuoteId "quoteId"
  let payment <- take PaymentToken "paymentToken"

from_ shop
to_ bank
emit "payment authorization requested"
  [field "quoteId" quoteId, field "paymentToken" payment]
```

`receive` and `emit` both denote grammar terminals; they differ only in the viewpoint used when projecting a global scenario to executable participant code.

5.2 Vague messages first, typed alphabets later

Early requirements should not require a finished protobuf schema. A vague message has a stable semantic name and a list of named, typed fields. It is already precise enough to express ordering, correlation, and temporal properties.

```
emit "quote offered"
  [field "requestId" requestId, field "amount" amount]
```

Later, a refinement maps that terminal to a concrete alphabet:

```
refineMessage "quote offered" with Proto.Shop.QuoteOffered where
  request_id := field "requestId"
  quote_id   := field "quoteId"
  cents      := encodeMoney (field "amount")
```

The refinement obligation is language inclusion: every concrete implementation trace, after erasing representation details, must be admitted by the abstract grammar. Refinement may decide serialization, retry mechanics, and internal state; it must not invent externally visible behaviour forbidden by the requirements.

5.3 One concern per scenario

Scenarios should be small enough to read as stories and independent enough to render separately. The quote protocol can be decomposed as follows.

```
def quoteAccepted (customer shop : Actor) : Scenario :=
  scenario "quote accepted" do
    include (requestQuote customer shop)
    -- bind quoteId from the included scenario's exports
    use quoteId <- exported QuoteId "quoteId"
    from_ customer; to_ shop
    emit "quote accepted" [field "quoteId" quoteId]

def quoteDeclined (customer shop : Actor) : Scenario :=
  scenario "quote declined" do
    include (requestQuote customer shop)
    use quoteId <- exported QuoteId "quoteId"
    from_ customer; to_ shop
    emit "quote declined" [field "quoteId" quoteId]

def quoteOutcome (customer shop : Actor) : Scenario :=
  oneOf "customer decision"
    [ quoteAccepted customer shop
    , quoteDeclined customer shop
    ]
```

`include` is textual only in the rendered book view; semantically it is a reference to a named nonterminal. The compiler reads all referenced scenarios, checks exported bindings, detects cycles, and builds one grammar. Tools retain the scenario boundary for navigation, diagrams, tests, and counterexamples.

5.4 Interaction diagram: accepted quote

The first rendering of any scenario is a sequence-style interaction diagram. It shows observable messages and parameter flow without suggesting an internal implementation architecture.

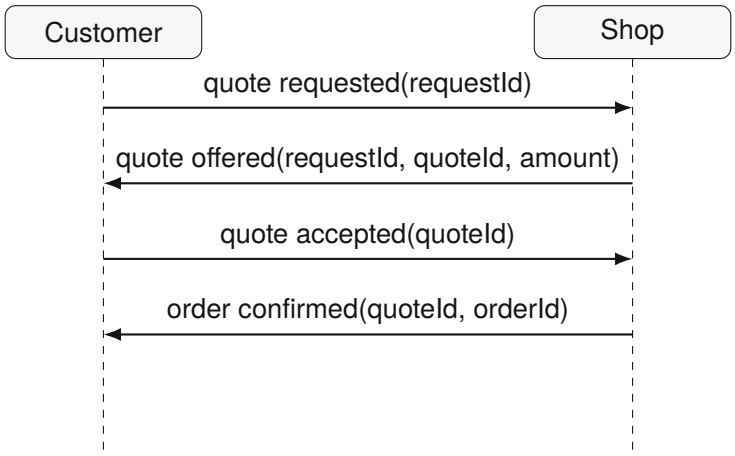


Figure 5.1: Interaction diagram derived from the accepted-quote scenario.

5.5 Choices and unresolved probability

Nondeterminism and probability are different. `oneOf` says that multiple behaviours are legal without claiming how often they occur. `weighted` attaches a distribution when the abstraction has one.

```
def paymentOutcome : Scenario :=
  oneOf "payment outcome" [paymentApproved, paymentRejected]

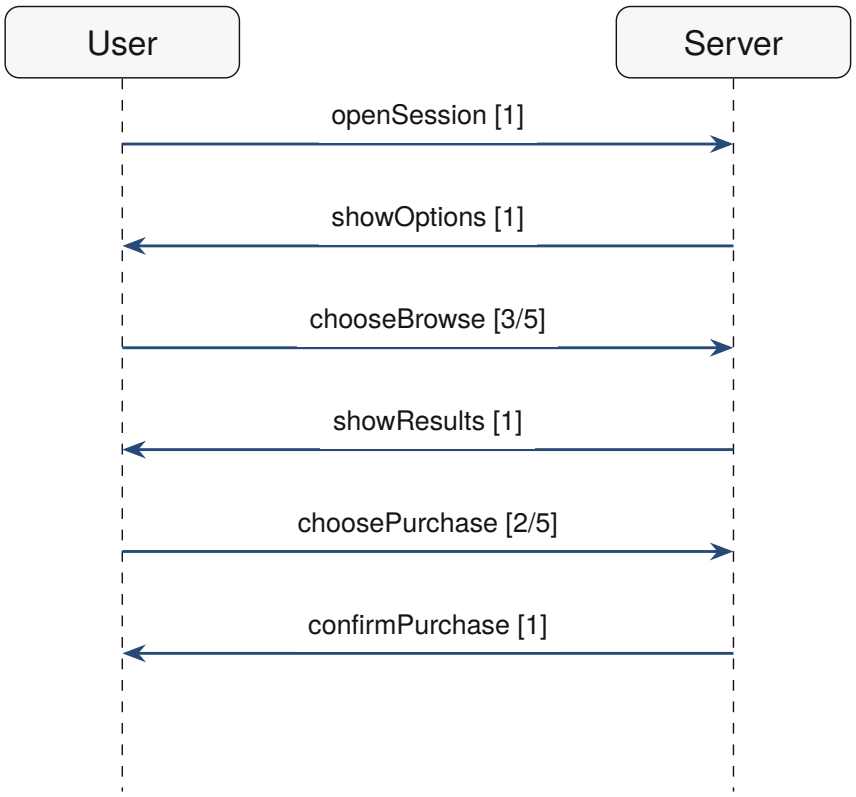
def estimatedCustomerDecision : Scenario :=
  weightedOneOf "customer decision"
    [ weighted (95 / 100) customerSubmitsPayment
    , weighted ( 5 / 100) customerAbandonsPayment
    ]
```

There is no separate `choose customer` annotation. The branch is resolved when its distinguishing terminal becomes bytes: the terminal names its source, so the trace already shows that the customer emitted the event that selected the branch. Probabilities belong to branches, not to an extra, potentially conflicting claim about a chooser. A deterministic service that consumes probabilistic user input remains part of the induced probabilistic system: its probability-one transitions carry the incoming path mass forward. An early requirement can use `oneOf`, a symbolic weight, or a range such as `between 0.90 0.99`. Implementation refinement resolves those claims to a well-defined stochastic model or to deterministic conditions. Refinement must preserve possibility even when a current implementation assigns probability zero to a legal abstract branch; otherwise changing operational policy would silently change the contract.

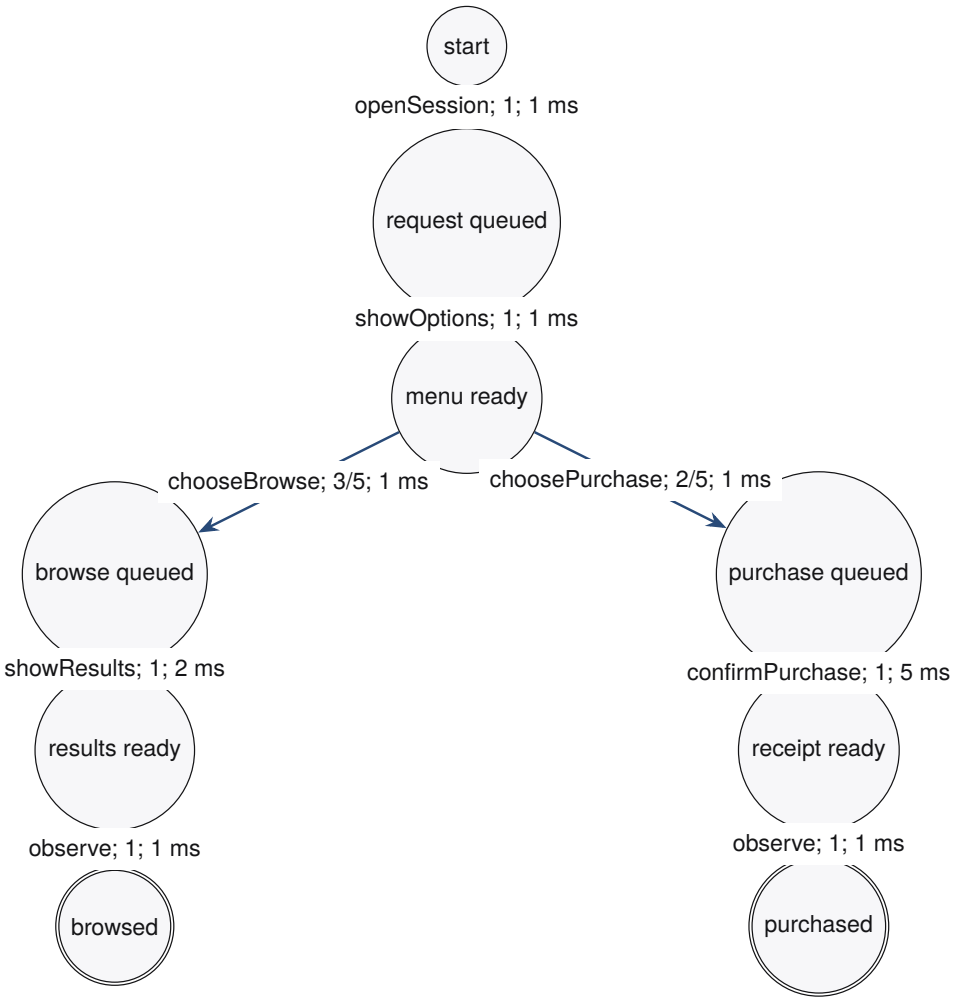
Two branches may share a prefix. In that case the choice remains unresolved through the common terminals and resolves only at the first different byte-level terminal. Branches whose complete terminal byte languages overlap are observationally ambiguous at that point; the requirement must merge them or add an observable discriminator if downstream actors need to distinguish them.

5.6 Worked MDP: a stochastic user and deterministic server

This finite example separates the sources of uncertainty. The user opens a session, receives two options and chooses browse with weight 3 or purchase with weight 2. The server does not toss a coin: for a fixed state and received message, it has exactly one next action. The composed system is probabilistic because the server transports the user's probability mass to its terminal responses.



The two user-choice arrows are alternatives after `showOptions`; they are not two messages in one execution. Every server arrow is labeled 1. The labels come from the checked `Choice` values rather than from a separately drawn explanation.



The branch weights are relative integers. Their sum supplies the denominator:

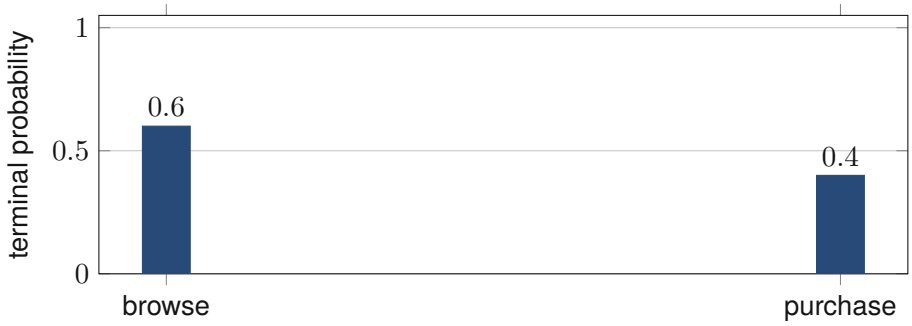
$$\frac{3}{5} + \frac{2}{5} = 1.$$

Read this as: “three fifths plus two fifths equals one.” The checker rejects an empty chance set and every zero-weight branch. Normalization divides each positive branch weight by the total weight; it does not require floating-point arithmetic.

Assign reward 1 to browsing and reward 8 to purchasing. The terminal paths and their durations give

$$\mathbb{E}[R] = \frac{3}{5}(1) + \frac{2}{5}(8) = \frac{19}{5}, \qquad \mathbb{E}[T] = \frac{3}{5}(6) + \frac{2}{5}(9) = \frac{36}{5}\ \text{ms} = 7.2\ \text{ms}.$$

Read these as: “expected reward is probability-weighted terminal reward and equals nineteen fifths; expected latency is probability-weighted end-to-end duration and equals thirty-six fifths milliseconds, or 7.2 milliseconds.”



`StochasticUserMDPExample.lean` checks the distribution, deterministic server states, terminal paths, mass conservation and both expectations before generating these views. The present evaluator expands a finite number of steps under a selected policy. It does not yet compute an optimal policy, solve an unbounded fixed point, or take minimum and maximum values over unresolved schedulers. Those are distinct extensions; this example establishes that exact probability propagation already works for a bounded policy.

5.7 Actor specifications and instances

An actor name in a grammar denotes a reusable specification, not necessarily one process. A finite population gives that specification concrete identities:

```
actor Client
  |>.instances "client-" 20 (sticky "session")

actor Server
  |>.instances "server-" 2 (shardHash "tenant_id")
```

The grammar may send a message from the `Client` role to the `Server` role; a generated scenario resolves those roles to instances such as `client-7` and `server-2`. Each instance has its own inbound and outbound queues and its own set of in-flight `(session,task)` pairs, while inheriting behavior and resource bounds from the actor specification. A population is a homogeneous service cluster: every member must implement that one actor specification. A cluster cannot quietly mix gateways, databases, and workers merely because they share a deployment boundary.

The cluster also declares how a message selects an instance. `sticky` keeps the value of an observable affinity field on the same member; `roundRobin` cycles through members in a declared order; and `shardHash` hashes an observable shard-key field. This decision changes the legal per-instance histories and therefore queue pressure, hot spots, failure impact, and replay. Kafka partition keys, MongoDB shard keys, and DynamoDB partition keys motivate the shard-hash abstraction, although each real system adds product-specific replication and routing semantics beyond this small model. Population size is finite and explicit so queue, memory, probability, and temporal checks still have a bounded state space.

5.8 Non-functional requirements: rates and scalability

Timing requirements are stated over boundary messages rather than intervals hidden inside an event. Suppose every client repeatedly creates a fresh server task. Each completed unit records a start

message, an end message that backpoints to that start, and a nonnegative amount of work. If one moved byte is one work unit and timestamps are milliseconds, an exhaustive observation list supports two different rates.

The word *load* is too ambiguous to serve as a variable. Let N be the declared upper limit on the number of active clients in the study, and let the parameter n be the active-client population for one observation, with $1 \le n \le N$. Separately, let λ_o be offered work per unit time, λ_a be work admitted per unit time, and X be work completed per unit time. Offering work, accepting it into a queue, and completing it are three different observable boundaries. The client-experienced, properly weighted rate is

$$X_{\text{client}}(n) = \frac{\sum_i \text{work}_i}{\sum_i (e_i.timeAt - s_i.timeAt)}.$$

Read this as: "client throughput at active-client population n equals the sum, over requests i, of work for request i, divided by the sum of end time $e_i.timeAt$ minus start time $s_i.timeAt$ for that request." Lowercase n is the varying client population; uppercase N is only its declared upper limit.

This is the duration-weighted mean of the individual observed rates. Long requests therefore contribute in proportion to how long a client experienced them. It is not the unweighted mean of per-request ratios.

The aggregate server throughput is instead

$$X_{\text{server}}(n) = \frac{\sum_i \text{work}_i}{\max_i(e_i.timeAt) - \min_i(s_i.timeAt)}.$$

Read this as: "server throughput at active-client population n equals total completed work over requests i, divided by the latest request-end time minus the earliest request-start time." The maximum and minimum make the denominator one wall-clock window.

For an observation window of duration T, the boundary rates and backlog obey

$$\lambda_o = \frac{W_o}{T}, \qquad \lambda_a = \frac{W_a}{T}, \qquad X = \frac{W_c}{T}, \qquad B_1 = B_0 + W_a - W_c.$$

Read this as: "offered rate lambda sub o is offered work W_o divided by window duration T; admitted rate lambda sub a is admitted work W_a divided by T; completed throughput X is completed work W_c divided by T; and ending backlog B_1 is starting backlog B_0 plus admitted work minus completed work." The backlog identity assumes one conserved work unit and requires cancellations, expiry, and failures to be recorded as explicit terminal dispositions when they remove work.

Consequently, $\lambda_o > X$ does not mean that the system handled the offered load. The difference is rejected, remains queued or in flight, or receives another explicit terminal disposition. Nor does λ_a measure handled work: an implementation can admit requests faster than it completes them while latency and backlog grow without bound.

Its denominator is one wall-clock observation window, so overlapping client waits are counted once. The two quantities answer different questions and must not share a metric name. Plotting either explicitly chosen rate against the number of active client instances yields observations that can be fitted to the Universal Scalability Law,

$$X(n) = \frac{\gamma n}{1 + \alpha(n-1) + \beta n(n-1)}, \qquad 1 \le n \le N.$$

Read this as: "completed throughput X at active-client population n equals gamma times n, divided by one plus alpha times $n-1$ plus beta times n times $n-1$; n ranges from one through the upper limit N." This curve uses population or concurrency as its independent variable. It is not, without an explicit experimental mapping, a curve whose horizontal axis is offered work rate λ_o.

Gunther's *Guerrilla Capacity Planning* gives the practical capacity-planning setting for this law and develops it as the Universal Law of Computational Scaling [4]. The point is operational, not decorative: a requirements argument should expose contention, coherency, saturation, and retrograde scaling before the first production deployment supplies those facts by falling over.

Every USL observation must therefore retain n, λ_o, λ_a, X, latency, backlog, rejection or other terminal outcomes, server population, and observation-window boundaries. The event list supplies the observations; estimating γ, α, and β is a separate statistical step. The list must be exhaustive for the declared observation window, or omitted work and unfinished tasks bias both rates.

5.8.1 The three USL regimes and capacity

The scale parameter γ is analogous to clock speed: it is the single-client service-rate scale and multiplies the entire curve vertically. It is not literally a processor clock, because an observed unit of work may include networks, storage, and several actors. Changing γ does not move the population at which the dimensionless USL curve peaks.

The two overhead parameters have distinct effects:

$$\begin{aligned}
\alpha = 0,\ \beta = 0: &\quad X(n) = \gamma n, && \text{ideal linear scaling,}\\
\beta = 0: &\quad X(n) = \frac{\gamma n}{1 + \alpha(n-1)}, && \text{contention-limited saturation,}\\
\beta > 0: &\quad X(n) = \frac{\gamma n}{1 + \alpha(n-1) + \beta n(n-1)}, && \text{coherency-limited retrograde scaling.}
\end{aligned}$$

Read the three lines as follows. When alpha and beta are both zero, throughput at load n is gamma times n. When beta is zero, throughput is gamma times n divided by one plus alpha times $n-1$. When beta is positive, the denominator additionally contains beta times n times $n-1$.

For $0 \le \alpha < 1$ and $\beta > 0$, differentiation gives the finite capacity population

$$n_* = \sqrt{\frac{1-\alpha}{\beta}}.$$

Read this as: "the peak load n star equals the square root of one minus alpha, divided by beta." This fitted peak is a load value and need not equal the study limit N.

The observed integer capacity is one of the adjacent integer populations, whichever has the larger measured or fitted throughput. When $\beta = 0$ and $\alpha < 1$, the curve increases toward the asymptote γ/α but has no finite USL peak. When both parameters are zero, the mathematical model has no internal capacity limit; a real finite system must eventually reveal an omitted bottleneck or cease to fit that ideal case.

Figure 5.2 uses normalized throughput X/γ. The fitted example uses $\alpha = 0.1$ and, in the third panel, $\beta = 0.005$, giving $n_* \simeq 13.42$. Throughput to the right of the marked point declines even as more clients are added.

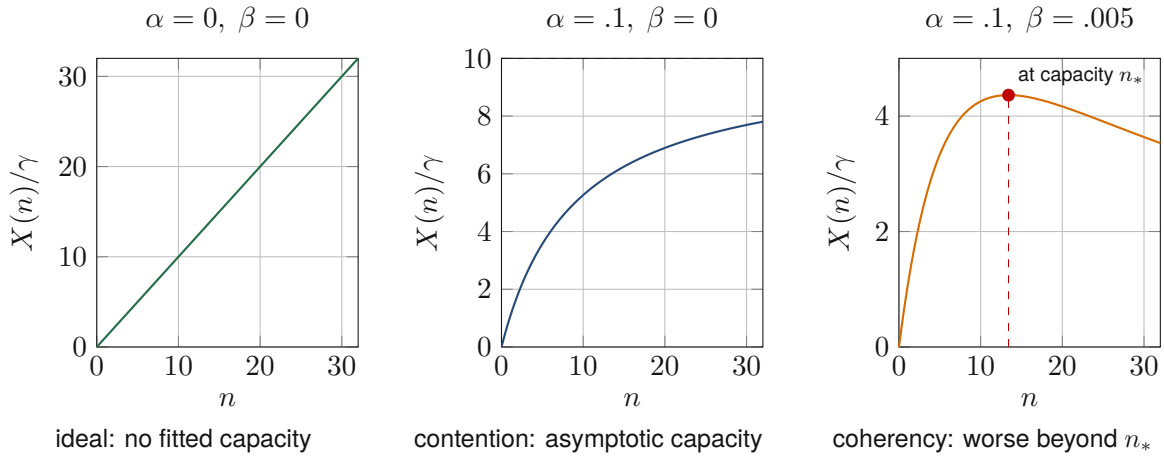


Figure 5.2: Universal Scalability Law regimes. γ rescales every vertical axis. Contention alone saturates; coherency creates a finite peak and declining throughput beyond capacity.

Population capacity is not the only overload boundary. For a fixed deployment with sustainable service rate μ, offered load λ is at capacity when $\lambda = \mu$. Below capacity, a simple queueing illustration has response time proportional to $1/(\mu - \lambda)$ and queued work proportional to $\lambda/(\mu - \lambda)$. At and above capacity these steady-state quantities are unbounded: completed throughput is capped by μ, while unfinished work, latency, queued bytes, and memory pressure accumulate. A finite queue converts that growth into blocking or rejection; exhausting the memory budget is a fatal terminal event rather than another slow sample.

For the queueing foundation behind this illustration—including workload characterization, service demand, utilization, response time, queueing networks, and cost/performance tradeoffs—see Menascé, Almeida, and Dowdy's *Performance by Design* [9]. USL and a single queue answer different questions and belong together: USL summarizes scaling across populations, while queueing models explain how offered work waits for finite service centers. Neither rescues a requirements model whose demand, service rates, queue bounds, observation window, and overload behavior were never stated.

This is why performance belongs in requirements rather than in a post-deployment incident report. Before implementation, the specification should make it possible to ask whether forecast demand fits below sustainable capacity, whether latency and queue bounds remain acceptable, what happens to rejected or blocked work, how many replicas are affordable, and which failure modes are fatal. The goal is not to predict every scheduler tick. It is to rule out designs that work only at toy load and collapse the first time real users arrive.

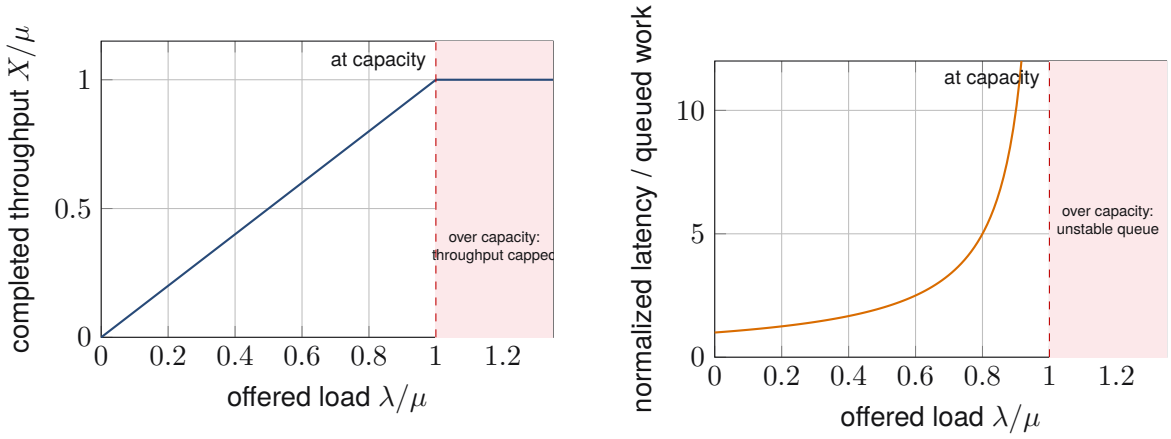


Figure 5.3: Offered-load capacity. Beyond $\lambda/\mu = 1$, throughput cannot increase but queued work and latency continue to worsen until backpressure, rejection, an outage, or fatal memory exhaustion intervenes.

5.9 Outages, recovery, and replica populations

Reliability assumptions belong to actor specifications and are inherited by their instances. A contract may set an outage probability and an expected mean time to repair (MTTR):

```
actor Server
  |>.instances "server-" 2 (roundRobin)
  |>.outageProbability (20 / 10000)
  |>.meanTimeToRepair 45.seconds
```

These are specified assumptions. A failed actor does not reliably announce its own failure: there is no ordinary `Unavailable` message to receive. Measurements instead come from evidence visible to an observer. Expected heartbeats stop arriving; an inbound queue ceases to drain; or messages expected from the actor's outbound side cease to appear. Each detector must declare its heartbeat or progress expectation, timeout, clock, observer, and attribution boundary. Silence without such an expectation is not evidence of failure.

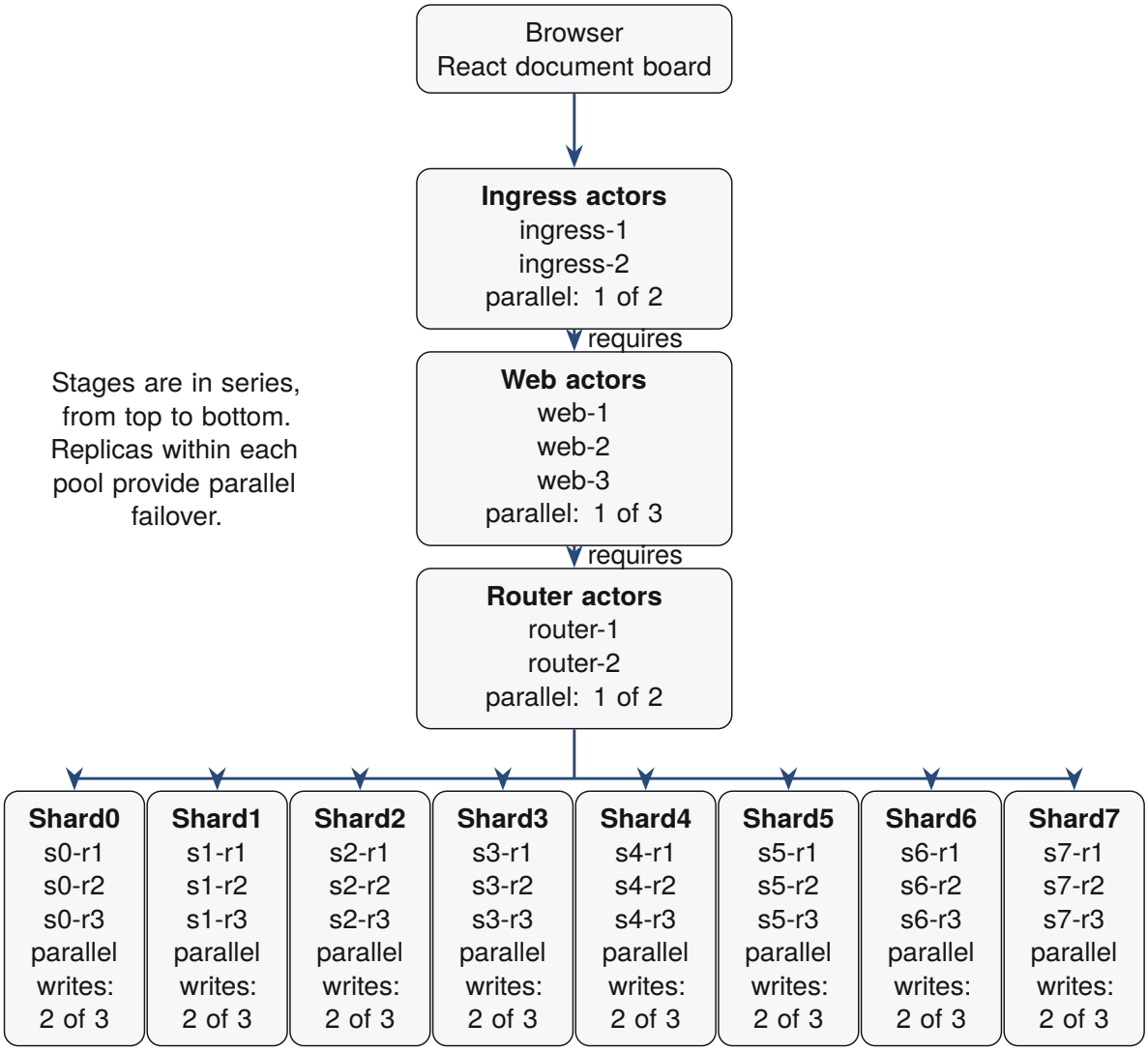
The detector may emit derived observations such as `AvailabilityLost` and `AvailabilityRestored`, but these are observer conclusions, not protocol messages sent by the failed actor. A loss observation retains the last known healthy time, the deadline that was missed, the detection time, and the evidence stream. Restoration retains the first new heartbeat or queue/message progress that justifies it and backpoints to the loss observation. The true failure time is generally bounded between the last healthy observation and detection rather than known exactly. If the declared accounting policy uses derived boundary times u_i and r_i, then for R replicas observed from t_0 through t_1, the estimated outage fraction and repair duration are

$$\widehat{O} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{R(t_1 - t_0)}, \qquad \widehat{MTTR} = \frac{\sum_i (r_i.timeAt - u_i.timeAt)}{|incidents|}.$$

Read these as follows. Estimated outage fraction, $\widehat{O}$, is the sum over incidents i of restoration time $r_i.timeAt$ minus outage-boundary time $u_i.timeAt$, divided by replica count R times window duration $t_1 - t_0$. Estimated mean time to repair, $\widehat{MTTR}$, is that same total outage time divided by the number of incidents. A hat marks an estimate from observations.

Availability over the same complete window is $1 - \widehat{O}$. The report must label whether u_i is the last-known-healthy time, missed deadline, or detection time; otherwise its outage and MTTR numbers are not reproducible. Both values can be plotted against time, replica count, offered load, or a deployment policy. This supports Kubernetes-style experiments in which pods are actor instances, restarts are recovery events, and changing replica populations yields different reliability and throughput observations. A configured outage probability or MTTR is not silently replaced by its observed estimate; comparing them is the check.

5.9.1 Redundant, load-balanced runtimes and blast radius



Shards are parallel data partitions with separate availability gates.
Each shard has parallel replicas; writes need a primary and a threshold quorum.
A targeted write selects one shard, not all shards.

Actor topology for a targeted document write. Generated from `RuntimeReliabilityDiagram.lean`, with provenance in `runtime-actor-network-provenance.txt`. Arrows show required provider paths, not logical implication or a message sequence. Replica counts are an illustrative configuration, not measured uptime. Metadata, storage and network dependencies must be added for a complete deployment analysis.

Series adds failure dependencies. A request needs ingress, a web actor, a router and its selected shard. Adding required stages cannot improve that path's availability; independent stage availabilities multiply. Long series paths therefore tend to reduce reliability even when each stage is reliable.

Parallel replicas provide failover. Proxy and web actors are duplicated in parallel within their series stages. Database shards run as parallel data partitions with separately calculated availability gates; each shard is itself replicated in parallel. A shard's quorum threshold determines how many of its replicas must communicate for writes. "Parallel shards" does not mean one shard can serve another shard's missing data. An additional usable alternative can improve a pool's availability. Shared failure domains and increased quorum thresholds mean that additional replicas do not provide an unconditional improvement.

Sharding divides offered work. Additional shards can increase handled load when requests distribute well. Hot keys, shared routing, storage and cross-shard operations can limit the gain. Shards are data partitions, not interchangeable failover copies; replication protects each partition.

USL models scaling penalties. Fit completed throughput against active load n: contention/serialization, including queueing for shared resources, contributes the α term; coherency or coordination contributes the β term [4]. Read these symbols as "alpha, the contention parameter" and "beta, the coherency parameter." USL is a throughput-scaling model, not a full queue-length or latency model. Retain offered/admitted/completed rates, queue observations and deadlines alongside the fit described earlier in this chapter.

The purpose of modelling a Kubernetes-like runtime is to determine what stops, for whom, and for how long when actors fail or lose capacity. Replica counts alone do not answer that question. The model combines timestamped protocol observations, uptime and MTTR estimates, operation-specific dependency gates, and the placement of instances in shared failure domains.

The standard runtime profile has a redundant ingress/reverse-proxy pool, round-robin stateless web actors serving React and API requests, and a sharded document service. A shard router selects the owner of each document; each owner is a replicated group rather than one interchangeable database process. Ingress, routers, metadata services, storage, and network connectivity are dependencies in their own right. Kubernetes restarts and schedules actors; it does not supply the application's quorum or consistency protocol.

Stateless pool. Zero ready instances makes new requests unavailable. With surviving instances, compare offered work with the pool's sustainable capacity under the declared latency bound. Overload may cause bounded rejection, growing queues, or deadline failures while heartbeats continue. Round-robin request counts need not balance work when requests differ in cost.

Replicated shard. Majority-acknowledged writes require a current primary and a communicating quorum of eligible configured members. Heartbeats received by one external observer do not establish mutual reachability. A restarted member must catch up before it counts as ready.

Read operation. Declare its consistency separately. Loss of write quorum does not automatically forbid local stale reads; neither does a surviving copy establish that a linearizable read is available.

Shared dependency. A failed ingress path or routing service can affect every shard. Metadata-service effects depend on the operation and whether valid routing information is already cached; do not mark every read or write unavailable merely because a metadata update cannot proceed.

For eight shards with three data-bearing voting replicas each, there are 24 shard replicas, plus any routers and metadata replicas. One failed member in each shard preserves the two-of-three write quorums, subject to primary election and recovery. Two failed members in one shard remove that shard's write quorum, even if all other shards are healthy. A targeted operation depends on its target shard; a multi-shard operation depends on all shards required by its semantics. Thus the relevant question is not "how many of the 24 are up?" but "which operation still has its required providers?"

Per-instance uptime is only the first reliability result. Services form a dependency network, and each dependency gate must say whether its members are required in *series* or provide redundancy in *parallel*. Let $u_i(t) \in \{0, 1\}$ be the derived up/down indicator reconstructed from one instance's detector observations and declared boundary policy. For a series gate, every member must be up:

$$u_{\text{series}}(t) = \prod_i u_i(t).$$

Read this as: "series availability at time t equals the product, over instances i, of instance i's up indicator at time t."

For a parallel gate, at least one member must be up:

$$u_{\text{parallel}}(t) = 1 - \prod_i (1 - u_i(t)).$$

Read this as: "parallel availability at time t equals one minus the product, over instances i, of one minus instance i's up indicator at time t."

More generally, a replicated service declares a threshold k and is available when $\sum_i u_i(t) \geq k$. Its measured availability is the fraction of the observation window for which that predicate is true. These formulas operate on synchronized observed indicators, so simultaneous and correlated failures are preserved. Products such as $\prod_i A_i$ or $1 - \prod_i (1 - A_i)$ may estimate expected availability from marginal availabilities only under an explicit independence assumption; they must not replace the interval calculation when joint outage observations exist.

A dependency edge points from a consumer service or task to the provider gate it requires. At each outage boundary, failure propagation repeatedly marks a consumer unavailable when one of its series requirements is unavailable or when a parallel/threshold requirement has too few surviving providers. The fixed point is the cascade for that particular set of failed services. The report must retain the initiating outage set, every derived unavailable service, the dependency edge that propagated each failure, affected tasks, client-visible outcomes, and recovery order. This makes questions such as "what happens if the database and one authorization replica fail together?" answerable without pretending that all outages are independent.

The standard reliability output therefore includes per-instance uptime, series/parallel/threshold service uptime, a time-aligned outage chart, and a dependency graph overlaid with each observed or proposed failure cascade. Named service-down scenarios should be replayable even when they have not yet occurred in production; such a replay is a model prediction and must be labeled separately from measured uptime.

Calculate impact before estimating its frequency. For a named initiating failure set F, let $C(F)$ be the unavailable-operation set obtained by repeatedly evaluating the dependency gates until no further operation becomes unavailable. Read $C(F)$ as "the failure closure of F." The closure is the structural blast radius. Retain its propagation paths and identify the affected shards, request classes,

tenants, and workload. A degraded read path and an unavailable write path are different outcomes, not one undifferentiated red service box. Minimal cut sets identify the smallest initiating sets that disable each operation.

Uptime estimates help assign frequencies or probabilities to these scenarios; MTTR estimates help assign recovery durations. Neither marginal uptime nor mean repair time determines a cascade by itself. The dependency graph determines which operations are exposed, while correlated failure and repair models determine how often and how long. If scenarios F_j form a mutually exclusive, exhaustive partition of a declared snapshot model, with probabilities π_j, then the expected number of unavailable operations is

$$\mathbb{E}[B] = \sum_j \pi_j \, |C(F_j)|.$$

Read this as: "the expected blast-radius count B is the sum, over scenarios j, of scenario probability pi-sub-j times the number of operations in that scenario's failure closure." A count weights every operation equally; a report may instead use an explicitly declared demand or business-impact weight. Overlapping outage scenarios cannot simply be added as if they were disjoint.

For a time-aligned trace, let $C(t)$ be the unavailable-operation set at time t, and let $\lambda_o(t)$ be offered requests per second for operation o. The offered-request exposure over window $[t_0, t_1]$ is

$$L = \int_{t_0}^{t_1} \sum_{o \in C(t)} \lambda_o(t) \, dt.$$

Read this as: "L is the integral, from t-zero to t-one, of the sum of offered request rates for operations unavailable at time t." L has units of requests. It is exposure, not necessarily lost work: queues, retries, deadlines and recovery determine which requests ultimately fail or complete late. For overload, measure unmet work and deadline violations separately rather than counting every request as unavailable.

Repair the operation, not just the process. Actor MTTR is not automatically service MTTR. Service recovery may include failure detection, replacement scheduling, volume attachment, election, replication catch-up, routing refresh, and backlog draining. These stages can overlap, so compose their event dependencies rather than blindly adding their means. The end boundary is restored operation readiness or the declared latency target, not merely receipt of the first heartbeat. Open incidents at the end of the window remain censored observations, not zero-duration repairs.

A prediction from uptime and MTTR therefore also declares a failure/repair process, correlated failure domains, placement, workload and recovery policy. Independent-marginal estimates provide a baseline only. Rack loss, zone loss, network partition, operator shutdown and common deployment errors require named joint scenarios. The runtime report places assumed and observed values beside the generated dependency/cascade view, time-aligned availability, affected operations, request exposure, deadline outcomes, and recovery sequence. Existing `DesiredOutput` entries for composed availability and failure cascades provide the provenance boundary; additional calibrated forecasts require their own sources and reducers. This subsection defines the analysis contract, not a claim that a general Kubernetes deployment or forecast engine is implemented.

The generated artifact `paxos-quorum-shutdown-network.svg` gives a concrete majority-service example. A Kubernetes-like runtime manages a parallel pool of three stateless web actors; the one-of-three web pool is in series with a two-of-three database-replica gate. Each web actor has specified

uptime $9999/10000$; each replica has specified uptime $999/1000$. If replica outages were independent and each replica had uptime p, the quorum's assumed uptime would be

$$A_{\text{2-of-3}} = 3p^2(1-p) + p^3.$$

Read this as: "two-of-three availability equals three times the probability that exactly two replicas are up, p squared times one minus p, plus the probability that all three replicas are up, p cubed." This calculation is only an independent-marginal baseline. It does not describe an operator who deliberately stops correlated members.

The named shutdown cascade first stops `kv-1`. Two voters remain, so the successful first maintenance action appears to confirm that the cluster is redundant. Stopping `kv-2` next leaves `kv-3` running but destroys the majority: the quorum gate and the web-plus-quorum series path become unavailable. This is the operational trap familiar from ZooKeeper-backed Kafka deployments and Paxos- or Raft-like services. "A database process is still up" is not the service predicate; "a majority of the configured voting set is available and mutually communicating" is. Planned shutdowns, shared power, network partitions, and common automation are correlated failures and must be replayed as named sets rather than hidden inside multiplied uptime percentages. The runtime may create arbitrarily many healthy stateless web instances without creating another database vote. Autoscaling the parallel web tier therefore cannot repair the failed serial quorum gate; it may merely increase the number of requests waiting on that unavailable dependency.

An actor inferred unavailable produces no observed heartbeat or outbound protocol progress and does not drain its observed inbound queue. Work waiting on it remains stalled, is rejected, or ends in an explicit timeout/cancellation outcome according to the requirement. An ordinary send or receive attributed to that instance after a detector's loss boundary is evidence that either recovery occurred or the detector/attribution model was wrong; it is not silently discarded as an invalid trace. Thus an outage commonly falsifies a nominal liveness promise (for example, "every accepted request eventually receives a response") even when safety remains intact.

The nominal protocol language and its failure extension must therefore be distinguished. The nominal language describes ordinary successful and rejected conversations. The failure extension adds heartbeat expectations, queue and message-progress observations, derived availability boundaries, stalled or censored work, explicit failure outcomes, and legal resume/retry behavior. Cascade analysis suppresses modeled progress for every instance inferred unavailable at the dependency fixed point, then reports which nominal safety and progress properties remain true, become false, or cannot be decided before the observation window closes.

The same dependency graph supports denial-of-service analysis. A PageRank-like centrality calculation, with dependency edges directed toward the consumers whose operation they enable, highlights nodes whose failure influence is reinforced by other important services. Edge weights may incorporate observed message traffic, affected task value, or measured cascade frequency, but the chosen weighting is part of the requirement and must be shown in the report. Centrality is a prioritization heuristic, not an availability proof: ordinary PageRank can miss a low-traffic series dependency, a cut vertex, or the exact number of replica failures that crosses a quorum threshold. The denial-of- service report therefore places the centrality ranking beside single-node and named multi-node cascade simulations, minimal service cut sets, lost task capacity, and client-visible outcomes. This answers both "which node looks important?" and the stronger question "what actually stops if an attacker makes this set unavailable?"

5.10 Infrastructure cost and demand-constrained supply

A running service has at least two distinct time-based costs. Hardware is paid for while an instance is provisioned, whether or not the service is healthy. The service may also have a license, managed-service, or operational cost for each hour it is available. Those are not task-event costs and must not be multiplied by request count:

```
cost Server
  |>.hardwarePool "general-purpose"
  |>.hardwareCostPerProvisionedHour 12 USD.cent
  |>.serviceCostPerAvailableHour 3 USD.cent
  |>.sustainableCapacityPerHour 3600000 bytes
```

For R replicas, provisioned hours H_r, available hours A_r, hardware rate c_h, and service rate c_s, the accounting-period infrastructure cost is

$$C(R) = \sum_{r=1}^{R} (H_r c_h + A_r c_s).$$

Read this as: "cost at replica count R equals the sum from replica index $r = 1$ through R of provisioned hours H_r times hardware rate c_h, plus available hours A_r times service rate c_s."

Supply planning is a constrained optimization, not merely "pick the cheapest server." Let one replica sustain μ completed work units per hour, let a be the required availability fraction, and let D be the work rate that must complete within the declared latency boundary. A candidate replica count is feasible for demand D only when

$$R\mu a \geq D.$$

Read this as: "replica count R times per-replica completed-work capacity mu times required availability fraction a must be at least required completed demand D." This inequality is necessary but not sufficient: routing imbalance, queueing, and latency can still make the observed system infeasible.

For total offered rate λ_o routed across R servers, let $\lambda_{o,r}$ be the rate offered to server r:

$$\lambda_o = \sum_{r=1}^{R} \lambda_{o,r}, \qquad \lambda_{o,r} = \frac{\lambda_o}{R} \text{ only under ideal balance.}$$

Read this as: "total offered rate lambda sub o equals the sum, from server index $r = 1$ through server count R, of the rate offered to server r; under ideal balance only, each server's offered rate is total offered rate divided by R." Handled work must still be measured at each completion boundary; neither formula says that the offered work completed.

With an ideally balanced routing policy, the nominal demand on each server is therefore D/R. Spawning more homogeneous servers lowers nominal average per-server demand. That statement alone cannot choose a replica count: every added server also adds provisioned hardware cost and, while healthy, service cost. The useful graph places D/R, observed load for each concrete instance, effective supply, and total hourly cost on the same replica-count axis.

LeanFM enumerates the declared finite set of permitted replica counts, rejects infeasible points, and chooses the remaining point with minimum expected hourly cost $R(c_h + ac_s)$. The derived

XY artifact plots demand, effective supply, and cost against replica count and marks the feasible optimum. It also reports headroom and rejected demand so an apparently cheap under-supplied plan cannot win. The average D/R is not a promise of balance. Routing policy matters here: shard hashing can create a hot shard even when aggregate supply exceeds demand, while sticky routing and round-robin make different balance assumptions. Product-specific replication, reserved pricing, autoscaling delay, and nonlinear volume tiers require additional declared constraints rather than silent arithmetic.

Server count by itself is not a scalability result. An unbounded thought experiment can pair unbounded offered work with unbounded servers, backlog, cost, and latency; it says nothing useful about work completed within a finite deadline. A valid capacity claim fixes a finite observation window and reports offered, admitted, completed, rejected, and still-outstanding work together with latency, server count, utilization or imbalance, and cost. “Handled” means completed within the declared outcome and latency boundary, not merely accepted somewhere in an arbitrarily large system.

5.11 Specified dwell time

A specification may attach a dwell time to an event step or to a named production. The dwell is a duration on the same monotonic time scale as event clocks:

```
emit "token requested" [field "code" code]
  |>.dwell 3.milliseconds

include exchangeAndFetch
  |>.dwell 12.milliseconds
```

Taking a step with dwell d advances the model clock from t to $t + d$ before timestamping the event that completes the step. A production dwell constrains the elapsed time between its entry and exit boundary events; it does not assign the same dwell independently to every contained event. Exact values, bounded ranges, or weighted dwell outcomes may be used when the corresponding timing information is available.

Dwell is part of the specification, so it can drive latency, throughput, timeout, and queue-pressure calculations. It does not create event order: `priorIds` supplies the predecessor relation, while the grammar separately states why a sequence or dependency is legal.

5.12 Estimating completion from an event stream

The same specification can consume observed events. A terminal event of any kind names its corresponding start event in its backpointer list. The reducer follows that identity rather than guessing from adjacency or selecting the first open `(session, task)` attempt. Each matched pair yields a task attempt:

```
structure TaskAttempt where
  key       : WorkKey
  started   : Event
  completed : Option Event
  outcome     : Option TaskOutcome

def latency (attempt : TaskAttempt) : Option Duration :=
```

```
  attempt.completed.map fun completed =>
    completed.timeAt - attempt.started.timeAt
```

For an observation window containing M starts, C matched terminal events, S successful terminals, and completed latencies $L_1, \ldots, L_C$, the basic estimates are

$$\widehat{P}(\text{complete}) = \frac{C}{M}, \qquad \widehat{P}(\text{success} \mid \text{complete}) = \frac{S}{C}, \qquad \widehat{E}[L \mid \text{complete}] = \frac{\sum_i L_i}{C}.$$

Read these as follows. Estimated completion probability is completed count C divided by started count M. Estimated success probability conditional on completion is successful count S divided by completed count C. Estimated latency conditional on completion is the sum of completed latencies L_i, over completed attempts i, divided by C. The vertical bar is read "given," and each hat marks an estimate from the observation window.

Attempts without a terminal event remain *in flight*. They contribute to the number started and to queue/load measurements, but not to completed-task latency or to the conditional success denominator. Calling them failures would bias both probability and latency whenever the observation window ends before they do.

The reducer can group these sufficient statistics by task, actor, time bucket, maximum concurrency, or queue occupancy and feed them directly to pie, time-series, and XY plots. Specified probabilities and dwell times are therefore predictions; stream-derived estimates provide measurements against which those predictions can be checked.

In the executable reducer, an entry event opens a `TaskAttempt`, progress events leave it open, and a success or failure event closes the open attempt with the same `(session, task)` key. The resulting attempts feed the estimates above.

5.13 Composing requirements means intersecting constraints

Scenario composition builds behaviour in familiar language-theoretic ways:

Operator	Language operation	Purpose
`then a b`	concatenation	a happens before b
`oneOf [a,b]`	union	either story is legal
`together a b`	shuffle/interleaving	independent concurrent stories
`repeat a`	closure	repeated conversation
`where_ p a`	trace filtering	context-sensitive legality

Combining requirements is subtler. A requirement usually does not add another message sequence; it removes histories that violate a concern. Therefore `requireAll` denotes intersection of accepted trace sets.

```
def quoteSystem : SystemSpec :=
  generatedBy quoteOutcome
  |> requireAll
      [ correlate "offered quote matches request"
          (sameField "requestId")
      , forbid "confirmation without acceptance"
```

```
      (event "order confirmed" && notPreviously "quote accepted")
  , ctl "every accepted quote is answered"
      (AG (accepted --> AF (confirmed || rejected)))
  , timing "answer within service objective"
      (accepted --> within 2.seconds (confirmed || rejected))
  ]
```

This distinction prevents an important category error: union and concatenation assemble possible stories; intersection layers independent obligations over those stories. A system is consistent only if the intersection remains nonempty. When it is empty, tooling should return a minimal conflicting set of requirements and the shortest available counterexample prefix.

5.14 State-machine rendering

The same scenario grammar can be rendered as a state machine. States are grammar residuals—the legal continuations after a trace prefix—rather than snapshots of private implementation memory.

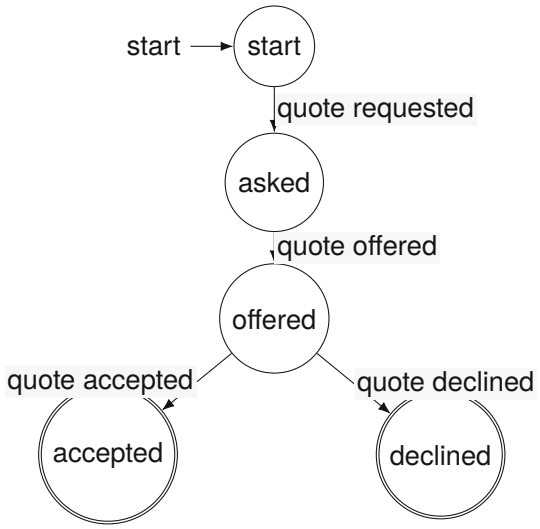


Figure 5.4: A state machine derived from grammar residuals.

5.15 Charts are declarative trace reductions

Charts are also specified in the Lean subset. A chart names an observable event source, a grouping or x expression, and an aggregate or y expression. It does not contain rendering code.

```
def outcomeShare : ChartSpec :=
  pie "quote outcomes" do
    source events
    where_ (kind == "quote accepted" || kind == "quote declined")
    groupBy kind
    value count
```

```
def responseLatency : ChartSpec :=
  line "quote response latency" do
    source (correlate events by "quoteId")
    x clock
    y (elapsed "quote offered" "customer decision")
    series "p50" median

def latencyByConcurrency : ChartSpec :=
  xyplot "latency by concurrent sessions" do
    source completedRuns
    x maxConcurrentSessions
    y (mean (elapsed firstEvent terminalEvent))

def queueByOfferedLoad : ChartSpec :=
  xyplot "queue occupancy by offered load" do
    source completedRuns
    x requestsPerSecond
    y (mean totalQueueOccupancy)

def propagatedGrantProbability : ChartSpec :=
  xyplot "user probability propagated through deterministic servers" do
    source parameterSweep
    x userGrantProbability
    y terminalGrantProbability
```

An XY plot preserves two numeric axes. It differs from a time series, whose x-axis is specifically an event clock, and from a categorical chart, whose x values are labels. The first two examples reduce each completed model run to one point. The third sweeps a model parameter and checks that deterministic downstream processing transports the user's probability mass.

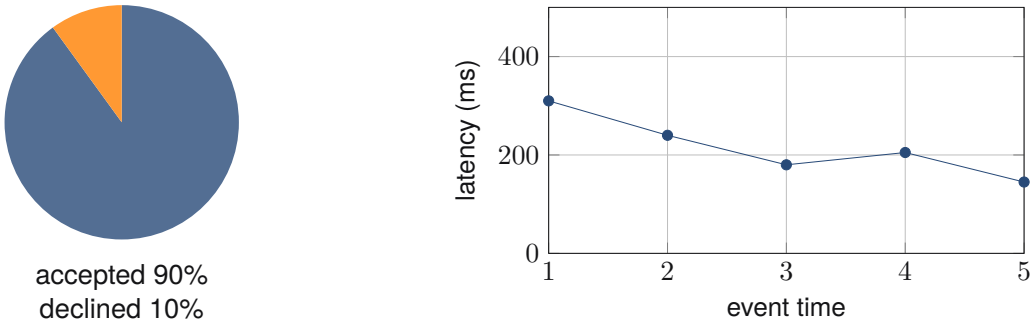


Figure 5.5: Example pie and line renderings from declarative chart specifications.

Interaction diagrams, state machines, and charts are projections of the same typed scenario and event definitions. Keeping them derived prevents a diagram from becoming a second, drifting specification.

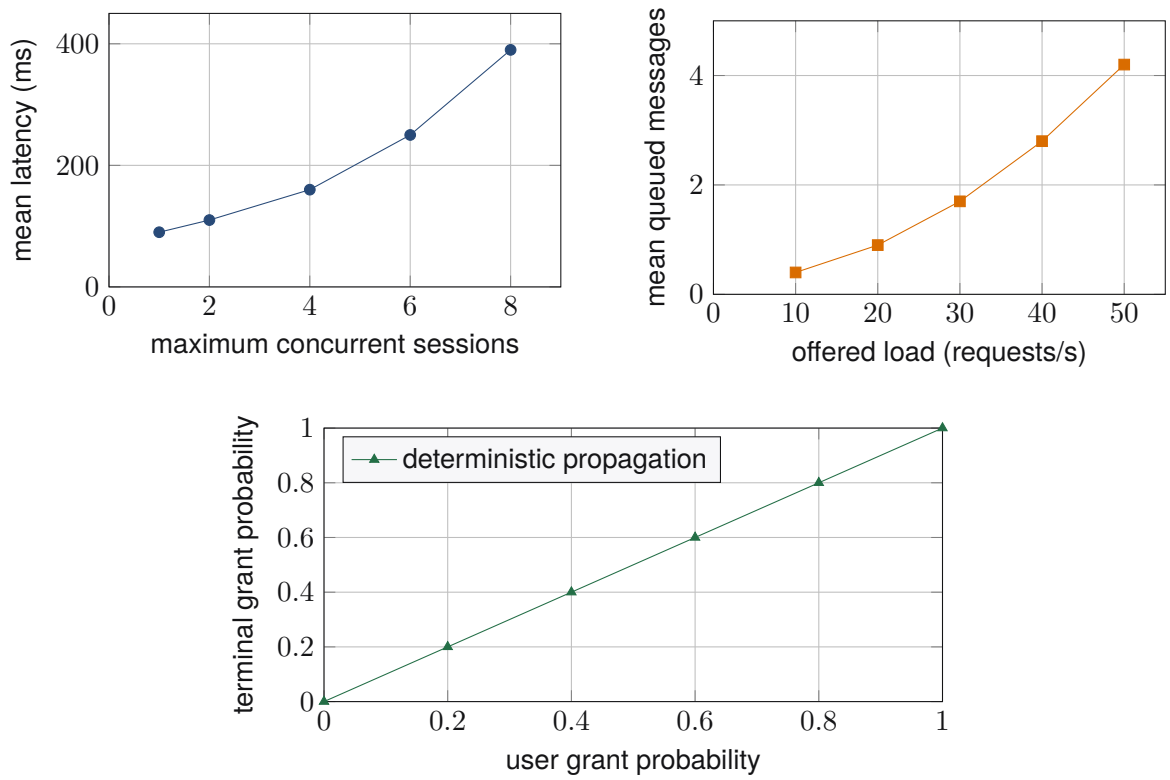


Figure 5.6: XY-plot examples. The values are illustrative model-run data: clocks and correlation produce latency, bounded queues produce occupancy, and probability-one server steps preserve user-originated grant mass.

Chapter 6

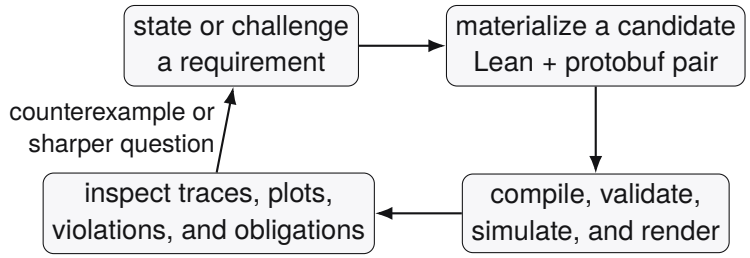
Arguing Requirements through the Lean Web Server

The LeanFM web server presents a conversational workbench, but conversation is not its product. Its product is a durable set of artifacts: a typed Lean requirements model, a protobuf description of the bytes exchanged by the participants, and a separate Lean implementation plan. The useful form of requirements vibecoding is therefore not "ask a model to write the system." It is an argument in which a person keeps challenging a proposed boundary until the remaining claims can be compiled, checked, measured, or left as explicit proof obligations.

Chat history is evidence about how a decision was reached. It is not the decision. A future reader must be able to discard the transcript, open the generated files, and recover the actors, observable events, alternatives, correlation rules, timing assumptions, resource bounds, and properties.

6.1 The argument loop

The interface resembles ChatGPT because natural language is convenient for finding omissions. The loop, however, has a formal boundary:



The human contribution is often adversarial. "The upload completes" should prompt: what observable event constitutes completion, who observes it, and does it mean receipt, durable storage, or policy acceptance? "The service handles load" should prompt: how many messages fit in each queue, how many `(session, task)` pairs may be active, and what happens at the bound? The assistant's job is to turn each answer into structure, not merely agreeable prose.

A productive exchange might proceed as follows.

Person: A submitted upload eventually completes.
Assistant: I will treat an `uploadCompleted` response as completion.
Person: No. Completion requires the storage actor's acknowledgment, and the client must be able to distinguish rejection from timeout. Both queues are finite.
Assistant: Then the grammar has accepted, rejected, and timed-out alternatives; acceptance contains a storage acknowledgment before the response. I will add queue capacities and a bounded-liveness obligation.
Person: Also show the behavior with many sessions in flight, estimate the completion distribution from an event stream, and fail the requirement if queued bytes exceed the memory budget.

Every turn should change a named assumption, artifact, check, or open obligation. If it changes none of them, it has not improved the specification.

6.2 The server is a staging environment

The executable `lake exe leanfm-server` serves the assistant workbench on the loopback interface. An authenticated session has its own generated `Requirements.lean`, `Requirements.proto`, and `Implementation.lean`. The first two specify observable behavior and values that resolve to bytes. They do not imply HTTP. The implementation file selects how those bytes travel—for example HTTP method and path, an in-process channel, TCP, or another adapter. The HTTP interface can read and replace these files through `/api/session/generated/requirements.lean` and `/api/session/generated/requirements.proto`, and `/api/session/generated/implementation.lean`. Validation, rendered views, metrics, reports, and OpenAPI descriptions are projections of the same model.

Session isolation is useful during an argument, but the current session files live below `/tmp/leanfm-sessions`. That is a workspace, not durable storage. A concluded argument must export or commit the accepted files—for example as `LeanFM/LLMGenerated/Requirements.lean`, `LeanFM/LLMGenerated/Requirements.proto`, and the separate `LeanFM/LLMGenerated/Implementation.lean`—together with any evidence needed by the project. Durability means that the specification has an ordinary name, history, review boundary, and reproducible checks independent of the chat service.

The server may use a language model to propose edits. Compilation and validation remain deterministic. The model is not the proof checker, and fluent prose is not evidence that an obligation holds.

6.3 Using the agent in Codex

Codex can drive the complete bootstrap, not only the final code-generation step. Open the LeanFM repository as the Codex workspace, or start its command-line interface from the repository root:

```
cd /path/to/leanFM
codex
```

The repository's `AGENTS.md` directs the agent to the LeanFM workflow in `SKILL.md`. Both files make `book/main.pdf` a critical prompt input rather than merely the publication produced by the source in `book/`. The agent must read the book together with the live language references and the existing source tree while developing LeanFM itself. Once that software is operational, the agent uses it to construct and check the durable requirements for the target system. Only accepted

requirements become input to a separately checked implementation specification, and only both accepted specifications authorize generation or revision of application code.

Thus the intended order is:

1. develop and verify the LeanFM tooling described by this book;
2. use the running tooling and the book to interrogate the target system, produce its durable requirements artifacts, and validate their prompt-derived views;
3. after explicit requirements acceptance, produce and validate the traceable implementation specification; and
4. after explicit implementation acceptance, generate or revise the actual application from the book, all three accepted artifacts, and the existing source tree.

A Codex prompt can state this bootstrap explicitly. For example:

```
Treat book/main.pdf as a required system-prompt input, not merely an output of
this repository. Read it with AGENTS.md, SKILL.md, the live LeanFM language
references, and the existing source tree. First develop and verify the LeanFM
software described by the book. Once LeanFM is operational, use it to develop
verified requirements for an order-cancellation service: a customer may cancel
an order until warehouse packing starts. Generate from the specification an
interaction diagram, state machine, cancellation-success rate, and p95
cancellation latency, each retaining its question, event source, reducer, unit,
and boundary. Stop for my explicit acceptance of Requirements.lean,
Requirements.proto, and those derived outputs. Then develop a separately
traceable Implementation.lean and stop for my explicit acceptance. Only after
both gates, implement the application from book/main.pdf, the three accepted
artifacts, and the existing source tree; add tests and run the repository checks.
```

The book remains in the input set throughout this process because it defines the language and method by which the typed artifacts are interpreted. The artifacts then add the target system's accepted facts: observable behavior in `Requirements.lean`, byte-resolving values in `Requirements.proto`, and runtime choices plus traceability in `Implementation.lean`. A person can reject either candidate specification and continue the argument; a successful build does not count as acceptance.

6.4 MCP: the agent-facing entrance

The browser UI is for stakeholders; the Model Context Protocol endpoint is for an LLM agent. LeanFM serves stateless JSON-RPC requests at `POST /mcp`. On initialization, the server tells the agent that the book is durable project context—effectively a maintained section of the system prompt—rather than optional background reading. The agent can enumerate and read fixed resource URIs for `book/main.pdf`, its dark rendering, the canonical requirements, protobuf and implementation files, and both live language references.

This interaction is generated from the implemented MCP dispatch surface:

The first surface is intentionally read-only. It provides resource and prompt discovery, the requirements interrogation, the post-acceptance code-generation prompt, and canonical artifact validation. It exposes no arbitrary pathname and no tool can accept a requirements or implementation gate.

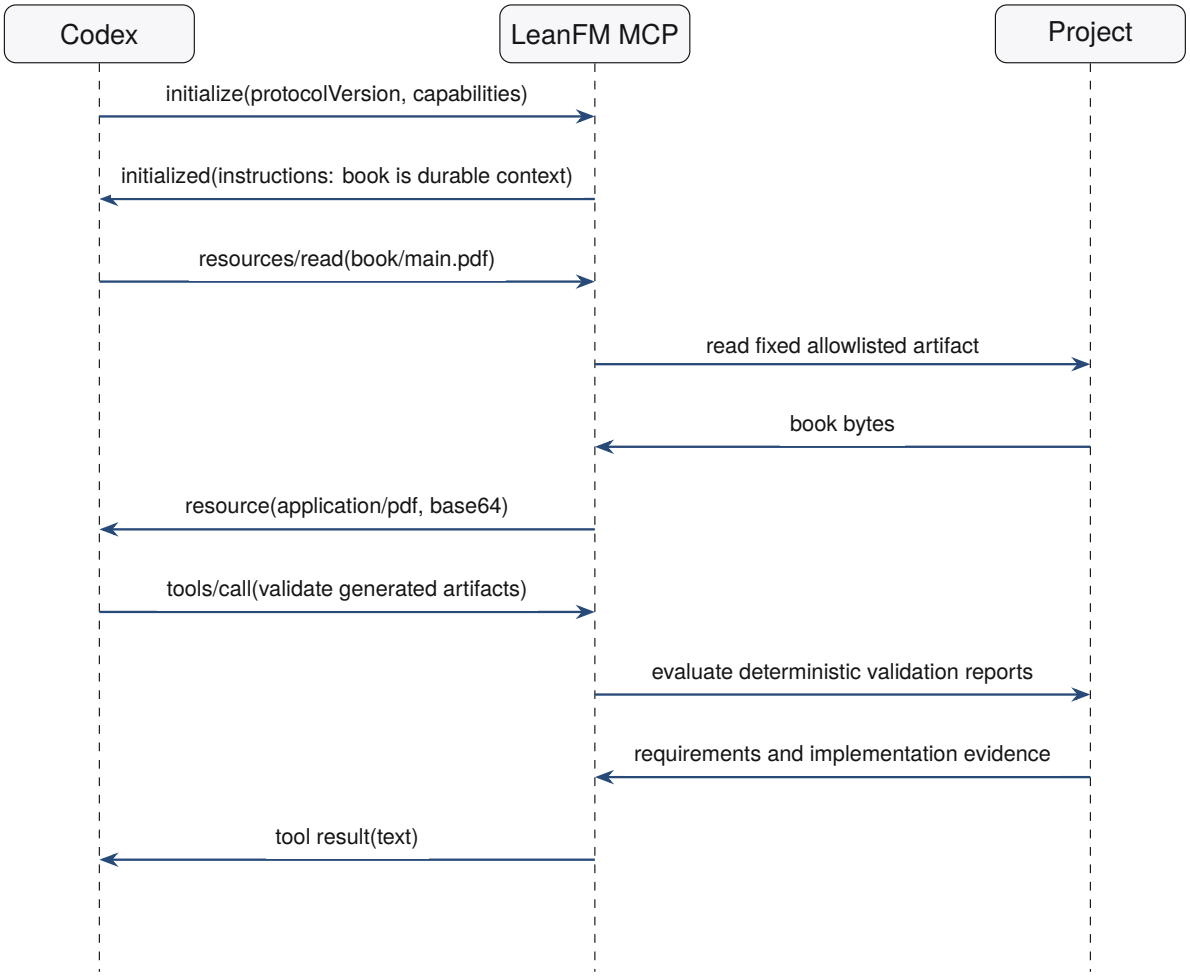


Figure 6.1: An agent obtains durable project context and deterministic evidence through MCP. Source: `LeanFM.MCP`; reducer and provenance: `book/generated/mcp-interaction-provenance.txt`.

Those are stakeholder decisions, not consequences of a successful tool call. MCP transports context and evidence; the existing LeanFM workflow controls revision and acceptance.

The bundled server listens only on loopback. A deployment that exposes MCP to a network must add authentication and authorization at the transport or reverse proxy before treating the resource contents as safe to publish. Statelessness does not make project data public, and a future writable MCP surface must bind every request to a user, project, revision and accepted snapshot.

A local Codex client can register the running endpoint and inspect the result:

```
codex mcp add leanfm --url http://127.0.0.1:8080/mcp
codex mcp list
```

The MCP interface does not replace the book. It makes the book, specifications, and live checker discoverable in a standard agent conversation so that a new session need not reconstruct the method from chat history.

6.5 One project, one directory

Project creation establishes one exclusive filesystem root. Everything owned by that project must be below it. This is both an operational requirement and a reproducibility boundary: deleting that one directory must remove the project without requiring a search for session files, generated diagrams, caches, or logs elsewhere on the machine.

The exact book used as a critical prompt input is part of the project itself. Every project always has `book/main.pdf`; a dark reading copy, `book/main-dark.pdf`, is generated from the same source. The specification paths are stable within the project and recorded by its manifest rather than rediscovered by searching. A representative exported layout is:

```
order-cancellation/
  project.toml
  book/main.tex
  book/main.pdf
  book/main-dark.pdf
  specification/Requirements.lean
  specification/Requirements.proto
  specification/Implementation.lean
  source/
  tests/
  generated/renders/
  generated/metrics/
  evidence/traces/
  evidence/reports/
  .cache/
  logs/
```

There is one canonical accepted `Implementation.lean` for each project revision. Multiple intentional implementation targets may add explicitly named siblings such as `Implementation.server.lean`; each must identify its target and retain complete requirement traceability. They may not become an unindexed collection from which an agent guesses.

The LeanFM repository is itself one such project. Its canonical self-hosting specifications currently live together under `LeanFM/LLMGenerated/`; its manifest and agent instructions make those

stable repository-relative paths explicit. Copying the complete project directory therefore produces a repository candidate that Codex can enter and use without external project-owned state.

The exported directory names may vary only when the manifest states their stable locations; the containment invariant may not. Configured output paths and symbolic links must resolve inside the project root. Temporary files must be created there and atomically renamed there. Export is an explicit copy of a declared artifact, not an unrecorded second project location. Import must copy an input into the root or record a content-addressed, immutable dependency; it must not silently turn an arbitrary external pathname into project state.

Globally installed LeanFM binaries and content-addressed dependency caches may be shared because they are tooling, not state owned by one project. They must not hold mutable project-specific data. A launcher may keep a global recent-project index, but that index is disposable: it must be reconstructible from manifests inside project roots and must not be needed to delete a project completely.

This changes the current staging design. A session directory under `/tmp/leanfm-sessions` and committed artifacts under a separate source tree cannot jointly be the durable project. A conforming implementation chooses the project root first, places session work beneath it, and promotes accepted artifacts by atomic moves within that same root. Project deletion resolves and validates the exact root, refuses filesystem-root or escaping targets, and then removes only that directory.

6.6 What the Lean artifact must settle

The Lean file is the semantic source. It records actors, messages, per-task state machines, multiparty grammar productions, CSP-style processes, framing, properties, proof obligations, probability summaries, charts, and explanatory Markdown. A serious argument should force answers in each of the following dimensions.

Observable behavior

Which actor sends and receives each event? Which fields can an observer see? A message carries its `session` and `task`, its clock timestamp, and a list of prior event identifiers. Thus a join may cite several prior events, while several events may fork from the same prior event.

Component interactions

Global grammar productions state legal conversations; actor projections state what each component may send or receive. Composition must preserve endpoints, message types, correlation, and predecessor references. A diagram is useful evidence, but the grammar is the compositional object.

Statistical properties

User choices make the input probabilistic even when every server component is deterministic. Those choices propagate through the composed system as an MDP. The specification distinguishes unconstrained alternatives from weighted choices, states dwell-time distributions, and names reducers for success, latency, throughput, and queue occupancy. An observed event stream can then estimate the same quantities and be compared with the model.

Queueing and concurrency

Every actor has finite inbound and outbound queues. Several `(session, task)` pairs may be in flight at once, so a requirement must not silently reason about only one conversation.

Full and empty queues produce observable blocking or sleeping behavior, and transport is a separate step from production and receipt.

Memory pressure

Message-count capacity alone is insufficient when payload sizes vary. A checkable requirement should declare a byte budget and derive queued bytes from the encoded envelopes. If active tasks retain additional state, the model must declare a bounded footprint for that state too. A representative obligation is

$$\mathrm{AG}(\mathrm{queuedBytes}(a) + \mathrm{inFlightBytes}(a) \leq \mathrm{budget}(a)).$$

Read this as: "necessarily, at every reachable observation, queued bytes for actor a plus in-flight bytes for actor a is no greater than actor a's byte budget." AG means "on every permitted future, at every position"; the repeated argument a identifies the same actor throughout. This is a modeling obligation for LeanFM, not a claim that the present validator already proves heap usage of an arbitrary implementation.

6.7 Interrogate, characterize, synthesize, replay

The conversational interface is not merely a code generator. Its first job is to discover whether the desired claims are identifiable. It asks for scenario and task boundaries, start-to-end backpointers, concrete actor populations, clock and work units, observation windows, queue and memory limits, outage and recovery events, and the provenance of probabilities. A missing answer is reported as an indeterminate metric rather than replaced by an invented value.

Once those fields are available, one event stream should produce a standard characterization out of the box:

- one interaction diagram and derived state machine per scenario;
- XY line graphs for time-, load-, population-, and replica-indexed metrics;
- pie-chart histograms for categorical outcomes and distributions;
- uptime, outage fraction, reliability, and MTTR;
- client-experienced and server-aggregate throughput; and
- latency distributions and named percentiles.

Some streams contain observations; other requirements contain only expected distributions. These are labeled separately. An expected distribution can drive generation, but it is not presented as a measurement.

Generative similarity closes the loop. Reducers first turn the source stream into a named metric vector, including histogram bins as individual exact ratios. The generator produces a candidate scenario, serializes it to bytes, decodes it, and runs the same reducers. The candidate is accepted only when every required metric lies within its declared relative tolerance. Thus "similar" means measurement-equivalent for the stated characterization, not visually plausible or textually similar. Several very different event processes may satisfy the same finite metric vector; the grammar, invariants, and byte-level language still constrain which candidates are legal.

6.8 The protobuf artifact is part of the requirement

The parallel protobuf schema answers a question that a state diagram cannot: what bytes can cross a component boundary? Atomic payload messages and a `RequirementEnvelope.oneof` discriminator connect a grammar terminal to its wire representation. Validation checks, among other things, that field tags are nonzero and unique and that framing is explicit.

This pairing exposes useful disagreements. A Lean event may require a `priorIds` list while the wire schema provides only one predecessor. A latency property may depend on a clock that is never transmitted or recorded. A memory budget may be stated in bytes while an unbounded protobuf field remains. These are requirements defects, not serialization trivia.

The exact wire size is also an input to resource analysis:

```
queuedBytes actor observation :=
  (observation.inbound actor).sum encodedSize
  + (observation.outbound actor).sum encodedSize

memoryPressure actor observation :=
  queuedBytes actor observation
  + inFlightFootprint actor observation.activeWork
```

The first term can be computed from concrete envelopes. The second requires a declared abstraction of implementation state and must remain visibly labeled as such.

6.9 A ladder of checks

No single green check answers every kind of requirement. The workbench should present a ladder of evidence:

1. Lean elaboration and protobuf parsing establish well-formed artifacts.
2. Structural validation rejects missing actors, endpoints, terminals, framing, terminal states, process coverage, or real proof obligations.
3. Grammar and projected state-machine checks cover ordering and component interaction.
4. Temporal checking covers safety, reachability, and appropriately bounded liveness over observable states.
5. Probability and dwell reducers predict completion probability and latency; event-stream reducers estimate them from observations.
6. Queue and byte-budget checks cover capacity, backpressure, concurrency, and modeled memory pressure.
7. Rendered interaction diagrams, state machines, plots, and reports make the same typed model reviewable by people.

The distinction between model prediction and measurement must remain visible. An incomplete observation window may censor slow tasks; a frequency estimate is not automatically a proof of the declared probability; and a bounded trace does not establish unbounded liveness. Good generated reports say which claim was proved, calculated, sampled, or merely assumed.

6.10 How to argue well

The most useful prompts attack boundaries rather than ask for polish:

- “Show me the observable event that makes that sentence true.”
- “Which alternative ends here, and how is it bracketed in the grammar?”
- “What happens when this inbound or outbound queue is full?”
- “Run two sessions and two tasks concurrently; which events correlate?”
- “Which random choice belongs to the user, and how does it change the downstream completion and latency distributions?”
- “Derive the plot from the event stream, including incomplete tasks.”
- “Show the corresponding protobuf payload and its encoded-size bound.”
- “Is this result proved, model-checked, simulated, measured, or assumed?”

The argument is finished enough to review when deleting the transcript loses no normative information; a clean checkout can compile and validate the artifacts; every diagram and plot is derived rather than independently drawn; failures have observable outcomes; quantitative assumptions have units and scope; and every unsettled claim is an explicit obligation. That is the requirements-centered meaning of vibecoding in LeanFM: conversation discovers the specification, but the checked artifacts outlive the conversation.

Chapter 7

Worked Example: OAuth Authorization Code Flow

This chapter is a behavioural sketch inspired by the OAuth authorization-code flow. It is not a replacement for the OAuth specification and deliberately begins above HTTP and protobuf. Its purpose is to exercise scenario decomposition, parameter flow, visible evidence, alternatives, requirements, and rendering.

7.1 Actors, values, and observable events

The example has four participants: a person using a browser, an application, an authorization service, and a resource service. The browser is modelled separately from the person because redirects are observable messages between software participants.

```
inductive Actor where
  | user | browser | app | authorizationServer | resourceServer

opaque State       : Type
opaque AuthCode    : Type
opaque AccessToken : Type
opaque Resource    : Type
opaque Profile     : Type
opaque Password    : Type
opaque Bytes       : Type
```

Every terminal elaborates to the common event envelope. For example, the vague message “authorization redirected” becomes conceptually:

```
{ timeAt := t2, id := e2, priorIds := [e1]
, session := browserSession, task := oauthAuthorizationTask
, from := Actor.app, to := Actor.browser
, kind := "authorization redirected"
, fields := [field "state" state, field "redirectUri" callback]
}
```

Authentication detail used by Production 2

Before the redirect returns, the authorization service may authenticate the person. OAuth does not prescribe the authentication mechanism; we include a password branch to exercise knowledge flow without pretending that the password is an OAuth protocol parameter.

```
def authenticateAtAuthorizationServer (password : Password) : Scenario :=
  scenario "authenticate resource owner" do
    from_ .browser
    to_ .authorizationServer
    emitBytes "credentials submitted"
      (protectFor .authorizationServer
        (encode [field "password" password]))

    from_ .authorizationServer
    to_ .browser
    oneOf "authentication result"
      [ emitBytes "authentication accepted" (encode [tag 1])
      , emitBytes "authentication rejected" (encode [tag 0])
      ]
```

The wire bytes are specified even though the transport is not. The abstract `protectFor` constructor promises confidentiality to the named endpoint. A later refinement may realize that promise with TLS, message-level encryption, or a local trusted channel. Merely placing the password inside a field would not support the intended knowledge set: without protection, a network observer would also derive the password from the bytes.

7.2 Production 1: application request and redirect

```
def applicationRequestAndRedirect (resource : Resource) : Scenario :=
  scenario "application request and redirect" do
    let state <- declare State "state"
    let callback := Uri.https "app.example" "/oauth/callback"

    from_ .browser
    to_ .app
    emit "application request" [field "resource" resource]

    from_ .app
    to_ .browser
    emit "authorization redirected"
      [field "state" state, field "redirectUri" callback]

    export "state" state
    export "redirectUri" callback
    export "resource" resource
```

This production treats the application request and its redirect as one named unit. It establishes the correlation value `state` and exports it to the next production. Nothing here says how the application stores it.

7.3 Production 2: authorize, then grant or deny

```
def authorizationGranted : Scenario :=
  scenario "authorization granted" do
    use state <- imported State "state"
    let code <- declare AuthCode "code"

    from_ .authorizationServer
    to_ .browser
    emit "authorization code returned"
      [field "state" state, field "code" code]

    from_ .browser
    to_ .app
    emit "authorization callback"
      [field "state" state, field "code" code]
    export "code" code

def authorizationDenied : Scenario :=
  scenario "authorization denied" do
    use state <- imported State "state"
    from_ .authorizationServer
    to_ .browser
    emit "authorization error returned"
      [field "state" state, field "reason" "access_denied"]
    from_ .browser
    to_ .app
    emit "authorization error callback"
      [field "state" state, field "reason" "access_denied"]

def authorizeAndDecide (password : Password) : Scenario :=
  scenario "authorize and decide" do
    use state       <- imported State "state"
    use redirectUri <- imported Uri "redirectUri"

    from_ .browser
    to_ .authorizationServer
    emit "authorization requested"
      [field "state" state, field "redirectUri" redirectUri]

    include (authenticateAtAuthorizationServer password)
    oneOf "authorization decision"
      [ include authorizationGranted
      , include authorizationDenied
      ]
```

The authorization request and its two possible continuations form one production. At a more abstract rendering level, the entire production can be shown as one node; expanding it reveals authentication and the explicitly bracketed choice. The abstract contract makes both decisions possible. A product model may later attach an estimated or measured probability distribution without changing the set of legal traces.

7.4 Scenario 3: exchange the code and use the token

```
def exchangeAndFetch : Scenario :=
  scenario "exchange code and fetch resource" do
    use code     <- imported AuthCode "code"
    use resource <- imported Resource "resource"
    let token    <- declare AccessToken "accessToken"
    let profile  <- declare Profile "profile"

    from_ .app
    to_ .authorizationServer
    emit "token requested" [field "code" code]

    from_ .authorizationServer
    to_ .app
    emit "access token issued" [field "accessToken" token]

    from_ .app
    to_ .resourceServer
    emit "protected resource requested"
      [field "accessToken" token, field "resource" resource]

    from_ .resourceServer
    to_ .app
    emit "protected resource returned"
      [field "resource" resource, field "profile" profile]

    from_ .app
    to_ .browser
    emit "application response" [field "profile" profile]
```

The token itself is visible only at the endpoints where the abstract protocol says it is transmitted. The grammar makes no claim about the authorization server's database schema or the application's session representation.

7.5 Reading the scenarios together

```
def oauthAuthorization (resource : Resource) (password : Password) : Scenario :=
  scenario "OAuth authorization code flow" do
    include (applicationRequestAndRedirect resource)
    include (authorizeAndDecide password)
    after authorizationGranted include exchangeAndFetch

def oauthSystem (resource : Resource) (password : Password) : SystemSpec :=
  generatedBy (oauthAuthorization resource password)
  |> requireAll oauthRequirements
```

The two named productions provide the first abstraction level. A renderer may collapse either production to a single node or expand it to its messages and nested alternatives. The `after` line is a guarded nonterminal reference; the denial branch ends without exchanging a code.

7.6 The parallel protobuf alphabet

The scenario and its wire alphabet are maintained side by side. The scenario says which messages may occur, in what predecessor-order structure, and with which parameter relationships. The protobuf file assigns field numbers and concrete wire types to those same message atoms:

```
syntax = "proto3";

package leanfm.oauth;

enum Actor {
  ACTOR_UNSPECIFIED = 0;
  BROWSER = 1;
  APPLICATION = 2;
  AUTHORIZATION_SERVER = 3;
  RESOURCE_SERVER = 4;
}

message EventEnvelope {
  string event_id = 1;
  repeated string prior_ids = 2;
  string session_id = 3;
  string task_id = 4;
  uint64 clock_tick = 5;
  Actor src = 6;
  Actor dst = 7;

  oneof payload {
    ApplicationRequest application_request = 20;
    AuthorizationRedirect authorization_redirect = 21;
    AuthorizationRequest authorization_request = 22;
    CredentialsSubmitted credentials_submitted = 23;
    AuthenticationResult authentication_result = 24;
    AuthorizationCodeReturned authorization_code_returned = 25;
    AuthorizationCallback authorization_callback = 26;
    AuthorizationErrorReturned authorization_error_returned = 27;
    AuthorizationErrorCallback authorization_error_callback = 28;
    TokenRequest token_request = 29;
    AccessTokenIssued access_token_issued = 30;
    ProtectedResourceRequest protected_resource_request = 31;
    ProtectedResourceReturned protected_resource_returned = 32;
    ApplicationResponse application_response = 33;
  }
}

message ApplicationRequest {
  string resource = 1;
}

message AuthorizationRedirect {
  string state = 1;
  string redirect_uri = 2;
}

message AuthorizationRequest {
  string state = 1;
  string redirect_uri = 2;
}

message CredentialsSubmitted {
  bytes protected_credentials = 1;
}

message AuthenticationResult {
```

```
  bool accepted = 1;
}

message AuthorizationCodeReturned {
  string state = 1;
  bytes code = 2;
}

message AuthorizationCallback {
  string state = 1;
  bytes code = 2;
}

message AuthorizationErrorReturned {
  string state = 1;
  string reason = 2;
}

message AuthorizationErrorCallback {
  string state = 1;
  string reason = 2;
}

message TokenRequest {
  bytes code = 1;
}

message AccessTokenIssued {
  bytes access_token = 1;
}

message ProtectedResourceRequest {
  bytes access_token = 1;
  string resource = 2;
}

message ProtectedResourceReturned {
  string resource = 1;
  bytes profile = 2;
}

message ApplicationResponse {
  bytes profile = 1;
}
```

Each emitted scenario message maps to exactly one member of `EventEnvelope.payload`. The envelope carries the event identity, repeated `prior_ids`, `(session_id, task_id)` correlation key, monotonic clock tick, and endpoints outside the payload. The payload carries only the fields specific to that message.

Protobuf alone does not specify the protocol order, queue behavior, probability, or dwell time; those remain in the scenario. Conversely, the scenario does not invent byte tags. Validation checks that every scenario terminal has one protobuf variant, that its endpoints agree with the scenario, and that every referenced field has the expected protobuf type.

The schema determines valid protobuf encodings, but protobuf does not generally promise one canonical byte sequence. When byte-for-byte comparison is required, the refinement also fixes deterministic serialization and length-delimited message framing. An instantiated envelope can then be serialized and displayed as the actual bytes sent between the named actors.

7.7 Generate one legal trace and its artifacts

We can now run the specification in one direction. Grammar generation chooses productions and concrete field values; everything after that is a deterministic projection or reduction of the resulting events. For this run choose the authentication-accepted and authorization-granted alternatives and instantiate

```
session = "s1"              task = "oauth"
resource = "profile"        state = "x7"
redirectUri = "https://a/cb"
protectedCredentials = aa bb
code = 01 02                accessToken = 03 04
profile = 05 06
```

The production derivation is

```
oauth
=> applicationRequestAndRedirect ;
   authorizeAndDecide ;
   after(authorizationGranted, exchangeAndFetch)
=> applicationRequestAndRedirect ;
   authorizationRequested ;
   credentialsSubmitted ; authenticationAccepted ;
   authorizationGranted ; exchangeAndFetch
```

Because `authorizationGranted` was selected, the `after` guard admits `exchangeAndFetch`. Selecting the denial alternative would disable that production. Expanding every remaining nonterminal yields this terminal word:

The choice has now disappeared into ordinary terminals. Event `e5`'s payload bytes select the accepted authentication result. Event `e6`'s `authorizationCodeReturned` oneof tag, source, and bytes distinguish the grant branch from the error-returned branch. No independent "chooser" value is needed.

7.7.1 Envelope stream and concrete bytes

Assign monotonically increasing clock ticks and event identifiers. This example has a linear predecessor chain, so `priorIds(e1)` is empty and `priorIds(e`n`)` is `[e`$n-1$`]`. A fork or join would make these lists non-linear without changing the procedure. Deterministic protobuf serialization of the schema on the preceding pages produces:

```
# Deterministic protobuf encodings of one legal OAuth trace.
# Columns: event-id, clock, payload alternative, encoded length, envelope bytes.
e1   0  application_request           31  0a0265311a02733122056f6175746830013802a201090a0770726f66696c65
e2   2  authorization_redirect        46  0a026532120265311a02733122056f617574682802300238 01aa01120a027837120c68747470733a2f2f612f6362
e3   4  authorization_request         46  0a026533120265321a02733122056f617574682804300138 03b201120a027837120c68747470733a2f2f612f6362
e4   5  credentials_submitted         32  0a026534120265331a02733122056f6175746828053001380 3ba01040a02aabb
e5   7  authentication_result         30  0a026535120265341a02733122056f617574682807300338 01c201020801
e6   9  authorization_code_returned   36  0a026536120265351a02733122056f6175746828093003380 1ca01080a02783712020102
e7  11  authorization_callback        36  0a026537120265361a02733122056f6175746828 0b30013802d201080a02783712020102
e8  14  token_request                 32  0a026538120265371a02733122056f61757468280e30023803ea01040a020102
e9  16  access_token_issued           32  0a026539120265381a02733122056f6175746828103003380 2f201040a020304
e10 18  protected_resource_request    42  0a0365313012026539 1a02733122056f6175746828123002380 4fa010d0a020304120770726f66696c65
e11 21  protected_resource_returned   43  0a0365313112036531301a02733122056f617574682815300438028020 0d0a0770726f66696c6512020506
e12 23  application_response          34  0a0365313212036531311a02733122056f617574682817300238018a02040a020506
```

These are complete `EventEnvelope` encodings, not illustrative message tags. They were generated with deterministic protobuf serialization. The twelve envelopes occupy 440 bytes; the largest

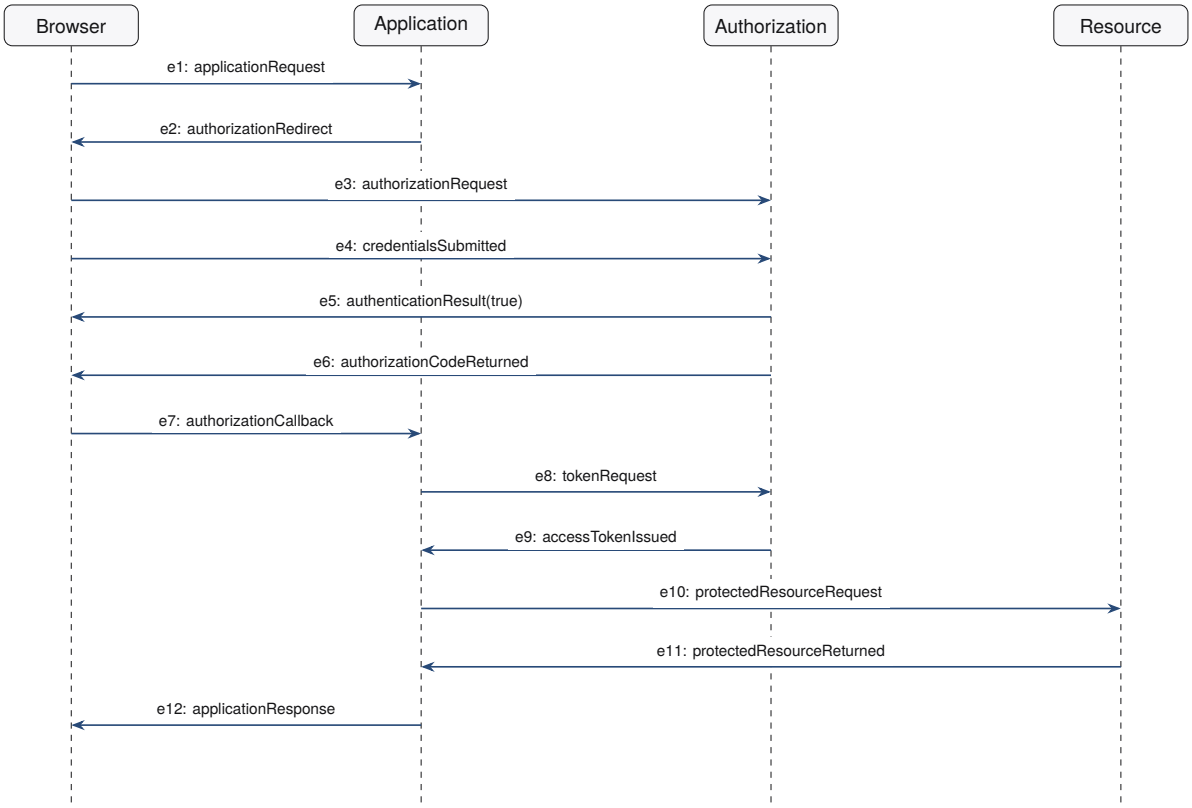


Figure 7.1: The selected terminal word as an interaction diagram. Event IDs label messages on arrows; no actor-to-actor arrow is written as implication-like text.

is 46 bytes. Protobuf deterministic mode makes this fixture reproducible for a given implementation, while the grammar's semantic requirement remains successful decoding to the same terminals and field values.

7.7.2 Event graph with stored backpointers

Parsing each envelope selects a payload alternative, checks its declared `src`/`dst`, and consumes one grammar terminal. For this selected trace, the useful concrete view is the event graph itself:

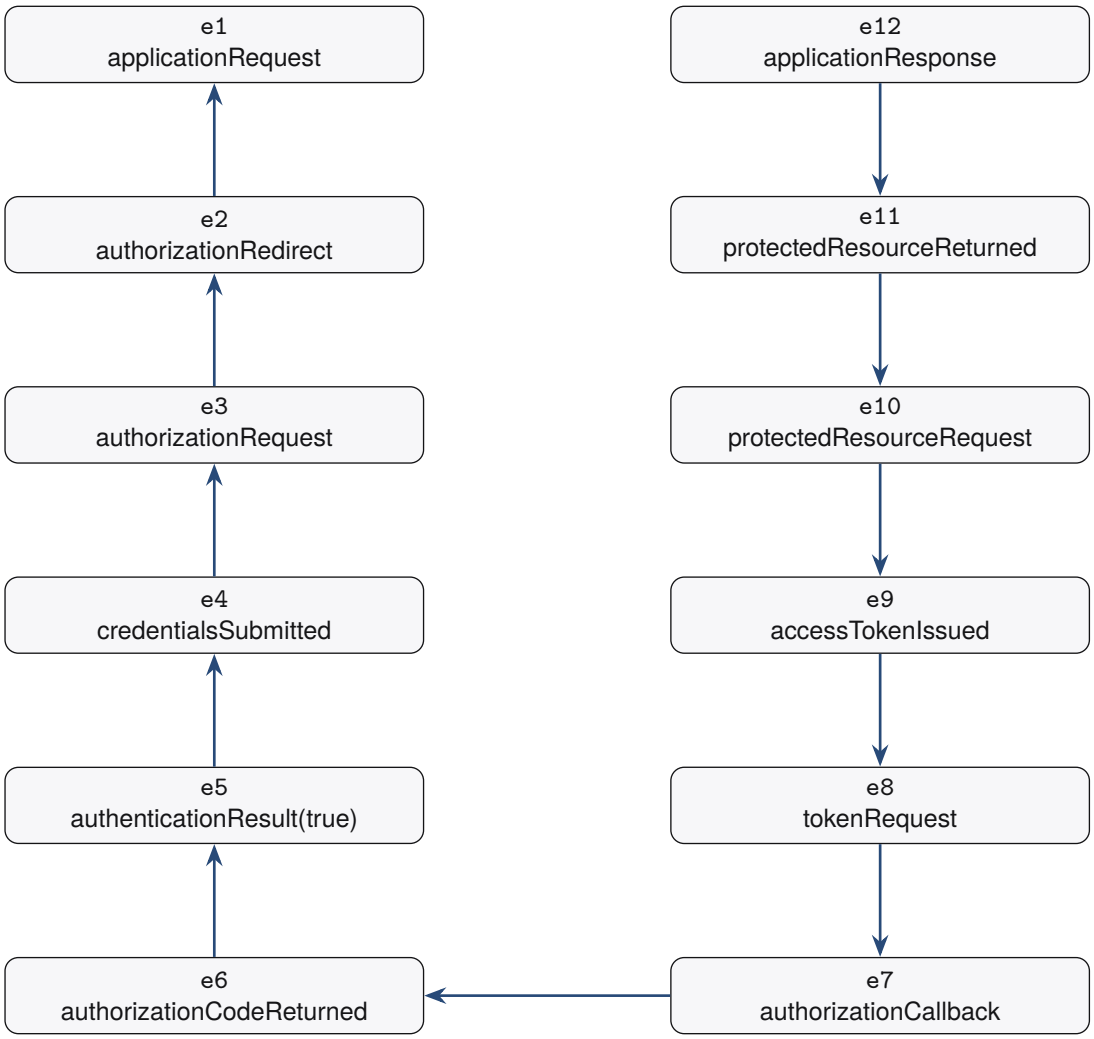


Figure 7.2: The accepted trace as an event graph. Arrows are stored backpointers: `e1` has none and each later event points to its immediate predecessor.

This graph is one selected path. The full residual state machine is derived from every legal grammar residual and also contains the rejected-authentication and denied-authorization branches; it should be rendered as a separate graph when those alternatives are under discussion.

7.7.3 Actor projections

Projection erases terminals an actor neither sends nor receives while preserving the order of the rest. The same word generates four local replay scripts:

Actor	Projected actions
Browser	send e1; receive e2; send e3; send e4; receive e5; receive e6; send e7; receive e12
Application	receive e1; send e2; receive e7; send e8; receive e9; send e10; receive e11; send e12
Authorization	receive e3; receive e4; send e5; send e6; receive e8; send e9
Resource	receive e10; send e11

Each line can become a component conformance test. The global word remains the authority for cross-component order and parameter equality; a local projection does not grant an actor knowledge of erased events.

7.7.4 Event order, queues, and memory

The `priorIds` fields generate a twelve-node order graph with edges $e_i \to e_{i+1}$. The interaction diagram is the same graph laid out by actor; the state-machine path lays it out by grammar residual. Neither is independently authored.

The message word does not uniquely determine a transport schedule. For a concrete queue artifact, choose the canonical schedule that performs, for each event, `produce; transport; receive` before generating the next event. With positive inbound and outbound capacities, each queue peaks at one message. The queued-byte peak for an actor is the largest encoded envelope waiting at that actor in this schedule, and the global queued-byte peak is 46. A different legal interleaving may have a larger peak and must be explored when checking the universal memory-budget property. The selected schedule is a witness, not the capacity proof.

7.7.5 Measurements, plots, and verdicts

From the clocks and correlation key, the generated task attempt is

```
{ key = ("s1", "oauth"),
  started = { kind = applicationRequest, timeAt = 0 },
  completed = { kind = applicationResponse, timeAt = 23 },
  outcome = succeeded, latency = 23 ticks }
```

It contributes one point `(maxConcurrency=1, latency=23)` to the latency XY plot, a cumulative-byte series ending at `(clock=23, bytes=440)`, and one successful slice to the outcome chart. It does *not* establish a 100 percent success probability. The grammar-level weight predicts 0.87 mass on the grant branch; estimating that probability requires many generated or observed sessions, including incomplete ones.

Trace-local monitors accept the matching `state=x7`, the code flow `01 02`, and the fact that no exchange follows denial. CTL obligations such as “every decision reaches the application” are not proved by this trace; the checker must examine every reachable grammar residual. The complete artifact pipeline is thus

```
grammar + field values
  -> legal terminal word
  -> typed EventEnvelope stream
  -> framed protobuf bytes
  -> partial-order graph + interaction diagram
  -> residual FSM path + per-actor projections
  -> queue schedule + byte-pressure samples
  -> task attempt + probability/latency/XY reducers
  -> replay fixture + trace-local property verdicts
```

Every arrow is reproducible. Only the initial branch and value generation, and the separately stated transport schedule, add choices.

7.8 Interaction diagrams by scenario

7.9 Requirements as observable constraints

```
def oauthRequirements : List Requirement :=
  [ require "callback state matches request"
      (on "authorization callback" (previouslySame "state"
        "authorization requested"))

  , forbid "code exchange after denial"
      (previously "authorization denied" && event "token requested")

  , require "issued token came from a code"
      (on "access token issued" (previouslySame "code" "token requested"))

  , ctl "every decision reaches the application"
      (AG (decisionAtAuthorizationServer --> AF callbackAtApplication))

  , ctl "a protected result is never returned without a token request"
      (AG (resourceReturned --> previously tokenRequested))
  ]
```

7.9.1 Who knows the password?

Observable bytes are not the same as knowledge of their plaintext. Rather than introducing “secrecy” as a special property, calculate the actors that can derive the password from initial knowledge and the event list.

```
def passwordKnowers (trace : Trace WireEvent) : List Actor :=
  allActors.filter fun actor =>
    Derives (knowledgeFrom actor trace) (value password)

theorem oauth_password_knowers
    (trace : Trace WireEvent)
    (accepted : (oauthSystem resource password).Accepts trace)
```

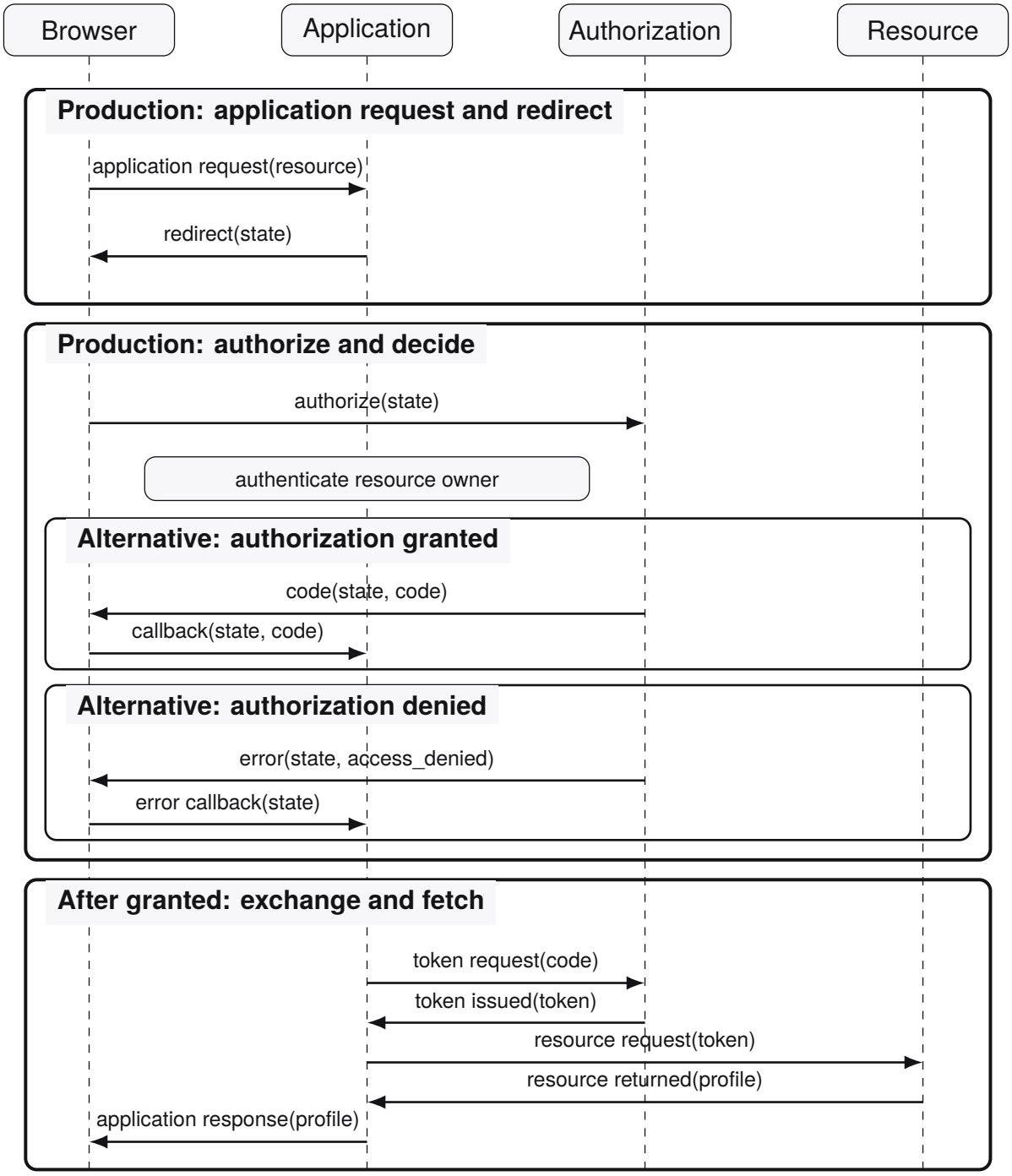


Figure 7.3: OAuth interaction view. Production frames provide collapsible abstraction boundaries. The authorization production contains its request and both alternatives; the guarded exchange production is enabled only by the granted alternative.

```
    (initial : InitiallyKnown password [.browser, .authorizationServer])
    (honest : Uncompromised [.browser, .authorizationServer])
    (protection : PerfectProtection protectFor) :
    passwordKnowers trace = [.browser, .authorizationServer] := by
  exact knowledgeClosureOfProtectedField accepted initial honest protection
```

The result says directly who knows the value. A password typed into the modeled browser is derivable by that browser; hiding this behind the word “secret” would lose information. If a narrower result is required, the actor model must represent a trusted credential component separately. The calculation remains conditional: a compromised endpoint, logging event, newly visible key, or failed protection refinement changes the knowledge closure and therefore changes the returned set.

7.9.2 Byte-level refinement without choosing a transport

Every vague OAuth message can be assigned a canonical byte term before HTTP or protobuf is chosen:

```
refineMessage "authorization requested" where
  term := tuple
    [ tagBytes 0x01
    , namedBytes "state" (encode state)
    , namedBytes "redirectUri" (encode callback)
    ]
  bytes := encodeCanonical term

refineMessage "token requested" where
  term := tuple [tagBytes 0x02, namedBytes "code" (encode code)]
  bytes := encodeCanonical term
```

This fixes framing, tags, and value relationships at an abstract byte-algebra level. A subsequent HTTP mapping might put those bytes into form fields; a protobuf mapping might use numbered fields. Either concrete alphabet must prove that decoding and re-encoding preserve the abstract terminal and its parameter bindings.

The use of `previously` is deliberately a trace predicate in this first version. It can later receive dedicated past-time syntax without changing the event model.

7.10 Probabilities and charts

The authorization service need not be random. The user supplies random input by granting or denying consent; the browser, queues, transport, and authorization service then react deterministically to the input they receive:

```
def consentDecision : Scenario :=
  weightedOneOf "consent decision"
    [ weighted (87 / 100) authorizationGranted
    , weighted (13 / 100) authorizationDenied ]

def oauthCharts : List ChartSpec :=
  [ pie "authorization outcomes" do
      source events; groupBy terminalOutcome; value count
```

```
  , line "end-to-end authorization latency" do
      source (correlate events by "state")
      x clock; y (elapsed "authorization requested" "application response")
  ]
```

No separate field says that the user "owns" this choice. The grant and denial branches resolve to different consent-event bytes, and that terminal's source identifies the user-facing actor that emitted them. After those bytes select a branch, every transition may have probability one and the global model is still probabilistic. The grant branch carries probability mass 0.87 through the outbound queue, transport, the server's inbound queue, and the deterministic grant response. The denial branch similarly carries mass 0.13. Deterministic processing transports probability mass; it does not remove it.

For the simplified path with no loss or timeout,

$$P(\text{grant callback}) = 0.87 \cdot 1 \cdot 1 \cdot 1 = 0.87, \qquad P(\text{denial callback}) = 0.13.$$

Read these as: "the probability of a grant callback equals 0.87 times three deterministic factors of one, hence 0.87; the probability of a denial callback equals 0.13." P denotes probability, and the parenthesized text names the event whose probability is measured. The factors of one are deterministic enqueue, transport, and server steps. Their dwell times still affect the latency distribution even though they do not split the probability mass.

With queue scheduling and other unresolved choices included, the complete system is therefore a Markov decision process. Probabilistic edges describe inputs or measured outcomes; nondeterministic edges describe decisions not assigned a distribution, such as which enabled actor is scheduled next. An analysis first fixes or quantifies over a scheduling policy and then propagates the user-originated mass through the resulting paths.

For several sessions in flight, the MDP state contains every actor's bounded queues and its per-`(session, task)` residuals. One user's random event can change later queue pressure and scheduling possibilities for other sessions, so the probabilistic state is the composed system state rather than a special marker attached only to the user actor.

The percentages remain model inputs or measurements, not truths inferred from the grammar. The grammar defines which behaviours are possible; the probabilistic refinement says how mass is distributed over them.

7.11 Residual state machine

This automaton is intentionally behavioural. It does not contain browser cookies, database rows, server threads, or cryptographic library state. Those details may be essential to an implementation proof, but they are refinements of this contract, not the contract itself.

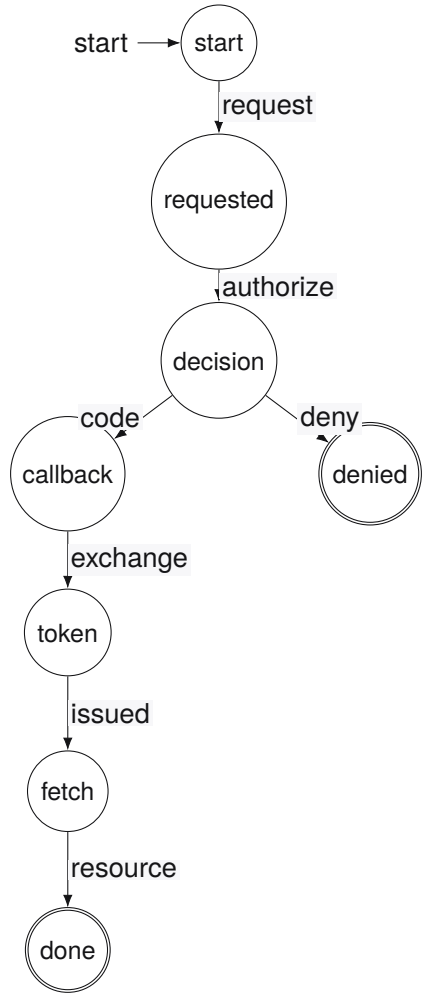


Figure 7.4: OAuth state machine derived from legal grammar continuations.

Chapter 8

CTL over Generated Futures

A grammar generates a branching transition system. CTL speaks directly about that branching, but its vocabulary becomes clearer when placed between basic modal logic and linear temporal logic. Path quantifiers distinguish *there exists a future* from *every future*, while temporal operators distinguish the next state, eventuality, invariance, and until.

The author's modal-logic background used here was learned mostly from Kröger and Merz [5]. This citation identifies the source of that education; it is not a claim that the authors originated modal or temporal logic.

8.1 Basic modal logic

Propositional logic evaluates a formula at one world or state. Basic modal logic adds an accessibility relation R between worlds and two operators:

$$
\begin{aligned}
w \models \Box p \quad &\text{iff} \quad p \text{ holds at every } v \text{ such that } wRv, \\
w \models \Diamond p \quad &\text{iff} \quad p \text{ holds at some } v \text{ such that } wRv.
\end{aligned}
$$

Read the first line as: "world w satisfies box p if and only if p holds at every world v for which w is related to v by accessibility relation R." Read the second with "diamond" and "some world v" in place of "box" and "every world v." The readings are "necessarily p" and "possibly p." Under classical negation they are dual: $\Diamond p \equiv \neg\Box\neg p$. The symbols do not fix the meaning of accessibility. Depending on the model, an accessible world may be a next computation state, a world compatible with an actor's knowledge, a normatively permitted world, or something else.

The basic normal modal logic K allows arbitrary Kripke frames. Additional frame conditions validate additional principles: reflexivity, transitivity, symmetry, and the Euclidean condition lead to familiar stronger systems. Those principles must not be imported merely because the English word "necessarily" sounds strong. LeanFM uses a directed computation-successor relation and does not silently treat it as reflexive, symmetric, or epistemic.

8.2 LTL: time along one path

Linear Temporal Logic (LTL) describes one linear path with operators including X (next), F (eventually), G (always), and U (until). Thus Fp says that p occurs at some suffix of the selected path, while Gp says that p holds at every suffix of that path.

LTL's temporal words answer *where along this path*; they do not themselves choose between one possible branch and every branch of a transition system. A model-checking convention often asks whether every initial path satisfies an LTL formula. That outer convention should not be confused with a path quantifier in the LTL syntax. For example, asking whether some legal execution eventually reaches recovery and asking whether all legal executions eventually reach recovery require different branching claims even though both contain the words "eventually recovery."

LeanFM does not currently implement LTL. It uses CTL because the requirements deliberately combine two independent axes:

- *always* or *eventually* says where along forward time a predicate must hold; and
- *necessarily* or *possibly* says whether it must hold on every permitted forward continuation or may hold on at least one.

This choice arose from trying to combine temporal logic with alethic logic while investigating whether all the familiar modal families could be combined. The construction retained by LeanFM is specifically *temporally alethic*: possible means possible now or at some point along a legal future. Ordinary language uses "always" both for universal modality and for temporal persistence. CTL keeps those coordinates visible long enough to prevent the ambiguity from disappearing inside an English gloss.

8.3 CTL: path modality combined with time

CTL pairs a path quantifier with each temporal operator. E is existential: there is at least one permitted continuation. A is universal: every permitted continuation satisfies the temporal condition. In the vocabulary of modal logic, E supplies temporal possibility and A supplies temporal necessity.

"Necessary" and "possible" are therefore future-relative readings here. They quantify over computations from the current observation, not over timeless logical worlds. The four most important combinations can be read literally:

$EF\,p$: possibly eventually p,	$AF\,p$: necessarily eventually p,
$EG\,p$: p always on some future,	$AG\,p$: necessarily always p.

Read these as: "there exists a future on which eventually p"; "on every future, eventually p"; "there exists a future on which p holds globally, at every position"; and "on every future, p holds globally." Thus E is "there exists a future," A is "every future," F is "eventually," and G is "at every position."

The temporal word and modal word do different work. F versus G chooses eventuality versus invariance *along* a continuation. E versus A chooses some versus all permitted continuations. Neither axis should be omitted when stating a branching-time requirement.

For requirements argument, the four formulas also form a useful modal square:

$EF\,p$: maybe possible p, $AG\,p \equiv \neg EF\,\neg p$: always necessary p,
$AF\,p$: always eventually p, $EG\,p \equiv \neg AF\,\neg p$: p always in some cases.

Read each equivalence sign as "if and only if." $EF\,p$ is "maybe possible p." $AG\,p$ is "always necessary p," equivalently "not: there exists a future that eventually reaches not-p." $AF\,p$ is "always eventually p." $EG\,p$ is "p always in some cases," equivalently "not: every future eventually reaches not-p."

These are exact CTL dualities, not a demand that the English glosses look syntactically dual. In particular, AF is "on every legal future, eventually p"—always eventually. EG is harder to name compactly: it means that some legal future exists on which p holds at every position—p always in some cases. The formulas expose two meanings often hidden by "always": A ranges over all legal futures, while G ranges over all positions along one future. Calling necessity "always" without naming the coordinate loses information precisely where a requirements argument needs it.

The dualities are also computationally central. LeanFM need not establish a universal claim by enumerating every complete future. It searches for an existential counterexample:

$$AG\,p \equiv \neg EF\,\neg p, \qquad\qquad AF\,p \equiv \neg EG\,\neg p.$$

Read these as: "$AG\,p$ if and only if not EF not-p," and "$AF\,p$ if and only if not EG not-p." Each negation applies to the expression immediately following it.

Thus a safety proof searches for a reachable violation of p, and a universal eventuality proof searches for a path on which $\neg p$ persists forever. A counterexample is a witness; absence of such a witness in the finite checked model establishes the corresponding universal result. Keeping the exact CTL form is what permits this reduction from universal checking to existential witness search through negation.

8.3.1 Least upper bounds and greatest lower bounds

Eventuality and strong until choose the least solution of their recursive equations. Over the lattice of state sets, begin with the empty set and repeatedly add states justified by the previous approximation:

$$\mathrm{Sat}(EF\,p) = \mu Z.(P \cup \mathrm{Pre}_{\exists}(Z)) = \bigcup_{n \geq 0} Z_n, \qquad Z_0 = \varnothing.$$

Read this as: "the states satisfying $EF\,p$ are the least fixed point of $Z = P$ union the states with some successor in Z. Starting with no states, the union of the successive approximations is their least upper bound." The same construction gives existential until:

$$\mathrm{Sat}(p\ EU\ q) = \mu Z.(Q \cup (P \cap \mathrm{Pre}_{\exists}(Z))).$$

Read this as: "include states satisfying q; then repeatedly include a state satisfying p when at least one successor is already included." AF and strong AU are also least fixed points, using universal predecessor. A cycle that never reaches the target therefore adds nothing to the least solution.

Invariance and weak until make the dual choice. They begin with all candidate states and remove unsupported states, producing the greatest fixed point—the greatest lower bound of descending approximations:

$$\mathrm{Sat}(EG\,p) = \nu Z.(P \cap \mathrm{Pre}_{\exists}(Z)) = \bigcap_{n \geq 0} Y_n.$$

Read this as: "the states satisfying $EG\,p$ are the greatest fixed point of $Z = P$ intersect the states with some successor in Z. Starting with all candidate states, the intersection of successive approximations is their greatest lower bound."

LeanFM implements these choices with cycle-aware searches rather than by materializing every approximation set. An unresolved revisit is false for `EF`, `EU`, `AF`, and strong `AU`, selecting the least fixed point. A predicate-preserving revisit is true for `EG` and weak until, selecting the greatest fixed point. `AG` is checked as $\neg EF\,\neg p$, and `AF` as $\neg EG\,\neg p$. Thus least upper bound and greatest lower bound are paired implementation choices; neither describes every CTL operator by itself.

An observation is immutable once recorded. A value changes only by moving forward to a successor observation; it is never updated retroactively inside the current state. This does not mean that values are monotone—a queue may grow and later shrink—but every change occurs on a forward transition.

Formula	Reading	Typical use
$EF\,p$	possibly, eventually p	temporal possibility / witness
$AF\,p$	necessarily, eventually p	termination / response
$EG\,p$	possibly, always p	viable invariant path
$AG\,p$	necessarily, always p	safety invariant
$EX\,p$	possibly, next p	enabled successor / one-step witness
$AX\,p$	necessarily, next p	required one-step consequence
$(p\ AU\ q)$	every path keeps p until q	constrained progress
$(p\ EU\ q)$	some path keeps p until q	possible progress
$(p\ AW\ q)$	every path: p until q, or p forever	necessary weak until
$(p\ EW\ q)$	some path: p until q, or p forever	possible weak until

For example, $AF\,p$ is not merely "eventually p" and $EF\,p$ is not a weaker spelling of the same linear claim. The first requires p on every legal future generated from the current trace prefix; the second needs one legal future as a witness. Branching is part of the requirement.

Strict CTL requires every temporal operator to be governed by A or E. CTL* is a more general branching-time language in which state formulas and path formulas can be nested more freely; LeanFM does not implement CTL*. This matters when translating prose: not every arbitrary alternation of "on all paths" and "eventually" is a formula of the small CTL datatype used here.

8.4 Projecting to LTL-style F and G

Requirements often want the simpler LTL surface forms Fp and Gp. LeanFM may display or accept those forms only with an explicit path-policy boundary. On a selected linear execution, the branching coordinate has already been fixed, so AFp and EFp both project to Fp, while AGp and EGp both

project to Gp. Under the common model-checking convention that an LTL formula must hold on every initial path, Fp corresponds to AFp and Gp corresponds to AGp.

Consequently, presenting frequently used AF and EG obligations as F and G is a deliberate interface reduction, not a CTL equivalence on a branching model. The durable artifact retains the exact originating CTL formula so the checker can still use existential witnesses, negation, and the correct counterexample search.

8.5 Persistence and recurrence

Nested CTL operators express two common requirements:

$$AF\,AG\,p, \qquad AG\,AF\,p.$$

Read the first, symbol by symbol: "On every path, eventually reach a state from which, on every path, p always holds." This is eventual permanent safety: for example, eventually finish key rotation and never again accept the old key. Read the second: "On every path, at every state, on every path from that state, eventually p holds." This is universal recurrence, often pronounced "always eventually p": for example, service keeps returning to a ready state.

For witness searches their duals are:

$$\neg(AF\,AG\,p) \equiv EG\,EF\,\neg p, \qquad \neg(AG\,AF\,p) \equiv EF\,EG\,\neg p.$$

Read: "Failure of eventual permanent safety means some path always retains some future reaching not-p. Failure of universal recurrence means some future reaches a state with some path on which not-p holds forever."

The corresponding existential forms are $EF\,EG\,p$ and $EG\,EF\,p$. Read: "Some path eventually reaches a state with some path keeping p forever," and "some path always visits states from which some path eventually reaches p." In the second formula those eventual witnesses may leave the outer path; it does not require infinitely many occurrences of p on that same path.

These forms cover useful LTL-style persistence and recurrence, not all LTL. Under universal LTL path semantics, GFp corresponds to $AGAFp$ on the usual total transition model. But $AFAGp$ is stronger than pathwise FGp: it requires eventually reaching a state where *all* continuations stay safe, not merely a separate stabilization point on each path. Terminal states require the same explicit continuation convention used by the core checker.

8.6 Knowledge, obligation, and permission

General modal logic need not have only one accessibility relation. A multimodal model can use $\Box_t/\Diamond_t$ for temporal reachability, $K_a/\widehat{K_a}$ for what actor a knows or considers possible, and a different pair for permission or obligation. Dynamic logic can index modalities by programs. Tense logics can add past-directed relations. These systems share the Kripke-style idea of evaluating modal formulas through accessibility, but their modalities are not interchangeable.

LeanFM's CTL A and E range only over legal forward computations generated by the current model. They do not mean:

- probability one versus positive probability;

- known versus merely believed by an actor;
- obligatory versus permitted by a policy; or
- physically necessary versus logically possible.

Knowledge and policy relations are documented modelling techniques, not built-in operators. Projects may define them explicitly using the relations below. The Kerberos video [7] illustrates their use; it does not verify the protocol.

For knowledge, let W be the declared set of legal histories and $\mathrm{obs}_a(h)$ actor a's local observation of history h:

$$K_a\varphi(h) \iff \forall h' \in W.\ (\mathrm{obs}_a(h) = \mathrm{obs}_a(h') \Rightarrow \varphi(h')).$$

Read: "Actor a knows φ at history h if and only if, for every history h' in W, if a observes the same thing in h and h', then φ holds at h'." Observations must declare visible messages, private keys, clock information, and retained memory. With the current history included in W, knowledge is factive: what is known is true in that model. Belief needs a different relation; it must not inherit this guarantee accidentally.

$$\widehat{K}_a\varphi \equiv \neg K_a \neg \varphi.$$

Read: "a considers φ possible exactly when a does not know that φ is false." This is epistemic possibility, not temporal reachability.

For policy, let $\mathrm{Ideal}_a(h)$ be the declared policy-compliant futures from h, and let ψ be a condition on a whole future path:

$$O_a\psi(h) \iff \forall \pi \in \mathrm{Ideal}_a(h).\ \psi(\pi), \qquad P_a\psi(h) \iff \exists \pi \in \mathrm{Ideal}_a(h).\ \psi(\pi).$$

Read: "a is required to satisfy ψ at h exactly when every ideal future π satisfies ψ; a is permitted to satisfy ψ exactly when some ideal future satisfies ψ." Require a nonempty set of ideal futures: contradictory policies must be reported, not certified by vacuous quantification. Actual legal executions may violate policy; detecting that violation is part of the analysis. These are obligations under a named policy, not judgments of morality.

$$P_a\psi \equiv \neg O_a \neg \psi.$$

Read: "ψ is permitted exactly when its negation is not required." This duality applies only when both operators use the same declared policy relation. For example:

$$O_a F\ \mathrm{audit}(r)(h) \iff \forall \pi \in \mathrm{Ideal}_a(h).\ \exists i \geq 0.\ \mathrm{audit}(r, \pi_i).$$

Read: "a must eventually audit request r at h exactly when, for every ideal future π, there is a position i, zero or later, where r is audited." Here F is a path operator; a project implementation must distinguish state formulas from path conditions.

Temporal, epistemic, and policy quantification can then be combined without confusing their domains. A legal future may be possible, unknown to Alice, and forbidden by policy at the same time. Probability, belief, strategic ability, and program-indexed modalities remain separate design work; adding named relations does not implement every modal logic.

Past predicates remain useful for security: a ticket must have been issued before it is accepted. Compile bounded past conditions into forward monitor state where an equivalence can be established. Evaluate "previously issued" before updating the monitor with the current event. On concurrent

histories, declare whether "before" means causal ancestry or a chosen linearization; screen order is not evidence. Unbounded past logic is not thereby implemented.

Symbolic uncompromised-key assumptions do not establish computational cryptographic security. General knowledge and policy checking is deferred.

8.7 Possibility means preparedness

There is a practical consequence to treating possibility as a first-class temporal mode. When a requirement declares $EF\,p$—"maybe possible p"—it does more than acknowledge that a model checker can find an accidental path to p. It says that p is a supported future for which the system must prepare.

A `possibly` obligation therefore contains three linked artifacts:

1. a legal grammar branch and witness trace reaching the condition;
2. a handling plan in `Requirements.lean`; and
3. generated implementation code whose justification cites that possibility requirement.

For example, "a bakery delivery may fail" requires a decodable `DeliveryFailed` message, a legal failure branch, and code that performs the specified refund or notification behavior. Removing every failure branch does not implement reliability; it makes the real failure trace out-of-spec.

Possibility is distinct from probability. A possible condition may have a small estimated probability, an expected distribution, or no known probability at all. Conversely, assigning positive probability to a condition makes its branch possible. The handling obligation comes from supported possibility, not from whether the estimated percentage crosses a threshold.

The separate operational obligation applies to $EG\,p$—"p always in some cases." It identifies at least one future on which p persists. The system must pay attention to the conditions that keep that future viable: monitor them, expose their loss, and, where required, control choices that would destroy the invariant path. $EG\,p$ alone does not promise that every execution preserves p; that stronger claim is $AG\,p$.

8.8 Strong implication and the Chewbacca argument

The name "Chewbacca defense" is a cultural reference to the *South Park* episode "Chef Aid," whose courtroom parody uses an irrelevant claim about Chewbacca to distract from the conclusion at issue. The reference supplies a memorable example of non sequitur; it is not a technical term originating in modal or temporal logic.

Material implication is weak in the precise sense that $p \to q$ is true whenever p is false. That is useful inside a state, but it permits a vacuous global claim when no reachable state ever satisfies the antecedent. LeanFM names the non-vacuous temporal combination

$$p \Rightarrow_s q \quad \stackrel{\text{def}}{=} \quad EF\,p \;\wedge\; AG(p \to q).$$

Read this as: "p strongly implies q is defined to mean: there exists a future eventually witnessing p, and on every future at every position, ordinary material implication from p to q holds." The subscript s names this strong, non-vacuous implication; the equals sign topped by 'def' means "is defined as."

The first conjunct requires a legal future witnessing p. The second requires that, necessarily always, every reachable occurrence of p also satisfies q. Together they imply $EF\,q$: the p witness cannot avoid q.

The deliberately absurd test uses

$$p = \text{"Chewbacca lives on Endor"},$$
$$q = \text{"you must acquit"}.$$

Read these as definitions of proposition names: "p means 'Chewbacca lives on Endor'," and "q means 'you must acquit'."

A Chewbacca argument cannot obtain q merely by mentioning p. To enter the requirements as strong implication, the model must admit a reachable p witness and establish $AG(p \to q)$ over every permitted continuation. If the observable grammar supplies no relation between residence and acquittal, that second obligation has no justification and is rejected. Logic cannot determine the meaning or relevance of arbitrary English propositions by itself; the useful point is that neither a missing antecedent nor a single rhetorical occurrence of it can make the formal implication pass vacuously.

In Lean the definition is direct:

```
def stronglyImplies (p q : CTL S) : CTL S :=
  CTL.and (CTL.ef p) (CTL.ag (CTL.implies p q))
```

The path quantifier is part of the until operator. We write `AU` and `EU` as binary algebraic operators, not an outer `A` or `E` wrapped around a bracketed until formula. Consequently a subexpression never changes meaning merely because another expression surrounds it:

```
def boundedUntilTerminal : CTL Observation :=
  atom withinCapacity AU atom isTerminal

def someAuthorizedContinuation : CTL Observation :=
  atom hasValidAuth EU atom isSucceeded
```

8.9 Weak until: AW and EW

Strong until requires the right operand eventually to occur. Weak until permits it never to occur, provided the left operand continues to hold. Thus

$p\ AW\ q$: on every future, p holds until q, or p remains forever,
$p\ EW\ q$: on some future, p holds until q, or p remains forever.

Read the first line as: “on every future, p holds until q, or if q never occurs, p remains forever.” Read the second with “on some future.” AW and EW are the universal-path and existential-path weak-until operators.

The boundary state satisfying q need not also satisfy p; p is required only before q. On a future where q never occurs, every state must satisfy p. This makes weak until useful for requirements such as “necessarily remain available until shutdown, while shutdown is permitted but not required.”

For existential paths, $p\,EW\,q$ is equivalent to $(p\,EU\,q) \vee EG\,p$. The analogous-looking $(p\,AU\,q) \vee AG\,p$ is *not* a correct definition of AW on a branching system. Some futures may reach q while other futures preserve p forever; neither global disjunct need hold even though every individual future satisfies weak until. LeanFM therefore implements `AW` directly.

```
def serviceMayRunUntilShutdown : CTL Observation :=
  atom available AW atom shutdown

def someWorkerCanRemainReady : CTL Observation :=
  atom ready EW atom retired
```

This convention also makes tree structure explicit. In `r && (p AU q)`, the right operand is the complete `AU` expression; there is no context-sensitive reinterpretation of `[p U q]`.

For a request event carrying identifier r, an expected response property might say that on every continuation a matching success or failure eventually occurs. Authentication might say that every successful use of a session has an earlier issuance event with the same session identifier.

8.10 Past claims

CTL is future-facing at the current state, but security and protocol claims often refer to evidence in the past: “this capability was previously issued” or “this response matches an earlier request.” Since states are trace prefixes, past facts can initially be expressed as predicates over the accumulated trace. This gives a clear baseline without prematurely committing the surface language to a particular past-time temporal logic.

There is still an open design question. Explicit past operators may make properties more readable and allow better checking algorithms. The project should compare trace predicates, CTL with past operators, and compilation of finite-history monitors into state. The criterion is maintainability of specifications as the event schema evolves.

8.11 Fairness and time bounds

AF does not by itself distinguish a participant that is perpetually denied a turn from one that violates the protocol. Fairness assumptions must therefore be named, scoped, and visible in theorem statements. Similarly, “eventually” is not “within 200 milliseconds.” Quantitative bounds belong in an explicit timed layer rather than being smuggled into timestamps and informal prose.

Chapter 9

Lean as the Rendered Language

The user-facing notation should render to a conservative subset of Lean. There is no second opaque specification file: domain declarations are Lean definitions, expected behaviours are Lean values, and requirements are Lean propositions or checked queries.

Throughout this book, every displayed mathematical formula is accompanied by a literal English reading that accounts for each operator, index, bound, and new glyph. The reading is part of the notation discipline: it must map back to the symbols one-to-one rather than merely paraphrase the formula's intent.

```
def checkout : Grammar CheckoutEvent :=
  request >>> (accepted <|> rejected)

def eventuallyAnswered : CTL Observation :=
  CTL.ag (CTL.implies requestPending
    (CTL.af (CTL.or requestAccepted requestRejected)))
```

Most semantic machinery should be imported from a fixed library. A system author normally supplies only:

- typed actors, identifiers, event kinds, and payloads;
- parameterised grammar definitions;
- projections from trace prefixes to observable propositions;
- named temporal requirements and proofs or decidable checks.

This boundary matters for automation. Generated Lean remains small enough to review, while library proofs and algorithms are reused rather than regenerated.

9.1 A Ramanujan-inspired checked discrete account of $1+2+3+\cdots = -1/12$

Lean is not limited to validating data structures or executing finite checks. This section starts from the less formal result associated with Srinivasa Ramanujan. What is different here is not the value $-1/12$, but the deliberately elementary route used to make that value precise and checkable. The

construction does not invoke the Riemann zeta function, analytic continuation, complex analysis, or a theorem identifying $\zeta(-1)$. Its final proof is exact rational algebra over explicitly defined finite prefixes and correction tails.

The repository's `book/logic.lean` uses Mathlib to define the real function $A(x) = 1/(1-x)$ and prove that its derivative is $F(x) = 1/(1-x)^2$ away from the singularity at $x = 1$. It then moves from that analytic motivation to an exact, discrete construction over rational numbers. This construction explains precisely what the file means by the famous shorthand

$$S = 1 + 2 + 3 + \cdots = -\frac{1}{12}.$$

Read this as: "S, defined by the regularized expression one plus two plus three and so on, equals negative one twelfth." The qualification "regularized" is essential; the later formula defines what the ellipsis means here.

The derivative establishes only the origin of the chosen rational function F; it is not an appeal to zeta regularization. After that motivation, the definitions of `Tail_B`, `Sum_B`, `Tail_S`, and `Sum_S`, and the proof of the final identity, use discrete identities and rational arithmetic. Thus the checked claim stands on the definitions printed here rather than importing a higher-level theory that already assigns a value to the divergent series.

The complete source file follows, without elision. The listing starts on this page and continues onto the next page through the final theorem `Goal`. The listing preserves the idiomatic Lean notation used by the checked source, including Lean's $\wedge$ notation; the listing configuration maps those Unicode glyphs to matching mathematical symbols for pdfLaTeX:

Listing 9.1: The complete `book/logic.lean` source.

```
import Mathlib

/- Ramanujan-inspired motivation for the choice of F, using a real derivative.
   This is not a Riemann-zeta or analytic-continuation construction.  The exact
   result below is discrete rational algebra over defined prefixes and tails. -/
noncomputable def Ar (x : ℝ) : ℝ := 1 / (1 - x)
noncomputable def Fr (x : ℝ) : ℝ := 1 / (1 - x)^2
def F (x : ℤ) : ℚ                := 1 / (1 - x)^2

theorem deriv_Ar_eq_Fr (x : ℝ) (hx : x ≠ 1) :
    deriv Ar x = Fr x := by
  have h : 1 - x ≠ 0 := sub_ne_zero.mpr hx.symm
  have hd            := (((hasDerivAt_id x).const_sub 1).inv h).deriv
  unfold Ar Fr
  simpa only [one_div, id_eq, neg_neg] using hd

theorem F_eq_Fr (x : ℤ) :
    ((F x : ℚ) : ℝ) = Fr (x : ℝ) := by
  unfold F Fr
  push_cast
  rfl

/- define B from F -/
def Tail_B (n : ℕ) : ℚ := ((2 * (n : ℚ) + 1) * (-1)^n) / 4
def Sum_B (n : ℕ) : ℚ  := F (-1) - Tail_B n
def Bf (n : ℕ) : ℚ     := (Sum_B + Tail_B) n
def B : ℚ              := Bf 0

/- define S. Calculate (S - B) -/
def Sum_S (n : ℕ) : ℚ  := n * (n + 1) / 2
def Tail_S (n : ℕ) : ℚ := (4 * Sum_S n - (Sum_S (2*n) + Tail_B (2*n))) / 3 - Sum_S n
def Sf (n : ℕ) : ℚ     := Sum_S n + Tail_S n
```

```
def S : ℚ              := Sf 0

theorem selfSimilarS (n : ℕ) :
    (Sum_S (2*n) - Sum_B (2*n) = 4*Sum_S n) ∧
    (Tail_S (2*n) - Tail_B (2*n) = 4*Tail_S n) ∧
    (S - B = 4*S) := by
  have finite :
      Sum_S (2*n) - Sum_B (2*n) = 4*Sum_S n := by
    unfold Sum_S Sum_B Tail_B F
    simp
    ring_nf
  have tails :
      Tail_S (2*n) - Tail_B (2*n) = 4*Tail_S n := by
    unfold Tail_S Sum_S Tail_B
    simp
    ring_nf
  have splitS (k : ℕ) : S = Sum_S k + Tail_S k := by
    unfold S Sf Tail_S Sum_S Tail_B
    simp
    ring_nf
  have splitB (k : ℕ) : B = Sum_B k + Tail_B k := by
    unfold B Bf Sum_B
    simp
  refine ⟨finite, tails, ?_⟩
  calc
    S - B =
        (Sum_S (2*n) + Tail_S (2*n)) -
        (Sum_B (2*n) + Tail_B (2*n)) := by
      rw [splitS (2*n), splitB (2*n)]
    _ = (Sum_S (2*n) - Sum_B (2*n)) +
        (Tail_S (2*n) - Tail_B (2*n)) := by ring
    _ = 4*Sum_S n + 4*Tail_S n := by
      rw [finite, tails]
    _ = 4*(Sum_S n + Tail_S n) := by ring
    _ = 4*S := by rw [splitS n]

theorem whyTailS (n : ℕ) :
    Tail_S n =
      (4*Sum_S n - (Sum_S (2*n) + Tail_B (2*n)))/3
        - Sum_S n := by
  have split (k : ℕ) : Tail_S k = S - Sum_S k := by
    unfold S Sf Tail_S Sum_S Tail_B
    simp
    ring_nf
  have hn := split n
  have h2n := split (2*n)
  have derivation :
      (Tail_S (2*n) - Tail_B (2*n) = 4*Tail_S n) ↔
      (Tail_S n =
        (4*Sum_S n - (Sum_S (2*n) + Tail_B (2*n)))/3
          - Sum_S n) := by
    calc
      (Tail_S (2*n) - Tail_B (2*n) = 4*Tail_S n)
        ↔ (4*(S - Sum_S n) =
            S - (Sum_S (2*n) + Tail_B (2*n))) := by
          constructor <;> intro h <;> linarith
      _ ↔ (3*S =
            4*Sum_S n - (Sum_S (2*n) + Tail_B (2*n))) := by
          constructor <;> intro h <;> linarith
      _ ↔ (Tail_S n =
            (4*Sum_S n - (Sum_S (2*n) + Tail_B (2*n)))/3
              - Sum_S n) := by
          constructor <;> intro h <;> linarith
  exact derivation.mp (selfSimilarS n).2.1

theorem whyTailB (n : ℕ) :
```

```
  Tail_B 0 = F (-1) ∧
  Tail_B (n+1) =
    Tail_B n - (n+1 : ℚ) * (-1 : ℚ)^n := by
  unfold Tail_B F
  simp [pow_succ]
  ring_nf
  simp

theorem Goal (n : ℕ) :
  -1/12 = n*(n+1)/2 + Tail_S n := by
  unfold Tail_S Sum_S Tail_B
  simp
  ring_nf
```

The hypothesis `hx : x != 1` is not commentary: it is required to prove that the denominator is nonzero. The proof composes Mathlib's derivative rule for the identity function with subtraction and reciprocal, extracts the proved derivative, unfolds the local definitions, and asks the simplifier to match the resulting expression. Lean checks every type conversion and proof obligation; removing the singularity condition leaves the reciprocal rule unjustified.

The discrete part begins with the ordinary finite triangular sum

$$\texttt{Sum_S}(n) = \frac{n(n+1)}{2}.$$

Read this as: "`Sum_S` at cutoff n equals n times $n+1$, divided by two." Here n is a natural-number parameter naming the finite cutoff. It then defines an exact rational correction `Tail_S`(n), using the alternating construction `Tail_B`, so that the corrected value is independent of where the finite prefix is cut. The last theorem states, for *every* natural number n,

$$-\frac{1}{12} = \frac{n(n+1)}{2} + \texttt{Tail_S}(n).$$

Read this as: "negative one twelfth equals n times $n+1$, divided by two, plus `Tail_S` at that same cutoff n." Thus the ellipsis in $1 + 2 + 3 + \cdots$ is given explicit content: retain a finite prefix and pair it with the particular tail assigned by the preceding self-similarity identities. Moving the cutoff changes both terms but not their sum. This is a discrete, algebraically checked account of the regularized value $-1/12$; it is not the false claim that the ordinary partial sums of positive integers converge to a negative number. Lean makes that distinction hard to blur because the exact proposition—including the tail and the quantification over n—is visible in Listing 9.1.

At the end, Lean's goal is the fully elaborated proposition

```
⊢ ∀ (n : ℕ), -1 / 12 = ↑n * (↑n + 1) / 2 + Tail_S n
```

Read the Lean goal one symbol at a time as: "prove, for every n of type natural number, that negative one divided by twelve equals coerced n times coerced n plus one, divided by two, plus `Tail_S` applied to n." The upward arrows are coercions: they convert the natural number n into the rational-number type used by the arithmetic around it.

The single turnstile $\vdash$, pronounced "proves" or "derives," separates the local proof context on its left from the proposition to be derived on its right. A context may contain assumptions, variables, local definitions, and previously established facts. Here nothing is printed to the left, so there are no additional local entries: Lean is asking for a term that derives the universally quantified proposition from the imported theory and the definitions above.

For a LeanFM runtime judgment, define the double turnstile $\models$ relative to the declared transition model and initial state. It means "at every reachable state, the premises require at least one of these conclusions." Write it in a rule-like form, familiar from Prolog:

$$a \wedge b \wedge \cdots \wedge w \;\models\; x \vee \cdots \vee z.$$

Its executable definition is that the following CTL formula holds at the declared initial state:

$$AG\big((a \wedge b \wedge \cdots \wedge w) \rightarrow (x \vee \cdots \vee z)\big).$$

Read: "at every reachable state, if a and b and so on through w all hold, then x or so on through z holds." A means every continuation; G means every position along it, including the current state. This is entailment within the declared runtime model, not a redefinition of general entailment across every possible model. The model and initial state must accompany the judgment. The $\wedge$ symbols collect premises with "and"; the $\vee$ symbols collect alternative conclusions with "or." The conclusion need not be the same in every state.

For code, represent the left side as a list of premises and the right side as a list of conclusions. A counterexample is a reachable state where every premise is true and every conclusion is false:

```
counterexample(state, premises, conclusions) :=
  premises.all(p => evaluate(p, state)) &&
  !conclusions.any(q => evaluate(q, state))

entails(successors, initialState, premises, conclusions) :=
  !anyReachable(successors, initialState,
    state => counterexample(state, premises, conclusions))
```

The evaluator and successor relation come from the declared model. A sampled trace checks only that sample, not all reachable states.

The same counterexample condition can be constructed symbolically:

$$\neg EF(a \wedge b \wedge \cdots \wedge w \wedge \neg x \wedge \cdots \wedge \neg z).$$

Read: "there is no reachable state with a and b and so on through w, and not x and so on through not z." This is exactly the AG implication above, using $AG\,p = \neg EF\,\neg p$, read "always p is no reachable not-p."

With a single conclusion, the analogy is the Prolog rule `x :- a, b, ..., w`: x follows when all its premises hold. The displayed disjunctive conclusion is more general than an ordinary single-head Prolog rule; $\models$ does not specify Prolog's execution or search order. The compact notation $\Gamma \models p$, read "Gamma semantically entails p," collects the premises into a context Γ and names the conclusion p. The distinction remains: $\Gamma \vdash p$, read "Gamma derives p," asserts a derivation. Our runtime judgment checks that no reachable state makes all the premises true and p false. General semantic entailment uses all models of the assumptions instead; it must not be confused with this explicitly scoped CTL test.

For the proof-theoretic background behind derivations, natural deduction, sequent calculus, normalization, and cut elimination, see Mancosu, Galvan, and Zach's *An Introduction to Proof Theory* [8].

The completed proof establishes the displayed proposition for every n. The `def` declarations define the mathematics: the finite sums, correction tails, and the values assembled from them. The `theorem` declarations then state claims about those definitions and require proofs that Lean checks. Thus the final theorem is not a numerical experiment over a selection of cutoffs. It verifies the universally quantified identity that results from the definitions.

9.2 From specification to service

Lean can host executable code, including a small web server. That makes the formal artifact part of the running project rather than a document beside it. The same typed definitions can drive validation endpoints, test-trace generation, protocol schemas, visualisations, and runtime monitors. Extraction and integration choices must preserve the distinction between a verified behavioural boundary and an unverified infrastructure layer.

Chapter 10

Scope and Open Questions

The implementation backlog and audit findings live in `CRITIQUE.md`. This chapter distinguishes current evidence from open technical questions.

Self-hosting consolidation

The canonical requirements entry point now composes the existing LeanFM ChatUI specification with a proposed revision-bound requirements-to-PR workflow. The Worker protocol remains an example, not a specification of the whole product. `make selfhost-check` validates the typed composition and regenerates `SELF_HOSTING_REVIEW.md`, including its interaction diagram and state machine. A component registry and fresh-session handoff preserve decisions without requiring conversation history.

The consolidation proposes checker outcomes, acceptance invalidation, probability interpretations and PR evidence. Earlier proposals also cover modal indexes; built-in knowledge and policy checking is now deferred. These records are not implemented features or proofs. Error protocols, formula semantics and complete traceability remain prerequisites for a complete one-shot specification.

10.1 Proof obligations

The initial metatheory should make the following questions precise:

- Does a projected local behaviour faithfully implement its part of the global grammar?
- Under what finiteness restrictions is CTL checking decidable and executable?
- Which guards can be compiled into finite monitor state?
- When does modular composition preserve an established temporal property?
- Which fairness and timing assumptions are required by a liveness theorem?
- Which observation and policy relations justify each knowledge, permission, and obligation assertion, and which bounds make their combination with CTL executable?

The intended outcome is not merely a verifier. It is a development loop in which writing the behaviour is the fastest way to obtain working, reviewable software.

Part II

Security Protocol Exercises from the CSP Approach

Chapter 11

From CSP Roles to Multiparty Grammars

This part revisits examples from Ryan et al. [12] and its official Casper companion scripts. The goal is not to transliterate CSP operators. It is to use well-known security protocols as demanding exercises for the grammar-based method: multiple roles, fresh values, encrypted components, an active intruder, calculated knowledge sets, agreement, and messages whose legality depends on the preceding trace.

11.1 Can actors really be specified separately?

Yes, provided composition is defined as synchronization over the same observable events rather than as concatenation of independent scripts. Each actor accepts a language over its own projection of the global event alphabet.

```
structure RoleSpec where
  actor    : Actor
  accepts  : Grammar (ProjectedEvent actor)
  initialKnowledge : Knowledge actor

def composeRoles (roles : List RoleSpec) : GlobalGrammar :=
  synchronize roles where
    sendReceiveAgree := sameEventId && sameBytes && oppositeEndpoints
```

For a global trace t, let $t\restriction_a$ retain events sent or received by actor a. The composition accepts exactly when every local role accepts its view:

$$L(\mathrm{composeRoles}(R_1, \ldots, R_n)) = \{t \mid \forall i,\ t\restriction_{a_i} \in L(R_i)\}.$$

Read this as: “the language L accepted by the composition of roles R_1 through R_n is the set of global traces t such that, for every role index i, trace t restricted to actor a_i belongs to the language accepted by role R_i.” Here n is the number of roles, i is one role’s index, the vertical bar means “such that,” and $\restriction$ means “restricted to.”

One global message is therefore not duplicated into unrelated send and receive events. It has a single identity, byte string, sender, and recipient; both endpoint grammars constrain it. An unmatched send, an unexpected receive, or disagreement about bytes makes the intersection empty at that trace prefix.

11.1.1 Local state without global implementation state

A role may bind parameters from the events it observes. Those bindings are a parser environment for the trace, not a claim about implementation memory. A role can say “the nonce in this reply equals the nonce I previously sent” without specifying a table, variable layout, or retry mechanism.

11.2 A transport-neutral byte algebra

Security claims need more than message names. They need an algebra that says which bytes expose which values. We still need not choose HTTP, protobuf, ASN.1, or a particular cipher suite.

```
abbrev Bytes := ByteArray

inductive WireTerm where
  | atom    : Name -> Bytes -> WireTerm
  | tuple   : List WireTerm -> WireTerm
  | encrypt : KeyRef -> WireTerm -> WireTerm
  | sign    : KeyRef -> WireTerm -> WireTerm

structure WireEvent where
  timeAt : ClockTimestamp
  id     : EventId
  priorIds : List EventId
  session : SessionId
  task   : TaskId
  from   : Actor
  to     : Actor
  term   : WireTerm
  bytes  : Bytes
  encodedBy : bytes = encodeCanonical term
```

At this level, encryption is a symbolic constructor with an encoding into bytes. The intruder may concatenate, split visible tuples, encrypt with known keys, and decrypt only with a derivable inverse key. A concrete refinement must replace this idealized algebra with a transport and cryptographic implementation whose security assumptions justify the abstraction.

11.3 Yahalom as three separate actors

Yahalom is an excellent first exercise because an initiator, responder, and trusted server exchange fresh nonces and receive a session key. The companion model states secrecy for the session key and agreement claims involving the nonces and key.

Let K_A and K_B be long-term keys shared by the server with Alice and Bob, N_A and N_B fresh nonces, and K_{AB} the proposed session key. The following actor grammars use symbolic wire terms whose canonical encodings are byte strings.

11.3.1 Alice, the initiator

```
def aliceRole (alice bob server : Actor) (ka : SharedKey) : RoleSpec :=
  role alice "Yahalom initiator" do
    let na <- fresh Nonce "na"

    from_ alice; to_ bob
    send (atom "initiator nonce" na.bytes)

    from_ server; to_ alice
    receive (encrypt ka.ref
      (tuple [actor bob, sessionKey "kab", nonce na, nonce "nb"]))
      where_ field "na" == na
    let kab <- take SessionKey "kab"
    let nb  <- take Nonce "nb"
    let ticketForBob <- take WireTerm "ticketForBob"

    from_ alice; to_ bob
    send ticketForBob
    send (encrypt kab.ref (nonce nb))
```

Alice's `where_` clause is the context-sensitive step: the received nonce must equal the value generated earlier. The surface structure is context-free, but legality depends on the binding environment accumulated from the prefix.

11.3.2 Bob, the responder

```
def bobRole (alice bob server : Actor) (kb : SharedKey) : RoleSpec :=
  role bob "Yahalom responder" do
    from_ alice; to_ bob
    receive (atom "initiator nonce" ?naBytes)
    let na <- take Nonce "na"
    let nb <- fresh Nonce "nb"

    from_ bob; to_ server
    send (encrypt kb.ref
      (tuple [actor alice, nonce na, nonce nb]))

    from_ alice; to_ bob
    receive (encrypt kb.ref (tuple [actor alice, sessionKey "kab"]))
    let kab <- take SessionKey "kab"
    receive (encrypt kab.ref (nonce nb)) where_ field "nb" == nb
```

Bob need not know Alice's local control state. He constrains only the events visible at his boundary and the values learned from them.

11.3.3 The trusted server

```
def serverRole (alice bob server : Actor)
    (ka kb : SharedKey) : RoleSpec :=
  role server "Yahalom key server" do
    from_ bob; to_ server
```

```
    receive (encrypt kb.ref
      (tuple [actor alice, nonce "na", nonce "nb"]))
    let na  <- take Nonce "na"
    let nb  <- take Nonce "nb"
    let kab <- fresh SessionKey "kab"

    let bobTicket := encrypt kb.ref
      (tuple [actor alice, sessionKey kab])
    from_ server; to_ alice
    sendMany
      [ encrypt ka.ref
          (tuple [actor bob, sessionKey kab, nonce na, nonce nb])
      , bobTicket
      ]
```

11.3.4 Global assembly and context conditions

```
def yahalom : GlobalGrammar :=
  composeRoles
    [ aliceRole .alice .bob .server keyAlice
    , bobRole .alice .bob .server keyBob
    , serverRole .alice .bob .server keyAlice keyBob
    , intruderRole .ivo
    ]
  |> where_ (distinct [.alice, .bob, .server, .ivo])
  |> where_ (freshValuesNeverRepeat ["na", "nb", "kab"])
  |> where_ (longTermKeysKnownOnlyToOwnersAndServer)
```

The three honest roles can be authored and checked separately. Composition exposes incompatibilities: for example, if Alice expects the server's first encrypted tuple to name Bob but the server grammar could name another responder, the shared global event cannot satisfy both projections.

11.4 The attacker is another grammar plus deduction

An active network attacker observes public wire events, blocks them, replays known bytes, and emits any term derivable from current knowledge. It does not learn the plaintext of an encryption merely because its encoded bytes are observable.

```
inductive Derives : Knowledge -> WireTerm -> Prop where
  | initial : term in knowledge -> Derives knowledge term
  | tuple   : (forall x in xs, Derives knowledge x) ->
              Derives knowledge (.tuple xs)
  | project : Derives knowledge (.tuple xs) -> x in xs ->
              Derives knowledge x
  | encrypt : Derives knowledge key -> Derives knowledge body ->
              Derives knowledge (.encrypt key body)
  | decrypt : Derives knowledge (.encrypt key body) ->
              Derives knowledge (inverse key) -> Derives knowledge body
```

```
def intruderRole (ivo : Actor) : RoleSpec :=
  adversary ivo do
    observe publicWireEvents
    mayBlock
    mayReplay
    maySend fun knowledge term => Derives knowledge term
```

This is the grammar analogue of the CSP/Dolev–Yao attacker: possible hostile messages are generated from observable history and constrained by a context-sensitive derivability relation.

11.5 Knowledge and agreement become theorems

Knowledge is calculated for every actor from every accepted trace, not merely the happy-path scenario. A traditional secrecy claim is recovered by comparing that set with the actors permitted to know the value. For Yahalom:

```
theorem yahalom_session_key_knowers
    (trace : Trace WireEvent)
    (accepted : yahalom.Accepts trace)
    (honest : Uncompromised [.alice, .bob, .server]) :
    knowers (sessionKey accepted.kab) trace = [.alice, .bob, .server] := by
  -- proof uses the fixed symbolic-crypto library
  exact knowledgeClosureFromKeyOwnership accepted honest

theorem yahalom_bob_agrees_with_alice
    (trace : Trace WireEvent)
    (accepted : yahalom.Accepts trace)
    (bobDone : completed trace .bob) :
    exists run, peer run == .alice &&
      run.na == bobDone.na && run.nb == bobDone.nb := by
  exact agreementFromNonceChallenge accepted bobDone
```

The exact proof scripts depend on the library still to be built. The important design constraint is already visible: theorem statements refer only to accepted event traces, knowledge derived from those traces, and explicit compromise assumptions.

11.6 Otway–Rees: guards are not optional decoration

The Otway–Rees companion model introduces a run marker M and guards that the two participants differ. Each participant encrypts its nonce, marker, and actor names under its long-term server key. The server returns separate encrypted components.

This example adds three useful `where_` obligations:

```
where_ alice != bob
where_ response.field "marker" == request.field "marker"
where_ encryptedPart.names == [alice, bob]
```

The grammar shape remains a small sequence of messages. The accepted language is not context-free once equality of an unbounded marker and actor parameters matters. Keeping these

tests as named guards preserves the readable protocol skeleton while making the extra context explicit.

The example also asks whether an observer can distinguish executions after one candidate session key is substituted for another. That cannot be answered by a single knowledge set: it is a relational property over pairs of trace distributions, beyond a single CTL state formula, and should be represented as a separate theorem family.

11.7 Needham–Schroeder: public-key lookup as behaviour

The seven-message public-key example makes key discovery observable. Alice requests Bob's public key from a server, receives a signed binding, sends an encrypted nonce and identity to Bob, and later answers Bob's nonce. Bob performs the symmetric key lookup for Alice.

Actor separation is particularly valuable here:

- the key server grammar promises signed actor–public-key bindings;
- Alice's grammar accepts Bob's key only where the signature verifies and the named actor is Bob;
- Bob imposes the corresponding constraint for Alice;
- the nonce-response messages reuse previously bound values;
- the intruder grammar may substitute any derivable public key or replay any signed binding, but cannot manufacture a server signature.

```
receive (sign serverSigningKey (tuple [actor bob, publicKey "pkb"]))
  where_ verifiesWith serverPublicKey
  where_ field "actor" == bob

send (encrypt pkb (tuple [nonce na, actor alice]))

receive (encrypt aliceSecretView (tuple [nonce na, nonce "nb"]))
  where_ field "na" == na
```

This demonstrates why "all behaviour is observable" must not mean "all plaintext is public." The globally recorded event exposes endpoints, timing, kind, and wire bytes. Which symbolic values an actor can recover from those bytes is governed by the deduction relation.

11.8 What these exercises demand from the library

The three case studies produce a concrete implementation checklist:

1. actor-local grammar definitions and trace projection;
2. synchronization by common event identity and bytes;
3. typed symbolic variables, exports, imports, and prefix-dependent guards;

4. a transport-neutral term-to-byte relation;
5. explicit initial knowledge and compromise assumptions;
6. attacker closure under symbolic construction and decryption;
7. per-value knowledge sets, agreement, and relational indistinguishability theorem schemas;
8. counterexamples rendered simultaneously as a global interaction diagram and as each actor's local projection.

These are not security-specific embellishments. They are a rigorous test of whether multiparty context-sensitive grammars can remain modular while expressing the information flow on which real behavioural claims depend.

Part III

Machine Reference

Chapter 12

The LeanFM Web-Server Protocol

This chapter is the protocol reference for the LeanFM web server. It is primarily machine-oriented context for an implementation agent and may be read as a maintained portion of the system prompt. The normative behavioral source is `LeanFM/ChatUI/Requirements.lean`; the byte vocabulary is `LeanFM/ChatUI/Requirements.proto`; and transport selection is `LeanFM/ChatUI/Implementation.lean`. The tables and grammars below are generated from those files. They are not a second, hand-maintained API design.

Two versions of the protocol must not be confused. The *accepted protocol* is the complete typed grammar and intended HTTP mapping. The *running adapter* is the subset currently dispatched by `LeanFM/ChatUI/Server.lean`. A route in the accepted table is not a claim that its handler already exists.

12.1 One exchange across four representations

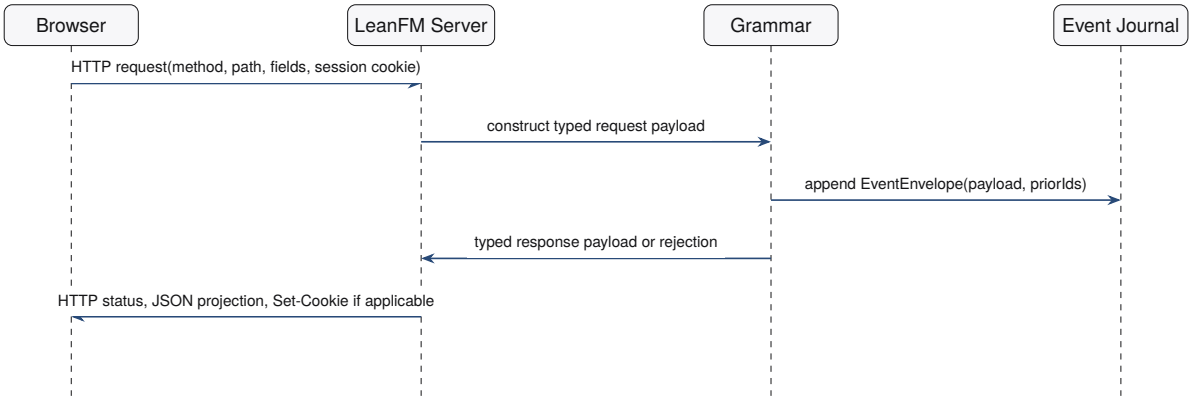


Figure 12.1: HTTP is an adapter around the typed LeanFM conversation. Source, reducer, question, and unit are recorded in `generated/web-protocol-provenance.txt`.

Read the figure as five transformations:

1. HTTP supplies a method, path, headers, cookies, and body octets.

2. The adapter authenticates the session, matches the method and path template, percent-decodes fields, and constructs one named protobuf request value.

3. That message may advance only the task grammar that names it. A legal accepted event is wrapped in an `EventEnvelope` carrying event, predecessor, session, task, time, source, destination, and exactly one `payload` alternative.

4. The selected task produces one of the response alternatives associated with the request. Internal provider messages remain in-process messages; they are not invented HTTP calls merely because the outer request used HTTP.

5. The adapter selects an HTTP status and projects the response fields to JSON. Session establishment additionally emits a secure cookie.

The protobuf value and HTTP request are therefore not the same bytes. Protobuf defines the typed event and its stable field tags. HTTP selects how a browser presents or receives that value. The current browser adapter uses `application/x-www-form-urlencoded` for implemented POST requests and JSON for responses. The project event journal is separately specified as length-delimited binary `EventEnvelope` values. A future binary HTTP adapter could use `application/protobuf` without changing the grammar, provided it preserves the same message value and correlation fields.

12.2 The conversion, field by field

For a request binding `httpRequest(method, pathTemplate)`, conversion is defined in this order:

1. Match the HTTP method exactly and split the path into decoded segments.

2. For each `{field}` in the template, take the protobuf field of that name, encode it as one path segment, and reject slashes, traversal, or a value that escapes the authenticated project root.

3. For the currently implemented POST subset, encode remaining browser input fields as UTF-8 form values. A protobuf `bytes` field is opaque bytes at the protocol boundary; a form adapter must use an explicitly declared textual encoding and must never place credentials in a URL, trace, diagnostic, or returned object.

4. Recover the authenticated user from the `leanfm_session` HttpOnly, SameSite cookie. The cookie is transport authorization, not a field silently added to every protobuf message. Its resolved user and session identifiers constrain which project-scoped message may be constructed.

5. Validate required fields and task `where_` constraints. On success, assign the envelope metadata and the oneof payload. On failure, emit the named rejection alternative when the grammar has one; otherwise return an HTTP error without pretending a successful protocol event occurred.

For a response binding `httpResponse(RequestName)`, conversion reverses the projection. The request name selects the finite legal response set. Scalar fields become JSON strings, booleans, or numbers; repeated fields become JSON arrays; nested messages become objects; and `bytes` become an explicitly identified download or Base64 field rather than accidental UTF-8. Secrets are

redacted before JSON rendering. Success commonly uses 200, creation uses 201, authentication failure uses 401 or 403, invalid input uses 400, missing owned objects use 404, revision or credential conflicts use 409, and provider failure uses 502. The named response message, not the status alone, determines the grammar transition.

12.3 The running adapter

The following table is the implemented ChatUI subset. Static assets, diagnostic reports, generated source downloads, MCP, and health checks are document resources or tool protocols outside the ChatUI task grammar; they must not be mistaken for user-task protobuf events.

POST `/api/accounts`
Request: AccountCreateSubmitted. Response: AccountCreated or AccountCreateRejected. Encoding: application/x-www-form-urlencoded.

POST `/api/login`
Request: LoginSubmitted. Response: LoginAccepted or LoginRejected. Encoding: application/x-www-form-urlencoded.

GET `/api/me`
Request: authenticated session. Response: user identity JSON. Encoding: cookie only.

GET `/api/projects`
Request: ProjectListRequested. Response: ProjectListReturned. Encoding: cookie plus path.

POST `/api/projects`
Request: ProjectCreateRequested. Response: ProjectCreated. Encoding: application/x-www-form-urlencoded.

GET `/api/projects/{project_id}/conversation`
Request: project identifier. Response: conversation JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/tasks`
Request: project identifier. Response: task model JSON. Encoding: path field plus cookie.

GET `/api/projects/{project_id}/artifacts`
Request: ArtifactListRequested. Response: ArtifactListReturned. Encoding: path field plus cookie.

POST `/api/projects/{project_id}/chat`
Request: ChatPromptSubmitted. Response: ChatResponseRendered or ChatTurnFailed. Encoding: application/x-www-form-urlencoded.

Every `/api/` route except public account creation and login requires a valid in-memory session. Project paths additionally resolve the authenticated owner's project root. A missing session yields 401; an unknown or unauthorized project is deliberately represented as 404. The server binds to loopback by default. Network exposure requires an authenticating reverse proxy or an equivalent transport boundary.

12.4 The accepted HTTP projection

This is the complete intended browser mapping from the accepted implementation specification. Rows absent from the running-adapter table remain implementation obligations.

POST `/api/login`
Request: `LoginSubmitted`. Response: `LoginAccepted`, `LoginRejected`.

POST `/api/accounts`
Request: `AccountCreateSubmitted`. Response: `AccountCreated`, `AccountCreateRejected`.

GET `/api/projects/{project_id}/tasks/{task_id}`
Request: `TaskModelRequested`. Response: `TaskModelReturned`.

GET `/api/projects/{project_id}/tasks/{task_id}/sessions/{session_id}/events`
Request: `MessageSequenceRequested`. Response: `MessageSequenceReturned`.

GET `/api/projects/{project_id}/events/{event_id}`
Request: `ProtobufObjectRequested`. Response: `ProtobufObjectReturned`.

POST `/api/projects/{project_id}/research`
Request: `WebResearchRequested`. Response: `WebResearchReturned`, `WebResearchFailed`.

POST `/api/projects/{project_id}/markerboard/assist`
Request: `MarkerboardAssistRequested`. Response: `MarkerboardSyntaxHelpReturned`, `MarkerboardPatchPro`

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/accept`
Request: `MarkerboardPatchAccepted`. Response: `MarkerboardDraftUpdated`.

POST `/api/projects/{project_id}/markerboard/assist/{artifact_id}/reject`
Request: `MarkerboardPatchRejected`. Response: `HTTP204(nopayload)`.

GET `/api/projects`
Request: `ProjectListRequested`. Response: `ProjectListReturned`.

POST `/api/projects`
Request: `ProjectCreateRequested`. Response: `ProjectCreated`.

GET `/api/projects/{project_id}`
Request: `ProjectOpenRequested`. Response: `ProjectOpened`.

DELETE `/api/projects/{project_id}`
Request: `ProjectDeleteRequested`. Response: `ProjectDeleted`.

POST `/api/credentials`
Request: `ProviderCredentialSubmitted`. Response: `ProviderCredentialAccepted`, `ProviderCredentialRej`

POST `/api/projects/{project_id}/conversations/{conversation_id}/messages`
Request: `ChatPromptSubmitted`. Response: `ChatResponseRendered`, `ChatTurnFailed`.

POST `/api/projects/{project_id}/markerboard/mermaid`
Request: `MermaidDraftSaved`. Response: `MermaidPreviewRendered`.

POST `/api/projects/{project_id}/markerboard/media`
Request: `MediaUploaded`. Response: `MediaRendered`.

POST `/api/projects/{project_id}/artifacts/{artifact_id}/promotions`
Request: `ArtifactPromoted`. Response: `CandidateRequirementUpdated`.

GET `/api/projects/{project_id}/artifacts`
Request: `ArtifactListRequested`. Response: `ArtifactListReturned`.

GET `/api/projects/{project_id}/artifacts/{artifact_id}/revisions/{revision}`
Request: `ArtifactOpenRequested`. Response: `ArtifactOpened`.

Messages between `LeanFMServer` and `ModelProvider` map to the bounded in-process channels `provider.requests` and `provider.results`. They retain their protobuf message identities and request identifiers even though they do not cross HTTP. This is important for concurrency: two users can use different provider credentials because the key, session, user, project, conversation, and request identities remain request-local instead of being stored in a process-wide environment variable.

12.5 Complete protobuf payload vocabulary

Every accepted event is a length-delimited `EventEnvelope`. Field 20 and above in its `payload` oneof identify the concrete message. The table below is generated from the Lean `MessageSchema` values; the checked `.proto` file supplies repeated and nested cardinality that the compact Lean field summary represents only by its scalar boundary.

`LoginSubmitted`
UserBrowser sends to LeanFMServer. Tagged fields: 1: username (string); 2: credential (bytes).

`LoginAccepted`
LeanFMServer sends to UserBrowser. Tagged fields: 1: user_id (string); 2: session_id (string).

`LoginRejected`
LeanFMServer sends to UserBrowser. Tagged fields: 1: reason (string).

`AccountCreateSubmitted`
UserBrowser sends to LeanFMServer. Tagged fields: 1: username (string); 2: login_credential (bytes); 3: provider (string); 4: api_key (bytes).

`ProviderKeyCheckRequested`
LeanFMServer sends to ModelProvider. Tagged fields: 1: provider (string); 2: verification_id (string).

`ProviderKeyVerified`
ModelProvider sends to LeanFMServer. Tagged fields: 1: verification_id (string); 2: provider_account (string).

`ProviderKeyRejected`
ModelProvider sends to LeanFMServer. Tagged fields: 1: verification_id (string); 2: reason (string).

`AccountCreated`
LeanFMServer sends to UserBrowser. Tagged fields: 1: user_id (string); 2: credential_ref (string).

`AccountCreateRejected`
LeanFMServer sends to UserBrowser. Tagged fields: 1: reason (string).

`TaskModelRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: task_id (string); 3: requirements_revision (string).

`TaskModelReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: task_id (string); 2: interaction_artifact_id (string); 3: state_machine_artifact_id (string); 4: message_sequence_artifact_id (string); 5: constraint_ids (string).

`MessageSequenceRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: task_id (string); 3: session_id (string).

`MessageSequenceReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: task_id (string); 2: session_id (string); 3: event_ids (string).

`ProtobufObjectRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: event_id (string).

`ProtobufObjectReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: event_id (string); 2: protobuf_type (string); 3: encoded_object (bytes); 4: redacted_json (string).

`WebResearchRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: conversation_id (string); 3: request_id (string); 4: question (string); 5: supplied_urls (string).

`WebResearchDispatched`
LeanFMServer sends to ModelProvider. Tagged fields: 1: request_id (string); 2: model (string); 3: research_mode (string).

`WebSearchPerformed`
ModelProvider sends to LeanFMServer. Tagged fields: 1: request_id (string); 2: queries (string).

`WebPageOpened`
ModelProvider sends to LeanFMServer. Tagged fields: 1: request_id (string); 2: url (string); 3: title (string).

`WebResearchReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: request_id (string); 2: parts (bytes); 3: source_urls (string).

`WebResearchFailed`
LeanFMServer sends to UserBrowser. Tagged fields: 1: request_id (string); 2: reason (string).

`MarkerboardAssistRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: artifact_id (string); 3: base_revision (uint64); 4: diagram_kind (string); 5: source (string); 6: instruction (string); 7: mode (string); 8: credential_ref (string).

`MarkerboardSyntaxHelpReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifact_id (string); 2: base_revision (uint64); 3: help_markdown (string).

`MarkerboardAssistDispatched`
LeanFMServer sends to ModelProvider. Tagged fields: 1: request_id (string); 2: artifact_id (string); 3: base_revision (uint64); 4: diagram_kind (string); 5: source (string); 6: instruction (string).

`MarkerboardAssistReturned`
ModelProvider sends to LeanFMServer. Tagged fields: 1: request_id (string); 2: proposed_source (string); 3: explanation (string); 4: input_tokens (uint64); 5: output_tokens (uint64).

`MarkerboardPatchProposed`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifact_id (string); 2: base_revision (uint64); 3: proposed_source (string); 4: explanation (string).

`MarkerboardPatchAccepted`
UserBrowser sends to LeanFMServer. Tagged fields: 1: artifact_id (string); 2: base_revision (uint64); 3: proposed_source (string).

`MarkerboardPatchRejected`
UserBrowser sends to LeanFMServer. Tagged fields: 1: artifact_id (string); 2: base_revision (uint64).

`MarkerboardDraftUpdated`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifact_id (string); 2: revision (uint64); 3: source (string).

`ProjectListRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: user_id (string).

`ProjectListReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: project_ids (string).

`ProjectCreateRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: display_name (string).

`ProjectCreated`
LeanFMServer sends to UserBrowser. Tagged fields: 1: project_id (string); 2: display_name (string).

`ProjectOpenRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string).

`ProjectOpened`
LeanFMServer sends to UserBrowser. Tagged fields: 1: project_id (string); 2: current_requirements_revision (string).

`ProjectDeleteRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: confirmation (string).

`ProjectDeleted`
LeanFMServer sends to UserBrowser. Tagged fields: 1: project_id (string).

`ProviderCredentialSubmitted`
UserBrowser sends to LeanFMServer. Tagged fields: 1: provider (string); 2: api_key (bytes); 3: retention (string).

`ProviderCredentialAccepted`
LeanFMServer sends to UserBrowser. Tagged fields: 1: credential_ref (string); 2: billing_source (string).

`ProviderCredentialRejected`
LeanFMServer sends to UserBrowser. Tagged fields: 1: reason (string).

`ChatPromptSubmitted`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: conversation_id (string); 3: message_id (string); 4: parts (bytes); 5: attachment_ids (string); 6: credential_ref (string).

`ModelRequestIssued`
LeanFMServer sends to ModelProvider. Tagged fields: 1: project_id (string); 2: conversation_id (string); 3: request_id (string); 4: model (string); 5: research_mode (string).

`ModelResponseReturned`
ModelProvider sends to LeanFMServer. Tagged fields: 1: request_id (string); 2: parts (bytes); 3: input_tokens (uint64); 4: cached_input_tokens (uint64); 5: output_tokens (uint64); 6: context_window_tokens (uint64).

`ChatResponseRendered`
LeanFMServer sends to UserBrowser. Tagged fields: 1: conversation_id (string); 2: message_id (string); 3: parts (bytes); 4: artifact_ids (string); 5: input_tokens (uint64); 6: cached_input_tokens (uint64); 7: output_tokens (uint64); 8: context_window_tokens (uint64); 9: usage_warning (string).

`ChatTurnFailed`
LeanFMServer sends to UserBrowser. Tagged fields: 1: request_id (string); 2: reason (string).

`MermaidDraftSaved`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: artifact_id (string); 3: revision (uint64); 4: diagram_kind (string); 5: source (string).

`MermaidPreviewRendered`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifact_id (string); 2: revision (uint64); 3: svg (bytes).

`MediaUploaded`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: artifact_id (string); 3: media_type (string); 4: content (bytes).

`MediaRendered`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifact_id (string); 2: media_type (string); 3: content_url (string).

`ArtifactPromoted`
UserBrowser sends to LeanFMServer. Tagged fields: 1: artifact_id (string); 2: revision (uint64); 3: target_status (string).

`CandidateRequirementUpdated`
LeanFMServer sends to UserBrowser. Tagged fields: 1: requirements_revision (string); 2: source_artifact_ids (string).

`ArtifactListRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: project_id (string); 2: kind_filter (string); 3: scenario_filter (string); 4: status_filter (string).

`ArtifactListReturned`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifacts (bytes).

`ArtifactOpenRequested`
UserBrowser sends to LeanFMServer. Tagged fields: 1: artifact_id (string); 2: revision (uint64).

`ArtifactOpened`
LeanFMServer sends to UserBrowser. Tagged fields: 1: artifact_id (string); 2: revision (uint64); 3: status (string); 4: media_type (string); 5: content (bytes); 6: provenance (string).

The envelope backpointers are semantically significant. `prior_ids` contains immutable predecessor event identifiers. A derived `map[Event][]Event` successor index may accelerate traversal, but it is not serialized as mutable truth. `session_id` separates users and concurrent conversations; `task_id` selects the grammar; and `src`/`dst` must agree with the selected message schema.

12.6 Complete LeanFM task grammar

The following is the whole accepted ChatUI conversation grammar, rendered from the typed `TaskGrammar` values. Pronounce `A ! B : Message` as “A sends Message to B.” `then` is ordered composition; `choose/or` is an explicit alternative; `optional` permits zero or one occurrence; `fork/join all` waits for every branch; and `race/join first` accepts the first completion and cancels the remaining branches. A message not admitted by the selected task’s current residual grammar is illegal even if its protobuf bytes decode.

12.7 Authenticate an individual user

Task: `authenticate_user`. Entry: `signed_out`; terminals: `authenticated, rejected`.

```
UserBrowser ! LeanFMServer : LoginSubmitted
then
choose
  LeanFMServer ! UserBrowser : LoginAccepted
```

```
or
  LeanFMServer ! UserBrowser : LoginRejected
```

12.8 Create an account using a user-supplied provider API key

Task: `create_account`. Entry: `registration`; terminals: `created, rejected`.

```
UserBrowser ! LeanFMServer : AccountCreateSubmitted
then
LeanFMServer ! ModelProvider : ProviderKeyCheckRequested
then
choose
  ModelProvider ! LeanFMServer : ProviderKeyVerified
  then
  LeanFMServer ! UserBrowser : AccountCreated
or
  ModelProvider ! LeanFMServer : ProviderKeyRejected
  then
  LeanFMServer ! UserBrowser : AccountCreateRejected
```

12.9 List, create, open, and delete owned projects

Task: `manage_projects`. Entry: `ready`; terminals: `listed, created, opened, deleted`.

```
choose
  UserBrowser ! LeanFMServer : ProjectListRequested
  then
  LeanFMServer ! UserBrowser : ProjectListReturned
or
  UserBrowser ! LeanFMServer : ProjectCreateRequested
  then
  LeanFMServer ! UserBrowser : ProjectCreated
or
  UserBrowser ! LeanFMServer : ProjectOpenRequested
  then
  LeanFMServer ! UserBrowser : ProjectOpened
or
  UserBrowser ! LeanFMServer : ProjectDeleteRequested
  then
  LeanFMServer ! UserBrowser : ProjectDeleted
```

12.10 Select user-funded or platform-funded model access

Task: `configure_provider`. Entry: `unconfigured`; terminals: `configured, rejected`.

```
UserBrowser ! LeanFMServer : ProviderCredentialSubmitted
then
choose
  LeanFMServer ! UserBrowser : ProviderCredentialAccepted
or
  LeanFMServer ! UserBrowser : ProviderCredentialRejected
```

12.11 Conduct one durable project-scoped requirements turn

Task: `argue_requirements`. Entry: `ready`; terminals: `rendered, failed`.

```
UserBrowser ! LeanFMServer : ChatPromptSubmitted
then
LeanFMServer ! ModelProvider : ModelRequestIssued
then
choose
  ModelProvider ! LeanFMServer : ModelResponseReturned
  then
  LeanFMServer ! UserBrowser : ChatResponseRendered
or
  LeanFMServer ! UserBrowser : ChatTurnFailed
```

12.12 Sketch and promote multimodal requirement artifacts

Task: `markerboard`. Entry: `editing`; terminals: `candidate`.

```
choose
  UserBrowser ! LeanFMServer : MermaidDraftSaved
  then
  LeanFMServer ! UserBrowser : MermaidPreviewRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
or
  UserBrowser ! LeanFMServer : MediaUploaded
  then
  LeanFMServer ! UserBrowser : MediaRendered
  then
  UserBrowser ! LeanFMServer : ArtifactPromoted
  then
  LeanFMServer ! UserBrowser : CandidateRequirementUpdated
```

12.13 Inspect current, accepted, superseded, and stale artifacts

Task: `inspect_artifacts`. Entry: `closed`; terminals: `listed, opened`.

```
UserBrowser ! LeanFMServer : ArtifactListRequested
then
LeanFMServer ! UserBrowser : ArtifactListReturned
then
choose
  empty
or
  UserBrowser ! LeanFMServer : ArtifactOpenRequested
  then
  LeanFMServer ! UserBrowser : ArtifactOpened
```

12.14 Inspect one task's linked interaction, state machine, constraints, and protobuf events

Task: `inspect_task_model`. Entry: `closed`; terminals: `linked, sequence_visible, object_visible`.

```
UserBrowser ! LeanFMServer : TaskModelRequested
then
LeanFMServer ! UserBrowser : TaskModelReturned
then
choose
  empty
or
  UserBrowser ! LeanFMServer : MessageSequenceRequested
  then
  LeanFMServer ! UserBrowser : MessageSequenceReturned
  then
  choose
    empty
  or
    UserBrowser ! LeanFMServer : ProtobufObjectRequested
    then
    LeanFMServer ! UserBrowser : ProtobufObjectReturned
```

12.15 Search the public web, follow supplied HTTP URLs, and return clickable citations

Task: `research_web`. Entry: `ready`; terminals: `returned, failed`.

```
UserBrowser ! LeanFMServer : WebResearchRequested
then
LeanFMServer ! ModelProvider : WebResearchDispatched
then
choose
  ModelProvider ! LeanFMServer : WebSearchPerformed
```

```
  then
  ModelProvider ! LeanFMServer : WebPageOpened
  then
  LeanFMServer ! UserBrowser : WebResearchReturned
or
  LeanFMServer ! UserBrowser : WebResearchFailed
```

12.16 Recall diagram syntax or propose AI-generated detail for one markerboard revision

Task: `assist_markerboard`. Entry: `editing`; terminals: `helped, updated, rejected`.

```
UserBrowser ! LeanFMServer : MarkerboardAssistRequested
then
choose
  LeanFMServer ! UserBrowser : MarkerboardSyntaxHelpReturned
or
  LeanFMServer ! ModelProvider : MarkerboardAssistDispatched
  then
  ModelProvider ! LeanFMServer : MarkerboardAssistReturned
  then
  LeanFMServer ! UserBrowser : MarkerboardPatchProposed
  then
  choose
    UserBrowser ! LeanFMServer : MarkerboardPatchAccepted
    then
    LeanFMServer ! UserBrowser : MarkerboardDraftUpdated
  or
    UserBrowser ! LeanFMServer : MarkerboardPatchRejected
```

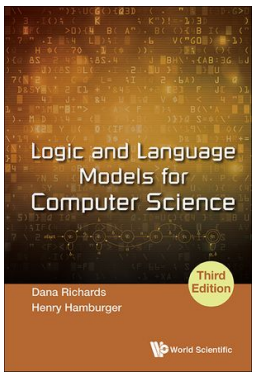
12.17 What is checked, and what remains

Lean elaboration checks the actors, message schemas, task states, transitions, grammars, constraints, desired-output provenance, and implementation mappings. The chapter generator rejects an accepted HTTP request with no named response mapping and rejects incomplete running-route records. The ordinary repository checks also validate the protobuf source against the typed requirements.

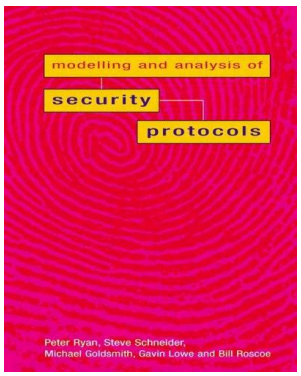
This does not yet prove that every accepted route has a handler: the running table makes that gap explicit. Nor does the current server yet append every browser exchange to the specified binary event journal. Completing those handlers and the schema-specific codec remains implementation work. Until then, the accepted grammar is the protocol contract, the running table is the honest conformance statement, and neither may be substituted for the other.

Referenced Books

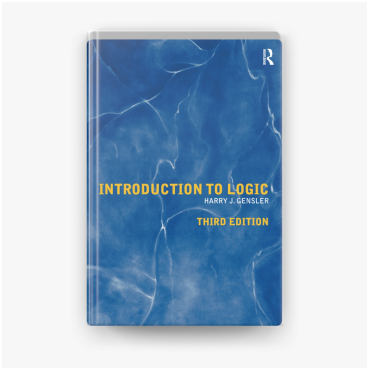
These cover snapshots identify the editions of the books referenced in this volume. They are reproduced at small size for bibliographic identification.



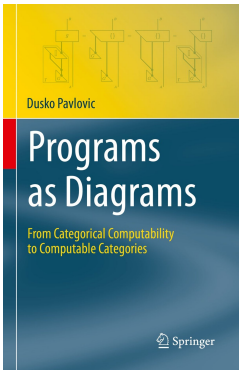
Logic and Language Models for Computer Science
Dana Richards and Henry Hamburger [11]



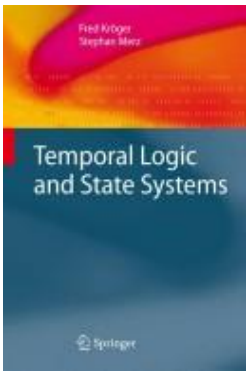
The Modelling and Analysis of Security Protocols
Peter Ryan, Steve Schneider, Michael Goldsmith, Gavin Lowe, and Bill Roscoe [12]



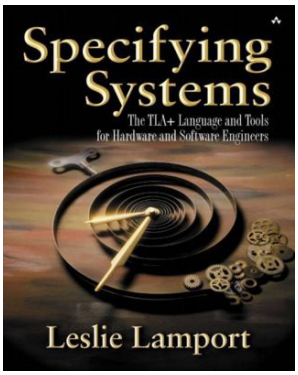
Introduction to Logic
Harry J. Gensler [3]



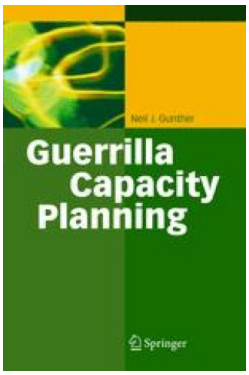
Programs as Diagrams
Dusko Pavlovic [10]



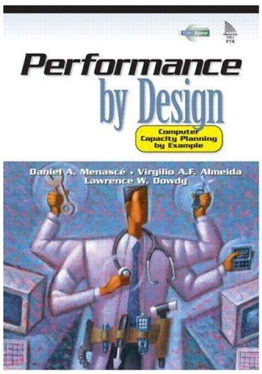
Temporal Logic and State Systems
Fred Kröger and Stephan Merz [5]



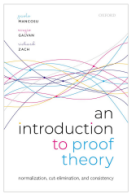
Specifying Systems
Leslie Lamport [6]



Guerrilla Capacity Planning
Neil J. Gunther [4]



Performance by Design
Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy [9]



An Introduction to Proof Theory
Paolo Mancosu, Sergio Galvan, and Richard Zach [8]

Cover-image sources, accessed September 25 and 30, 2026: World Scientific edition via Kobo for Richards and Hamburger; Open Library for Ryan et al.; Apple Books for Gensler; and Kobo's Springer edition for Pavlovic. The Kröger and Merz cover snapshot is from Open Library. The Lamport and Menascé–Almeida–Dowdy snapshots are from Open Library; the Gunther snapshot is from Springer Nature; and the Mancosu–Galvan–Zach snapshot is from Google Books. Cover artwork remains the property of its respective publisher.

Bibliography

[1] Edmund M. Clarke, E. Allen Emerson, and A. Prasad Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems*, 8(2):244–263, 1986. doi: 10.1145/5397.5399.

[2] Robert Fielding and LeanFM contributors. LeanFM: Grammar-based formal methods in Lean. GitHub repository, 2026. URL `https://github.com/rfielding/leanFM`. Accessed 2026-09-21.

[3] Harry J. Gensler. *Introduction to Logic*. Routledge, 3 edition, 2017. ISBN 978-1-138-91058-4. doi: 10.4324/9781315693361.

[4] Neil J. Gunther. *Guerrilla Capacity Planning: A Tactical Approach to Planning for Highly Scalable Applications and Services*. Springer, Berlin, Heidelberg, 2007. ISBN 978-3-540-31010-5. doi: 10.1007/978-3-540-31010-5.

[5] Fred Kröger and Stephan Merz. *Temporal Logic and State Systems*. Texts in Theoretical Computer Science. An EATCS Series. Springer, Berlin, Heidelberg, 2008. ISBN 978-3-540-68635-4. doi: 10.1007/978-3-540-68635-4.

[6] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. ISBN 978-0-321-14306-8.

[7] LeanFM. Kerberos: Time, knowledge, and obligation. YouTube video, 2026. Illustration of the proposed multimodal semantics, not an implemented general checker. `https://www.youtube.com/watch?v=f2efwxq8BaY`.

[8] Paolo Mancosu, Sergio Galvan, and Richard Zach. *An Introduction to Proof Theory: Normalization, Cut-Elimination, and Consistency Proofs*. Oxford University Press, Oxford, 2021. ISBN 978-0-19-289593-6. doi: 10.1093/oso/9780192895936.001.0001.

[9] Daniel A. Menascé, Virgilio A. F. Almeida, and Lawrence W. Dowdy. *Performance by Design: Computer Capacity Planning by Example*. Prentice Hall PTR, Upper Saddle River, NJ, 2004. ISBN 978-0-13-090673-1.

[10] Dusko Pavlovic. *Programs as Diagrams: From Categorical Computability to Computable Categories*. Springer Nature Switzerland, Cham, Switzerland, 2023. ISBN 978-3-031-34826-6. doi: 10.1007/978-3-031-34827-3.

[11] Dana Richards and Henry Hamburger. *Logic and Language Models for Computer Science*. World Scientific, 3 edition, 2017. ISBN 978-981-322-920-4. doi: 10.1142/10507.

[12] Peter Y. A. Ryan, Steve A. Schneider, Michael H. Goldsmith, Gavin Lowe, and A. W. Roscoe. *The Modelling and Analysis of Security Protocols: The CSP Approach*. Addison-Wesley, Harlow, England, 2001. ISBN 978-0-201-67471-2.

Index

agreement, 93
alphabet
  vague, 26

bytes, 90

chart specification, 45
Codex, 49
composition
  requirement intersection, 44
CSP, 89
CTL, 72

event, 8

formal methods, 2

grammar
  context-free, 20
  multiparty, 20

HTTP
  protobuf adapter, 97

intruder model, 92

knowledge
  derived from events, 67

Lean, 81
LTL, 72

modal logic, 72
Model Context Protocol, 50

Needham–Schroeder protocol, 94

OAuth, 57
observable event, 25
Otway–Rees protocol, 93

parameter flow, 23
past-time logic, 80
probability, 28
profit
  derived from events, 14
project
  directory boundary, 52
protocol
  LeanFM web server, 97
protocol role, 89

requirements
  durable, 48

scenario, 25
secrecy, 93

temporal logic, 72
time, 12
trace, 8

vibecoding, 48

waste
  derived from events, 14

XY plot, 14

Yahalom protocol, 90